计算机组成原理与系统结构



第7章 控制器

杭州電子科技大學
HANGZHOU DIANZI UNIVERSITY

第7章 控制器

7.1 控制器概述

7.2 Yy-z02模型机的系统结构与数据通路

7.3 硬布线控制器

7.4 微程序控制器

本章小结

7.1控制器概述

- 7.1.1 计算机硬件组成与CPU基本功能
- 7.1.2 控制器的功能与组成
- 7.1.3 时序系统及其控制
- 7.1.4 单周期、多周期与流水线实现
- 7.1.5 控制器的设计步骤



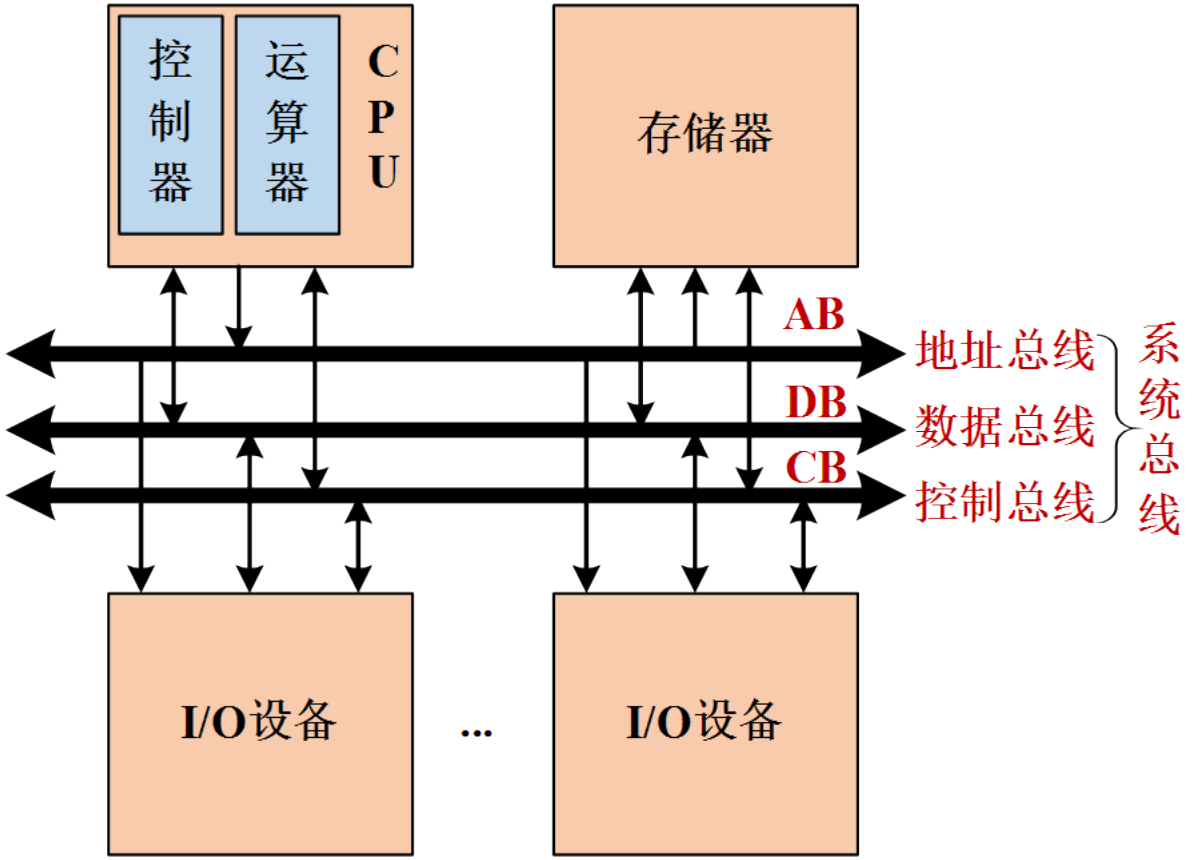
7.1.1 计算机硬件组成与CPU基本功能

- 1、计算机硬件组成结构
- 2、存储器结构
- 3、中央处理器CPU的基本功能
- 4、处理器类型

1、计算机硬件组成结构

❖ 冯•诺依曼体系结构计算机

- **五大部件：控制器、运算器、存储器、输入设备、输出设备**
- **三个模块：**
  - **I/O设备：完成计算机和外部世界之间的指令及数据的交换**
  - **存储器：存放指令和数据**
  - **CPU：执行指令，完成数据的处理**



❖ 总线

- **一组导线：在部件之间建立连接**
- **数据传输：源部件→总线→目的部件**

❖ 总线构成：

- **AB：CPU或总线主设备→AB→存储器或IO设备(单向)**
- **DB：各部件之间（双向）**
- **CB：包含许多不同的控制信号线和状态信号线(单/双向)**

2、存储器结构

❖ （1）普林斯顿结构：冯·诺依曼体系结构

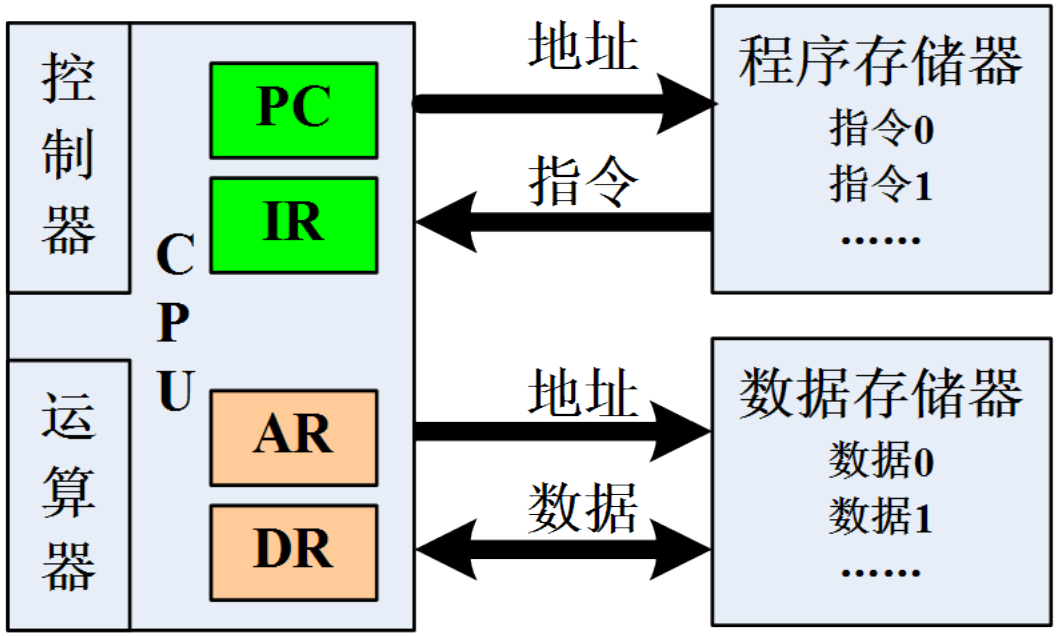
❖ 普林斯顿结构特征

- 程序和数据共同存储在同一个存储器
- 一套地址总线与数据总线
- 存储器瓶颈

❖ 普林斯顿结构的CPU

- Intel微处理器
- ARM7
- MIPS处理器

❖ （2）哈佛结构



❖ 哈佛结构特征

- 程序和数据分开存储：两个存储器
- 两套地址总线与数据总线
- 指令和数据宽度可以不同

❖ 典型的哈佛结构的CPU

- PIC系列、MC68系列、Z8系列、AVR系列
- ARM9、ARM10和ARM11
- 51单片机：改进型哈佛结构（指令和数据存储器分时复用同一条总线）

2、存储器结构

- 普林斯顿结构
- 哈佛结构

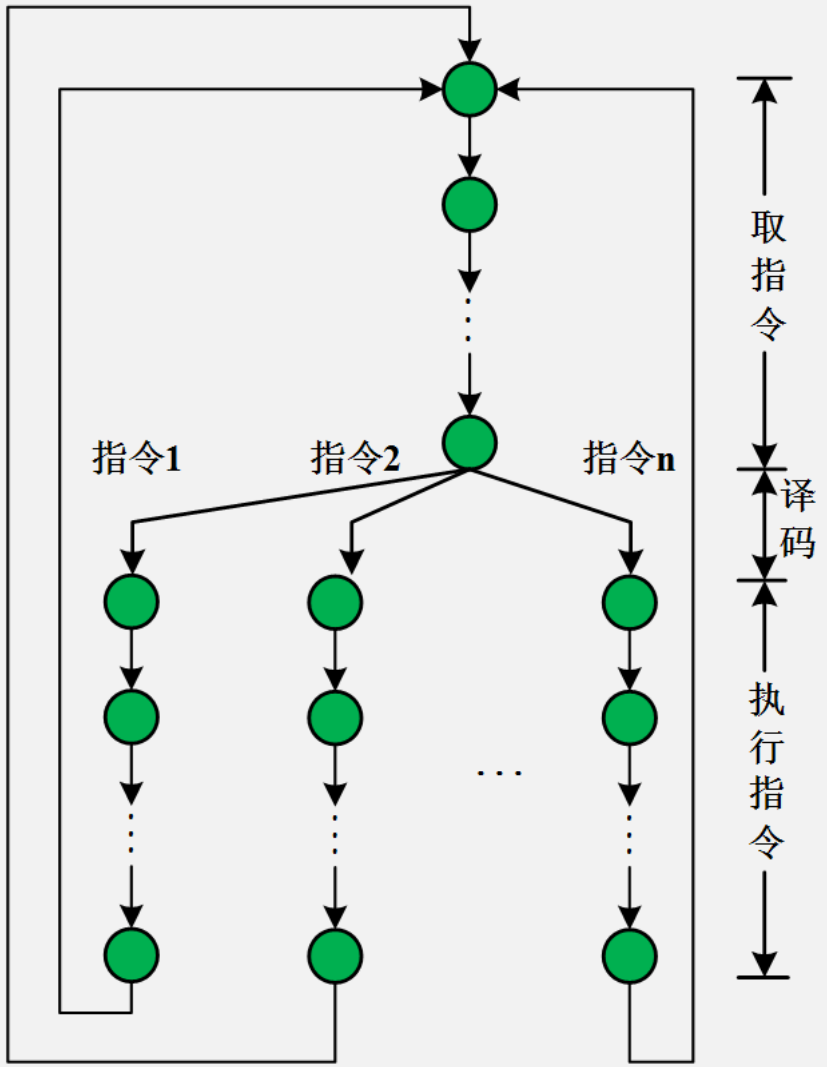
- 现代很多CPU：改进型哈佛结构
  - 外部：冯•诺依曼结构，单总线
  - 内部：采用分离Cache，即数据和指令Cache
  - 两者有效结合
- 另一种改进型哈佛结构：指令和数据存储器两个，但是共用一个存储空间
- RISC-V?
- 未明确说明

- 本课程研究对象：冯•诺依曼体系结构的SISD的电子数字计算机
- 实现RISC-V CPU的实验：采用哈佛结构

3、中央处理器CPU的基本功能

- 指令控制：确保计算机指令按程序的顺序执行
- 操作控制：控制微操作的产生、组合、传送和管理
- 时间控制：严格按照时间序列执行微操作和指令
- 数据加工：算术运算和逻辑运算

- CPU：实际上就是一个复杂的有限状态机
- 设计CPU的过程：确定三个内容
  - CPU的所有状态
  - 状态对应执行的微操作
  - 状态之间的转移控制（定序）
- 确定的依据：指令系统、系统结构



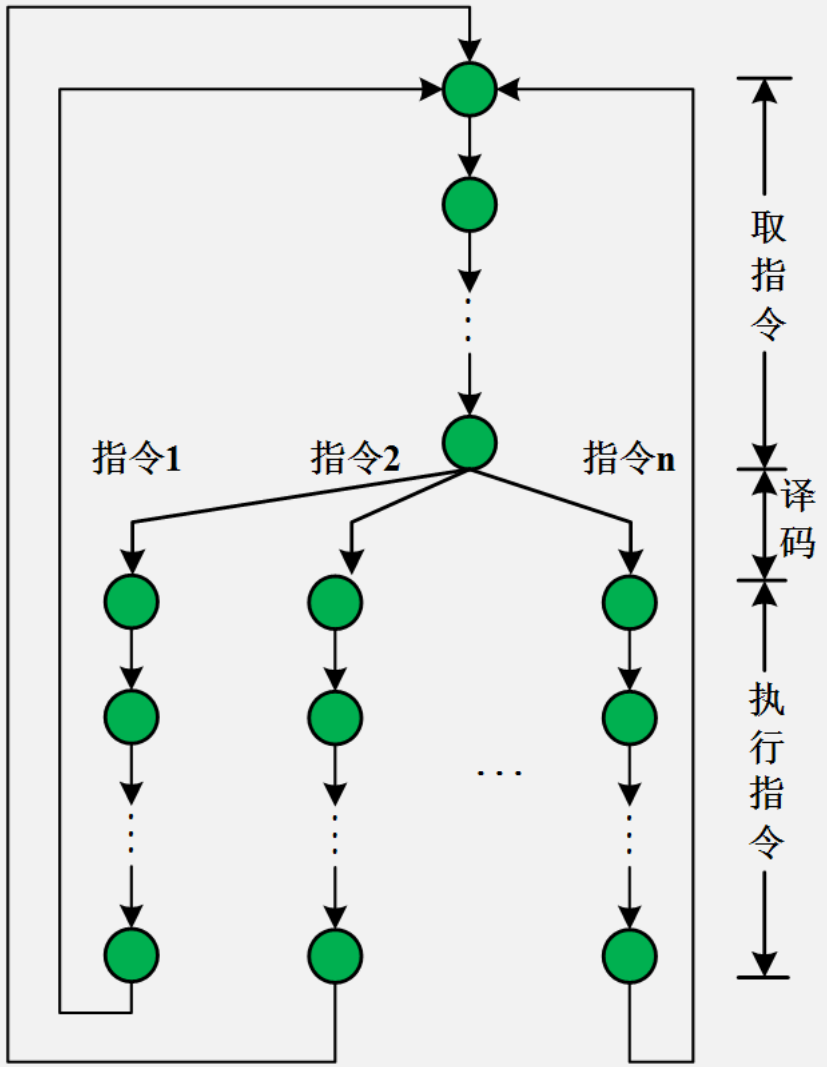
3、中央处理器CPU的基本功能

❖指令的执行可分为：三个阶段

- 取指令阶段：从存储器中取出一条指令送到CPU
- 指令译码阶段：对当前指令进行译码，分析是哪条指令、操作数的寻址方式等
- 执行阶段：执行该指令，控制相应部件完成指令的操作

❖译码阶段：

- 可能并不对应任何状态：只是取指令结束后到各条指令的执行周期之间的一个多路选择
- 可能包含好几个状态：流水线中进行各字段的逐步译码



4、处理器类型

- **中央处理器CPU**（Central Processing Unit）：运算器+控制器，泛指计算机系统的运算与控制核心
- **微处理器MPU**（Micro Processor Unit）：CPU + MMU + Cache，泛指微型机的CPU
- **微控制器MCU**（Micro Control Unit）：CPU+RAM +ROM+I/O接口，泛指单片机或者嵌入式处理器
- **图形处理器GPU**（Graphics Processing Unit）：专门执行图像图形相关运算工作的处理器，俗称显示芯片，+显存→显卡
- **加速处理器APU**（Accelerated Processing Unit）：AMD设计，高性能CPU+GPU
- **张量处理器TPU**（ Tensor Processing Unit ）：Google的AI芯片

7.1.2 控制器的功能与组成



1、控制器的功能

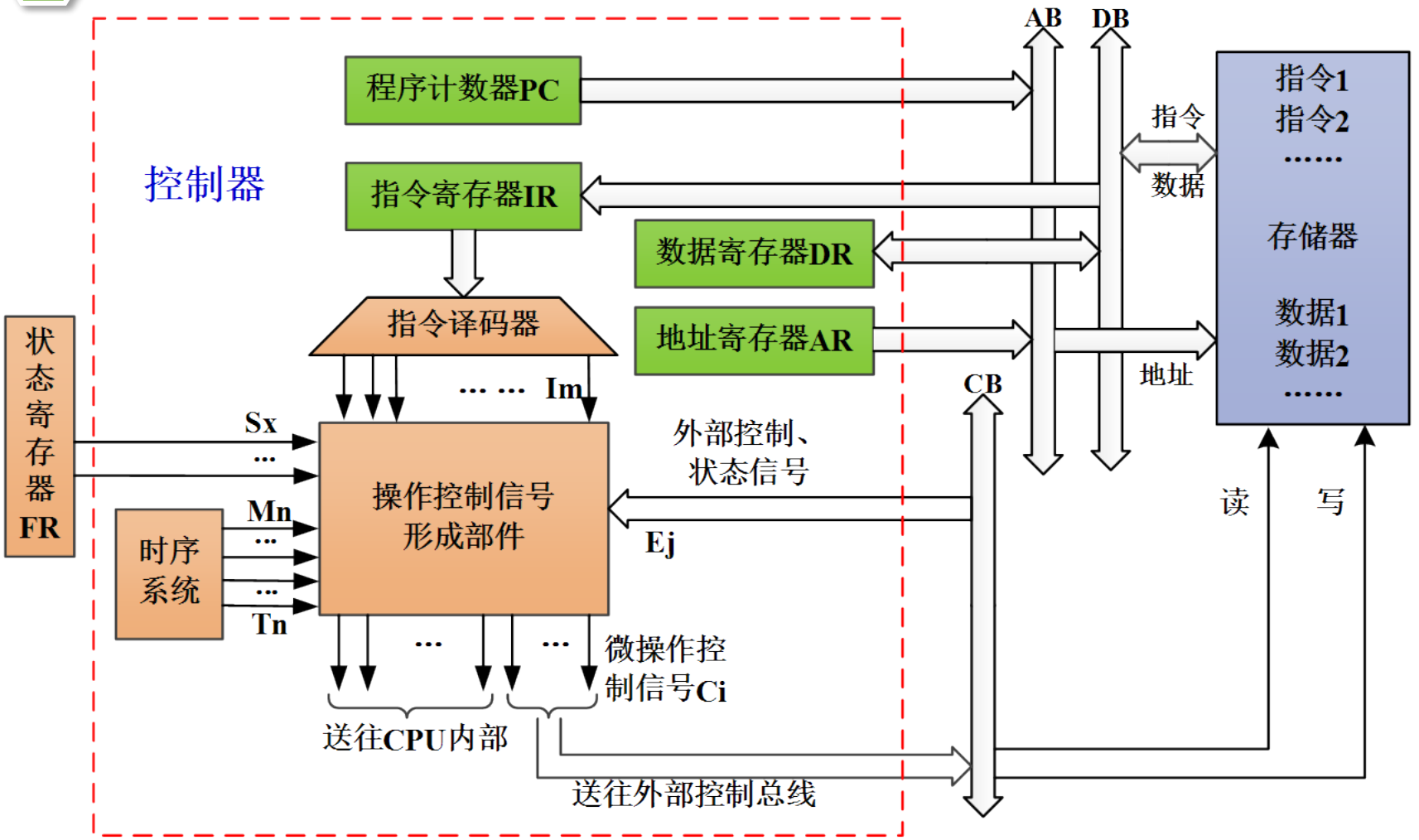
2、控制器的组成

1、控制器的功能

- **总体功能：** 从存储器中取指令、对指令译码、产生控制信号并控制计算机系统各部件有序地执行，从而实现这条指令的功能。
- **具体功能：**
  - ①取指令：Mem[PC]→IR，PC自增
  - ②分析指令：IR内容送指令译码器，进行操作码(OP)和寻址方式码(MOD)译码，识别机器指令及其寻址方式
  - ③执行指令： 控制各个部件计算地址、取操作数、运算、传送或者写回等
  - ④中断处理和响应特殊请求：处理CPU内部和外部的随机事件，包括中断请求和I/O系统的访问请求



2、控制器的组成



- （1）专用寄存器
  - PC、IR、AR、DR
- （2）指令译码器
- （3）时序系统
- （4）操作控制信号形成部件

2、控制器的组成

❖ （1）专用寄存器：

① 程序计数器（PC）：存放指令地址

- 顺序执行时，由PC+1 → PC产生下一条指令的地址
- 遇到转移指令时，转移地址→PC作为下一条指令的地址

② 指令寄存器（IR）：存放指令编码

- IR扩充为指令队列，或称指令栈，允许预取多条指令，以支持指令流水线。

③ 地址寄存器（AR）：用于存放CPU访问存储器或者I/O设备的地址码

④ 数据寄存器（DR）：用于存放CPU访问存储器或者I/O设备的数据

❖ AR和DR：总线接口单元BIU（Bus Interface Unit）的重要组成部分，CPU与存储器或者I/O系统的一个总线接口

2、控制器的组成

- （2）指令译码器ID：对指令操作码进行译码，以识别该指令所要求的操作
- （3）时序系统：负责提供时钟基准和各级时序信号，规定每个操作的时间
- （4）操作控制信号形成部件：产生指令所需的各种操作控制信号，以便正确地建立数据通路，完成取指令、分析指令和执行指令的控制。
  - 产生的依据：指令译码器ID输出的指令信号、时序系统产生的时序信号、标志寄存器的状态
  - 根据该部件设计方法及组成的不同，控制器分为：
    - 硬布线控制器：主要由组合逻辑电路构成，组合逻辑或常规逻辑控制器
    - 微程序控制器：主要由存储逻辑电路构成
  - 是控制器的核心部件，或称控制单元CU

7.1.3 时序系统及其控制

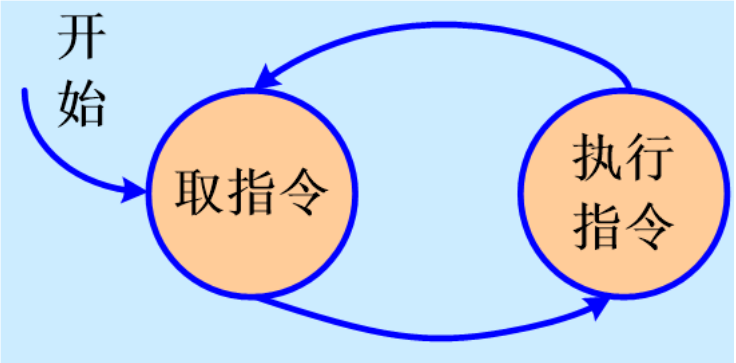
- 1、指令周期
- 2、时序信号
- 3、时序控制方式
- 4、时序部件

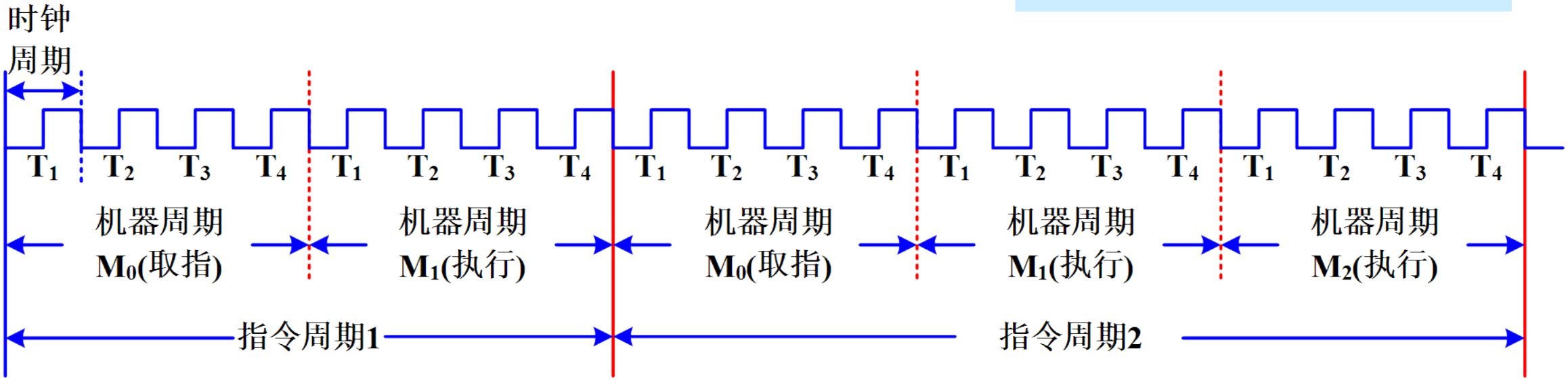
1、指令周期

- **指令周期：指计算机从内存取出一条指令并完成该指令的执行所需要的时间。**
  - **不同指令的指令周期是不相同的。**
  - **一个指令周期可能由若干个机器周期组成。**
- **机器周期：又称为CPU周期，用于完成1次内存的读或写操作，或者1次ALU的运算，或者1次总线传送**
  - **一般规定为CPU与内存交换1次信息（读或写内存）所需要的时间。**
  - **一个机器周期可能包含多个时钟周期。**
- **时钟周期：节拍/T周期，是指CPU执行一个微操作的最小时间单位，一般就是CPU主频脉冲的周期。**

1、指令周期

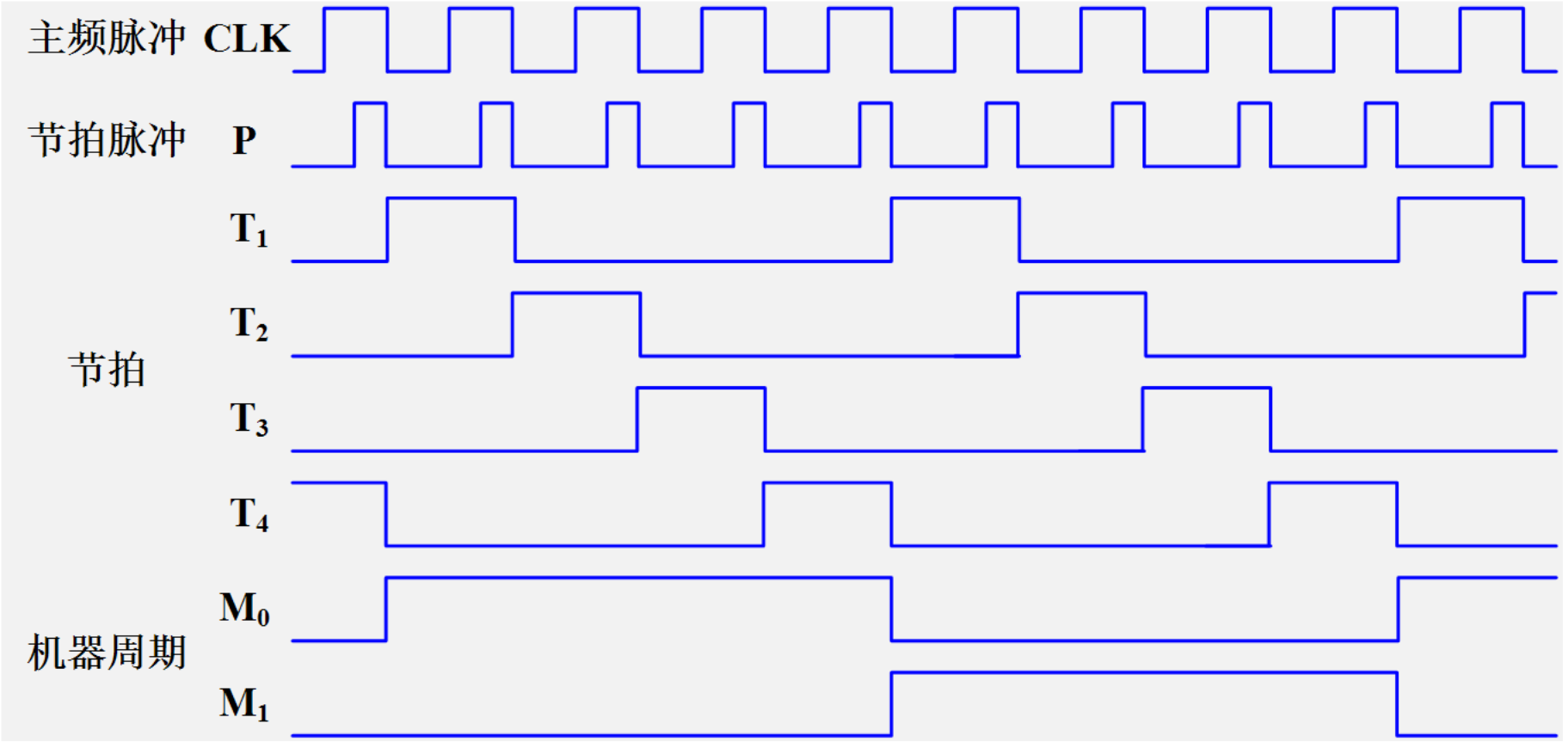
- 指令周期、机器周期、时钟周期的关系（多周期CPU）
- 指令1：含M_0、M_1两个机器周期
- 指令2：含M_0、M_1、M_2三个机器周期
- 每个机器周期：含4个时钟周期，即T_1~T_4节拍





2、时序信号

- 三级时序信号：
  - M：机器周期/CPU周期
  - T：节拍
  - P：节拍脉冲/工作脉冲，每个节拍1-2个，便于微操作的顺序控制



2、时序信号

- 如何使用三级时序信号来定序?
  - 控制信号需要与三级时序信号进行“与”操作，完成定序
- 8086 CPU：MOV指令，读存储器操作→1个机器周期
  - 假设是M_1周期，将微操作分配到4个节拍：
    - T_1节拍：地址锁存操作，控制信号ALE
    - T_2节拍：启动存储器读操作，控制信号$\overline{RD}$
    - T_3节拍：检测读操作是否完成
    - T_4节拍：锁存读出的数据到DR，控制信号clk_DR

- $ALE = MOV \cdot M_1 \cdot T_1$
- $\overline{RD} = \overline{MOV \cdot M_1 \cdot T_2}$
- $clk_DR = MOV \cdot M_1 \cdot T_4 \cdot P$

- MOV：由指令译码器ID译码输出的指令信号，指示当前指令是MOV指令



3、时序控制方式

- **定义：如何定时及同步不同的控制信号序列，并将信号序列衔接起来，从而保证计算机各部件有节奏地依次执行规定的各种操作。**
- **讨论的问题：一条指令有几个机器周期？每个机器周期又有几个节拍?**
- **例如：**
  - **ADD [1000H],AX**
    - **读主存Mem[1000H]**
    - **ALU执行加法**
    - **写主存Mem[1000H]**
  - **ADD BX，AX**
    - **ALU执行加法**
- **三种时序的控制方式**

3、时序控制方式

① 同步控制方式：又称为固定时序控制方式或无应答控制方式。

- 每条指令：相同的机器周期数、相同的节拍，执行时间都是相同的
- 以微操作序列最长的指令为标准，确定机器周期数和节拍数。
- 时序系统产生统一的、顺序固定的、周而复始的机器周期和节拍信号
- 例如：所有指令的指令周期都包含4个机器周期、每个机器周期都包含4个节拍。
- 优点：电路简单；
- 缺点：运行效率低、速度慢。

3、时序控制方式

② 异步控制方式：又称可变时序控制方式或应答控制方式

- 没有严格的时序信号对微操作进行同步
- 每条指令需要多少节拍，就产生多少节拍；当指令执行完毕，发出应答信号；时序部件收到应答信号后，才开始下条指令的执行
- 执行不同指令所需的时间完全由实际需要确定，不尽相同。
- 优点：指令的运行效率高；
- 缺点：控制器的电路比较复杂。
- 异步控制方式比较好地协调了主从设备速度不一致问题，在计算机中得到广泛的应用。
- 例如CPU对内存的读写；I/O设备与内存的数据交换等

3、时序控制方式

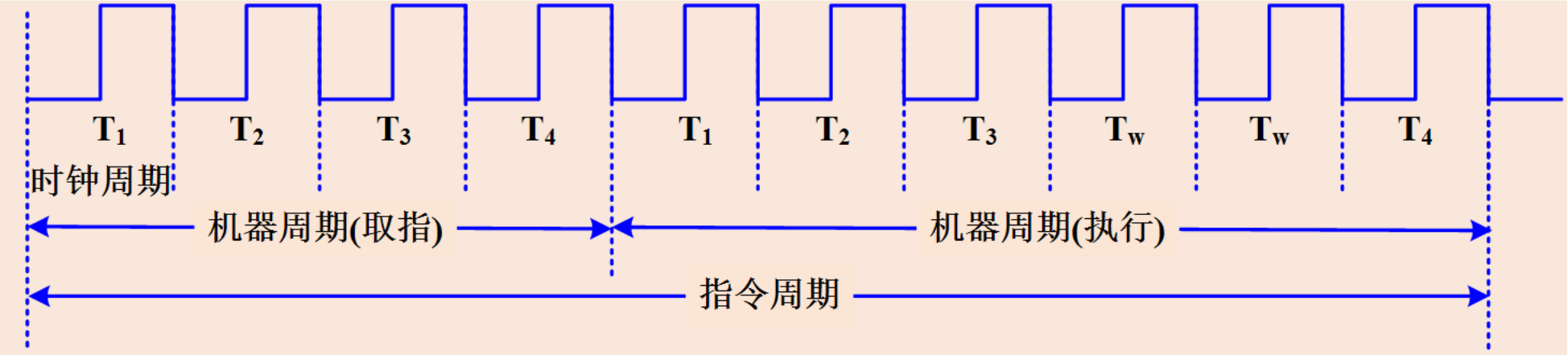
③ 联合控制方式：把同步控制方式和异步控制方式结合使用

- 例1：
  - 同步控制：大部分指令安排在统一的机器周期内完成
  - 异步控制：少数特殊指令、微操作序列过长或过短、微操作时间难以确定
- 例2：
  - 每个机器周期的节拍数统一
  - 指令所含的机器周期数可以根据需要设计为不同。
- 现代计算机系统大多采用联合控制方式，一般设计思想是：在功能部件内部采用同步控制方式，而在功能部件之间采用异步控制方式。
  - CPU内部：采用同步控制方式，按多数指令的需要设置节拍数
  - CPU与主存、I/O设备交换数据：采用异步控制方式，通过插入节拍或延长节拍来满足这些操作的需求

3、时序控制方式

③ 联合控制方式：常用方法

- **时钟周期插入：大多数机器周期采用相同的时钟周期数，对于复杂操作的机器周期，则插入等待时钟周期。**
- **如：8086CPU的机器周期数可变**
  - T_2节拍的后沿：检测READY信号
    - =1：表明访问就绪，后续节拍是T_3、T_4，为正常机器周期
    - =0：表明访问未就绪，后续节拍是T_3、T_W，即插入等待节拍T_W
    - 继续在T_3、T_W的后沿检测READY，直到=1才进入T_4节拍结束机器周期



3、时序控制方式

- 【例7.1】某计算机主频100MHz，平均每条指令包含3个机器周期，每个机器周期包含2个节拍
- 请计算：
- （1）该机的CPI是多少？执行速度是多少MIPS?
- （2）假设每条指令平均访存1.5次，每次访存需要**插入1个等待节拍**，请问：该机每条指令的平均执行时间是多少?

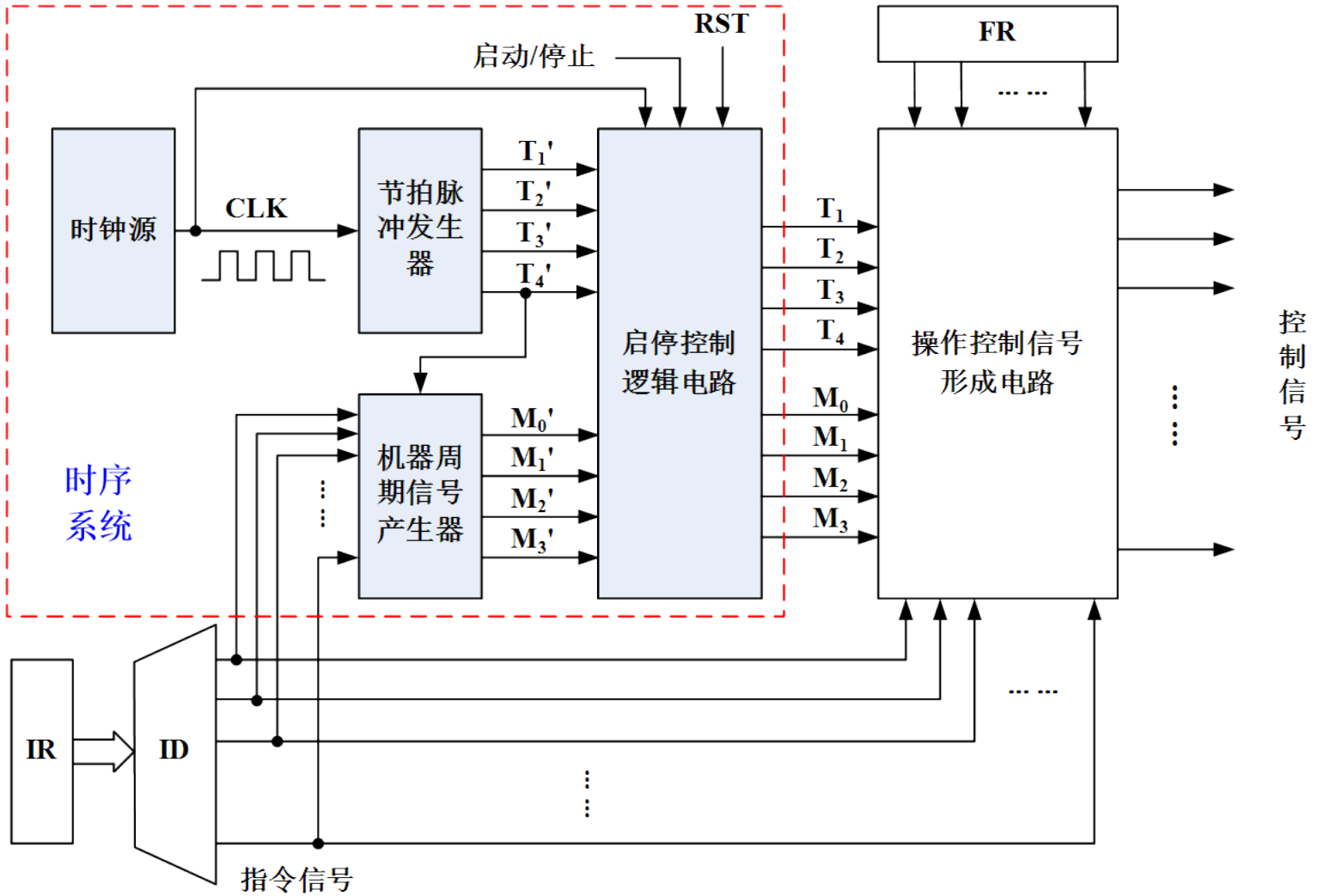
- （1）平均每条指令包含的时钟周期（节拍）数=3×2=6，故**CPI**=6
- 主频100MHz→**每个时钟周期**=1/（100×10^6）s=10^{-8}s
- **每条指令的执行时间**=6×10^{-8}s
- **每秒钟执行的指令条数**=1/（6×10^{-8}）=$100/6\times10^6$=16.67 MIPS

- （2）平均每条指令访存1.5次，每次访存增加1个时钟周期，故**每条指令的平均时钟周期数**=3×2+1.5=7.5个，每条指令的平均执行时间=7.5×10^{-8}s=75ns



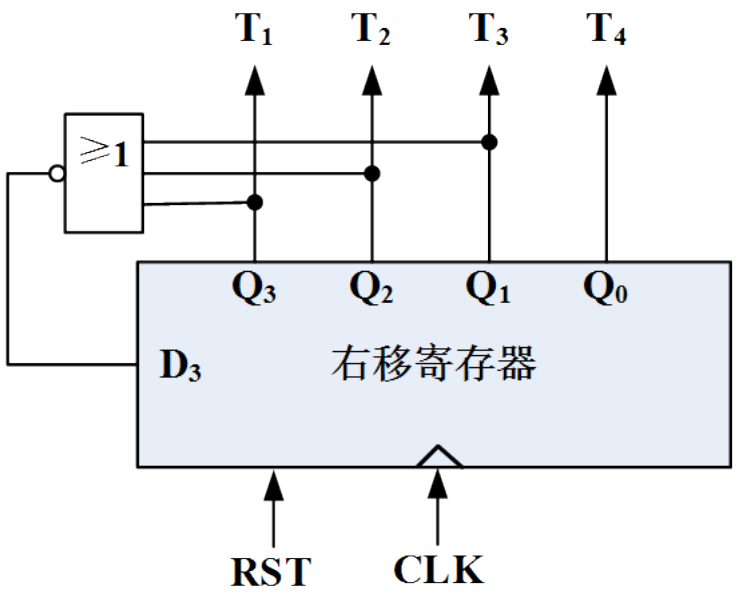
4、时序部件

- **时序部件：是产生指令周期中各时序信号的逻辑电路。**
- **包含4个部分**

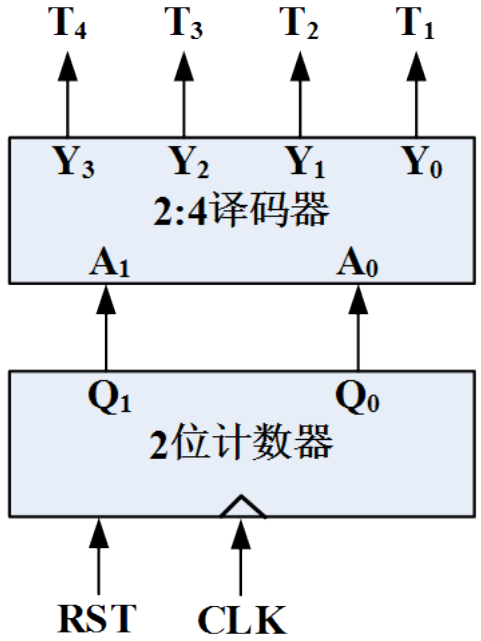


4、时序部件

- （1）时钟源：时钟脉冲信号CLK
  - 石英晶体振荡器产生的计算机的基准主频脉冲
- （2）节拍脉冲发生器：顺序脉冲发生器
  - 以CLK为时基，连续不断地产生一组有序的、间隔相等或不等的环形脉冲序列。



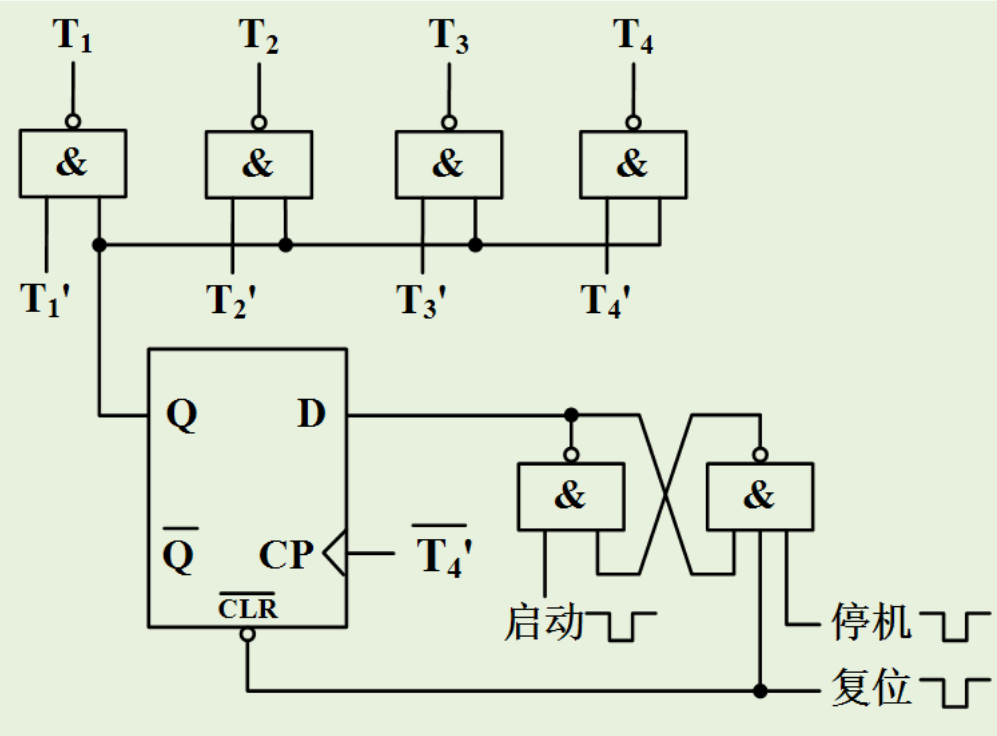
（a）环形计数器产生节拍脉冲



（b）计数器和译码器产生节拍脉冲

4、时序部件

- （3）机器周期信号产生器：也是一个顺序脉冲发生器
  - 机器周期个数，可能因为指令的不同而不同，比较适合使用计数器和译码器电路实现
- （4）启停控制逻辑电路
  - 准确可靠地开启或封锁计算机工作时钟
    - 控制微操作命令序列的产生或停止
    - 启动或停止计算机的运行



- Q=1：节拍脉冲$T_1' \sim T_4' \rightarrow T_1 \sim T_4$
- Q=0：关闭时序系统
- $\overline{R} - \overline{S}$锁存器输出→D触发器的输入
- T_4'下跳沿→D触发器的时钟
- 能保证：在T_1前沿开启时序部件，在T_4后沿关闭时序部件

7.1.4 单周期、多周期与流水线实现

- 指令的实现：有赖于CPU的系统结构和数据通路
- CPU实现指令的方式：单周期实现、多周期实现、流水线实现
  - 直接影响CPU的微架构/系统结构

- 1、单周期CPU
- 2、多周期CPU
- 3、流水线CPU

1、单周期CPU

- 每条指令均在一个时钟周期内完成
  - 所有指令的执行时间等长：CPI=1
  - 各部件一般采用直接连接
  - 存储器一般采用哈佛结构
  - 时钟周期
    - 上升沿：启动指令存储器取指令
    - 时钟周期内：进行指令译码及读寄存器、运算、读写存储器等
    - 下降沿：将结果写回目的寄存器或存储器或PC

- 优点：实现方案简单
  - 现代计算机CPU：并不采用单周期实现方式
- 缺点：性能和硬件成本低效
  - CPU执行时间=程序的指令数×CPI×时钟周期
  - CPI=1
  - 时钟周期比较长：由执行时间最长的指令决定，一般是lw指令的执行时间
  - 总体的CPU执行时间：并没有减少
  - 另外：单周期CPU的各功能部件必须单独设置，不能复用

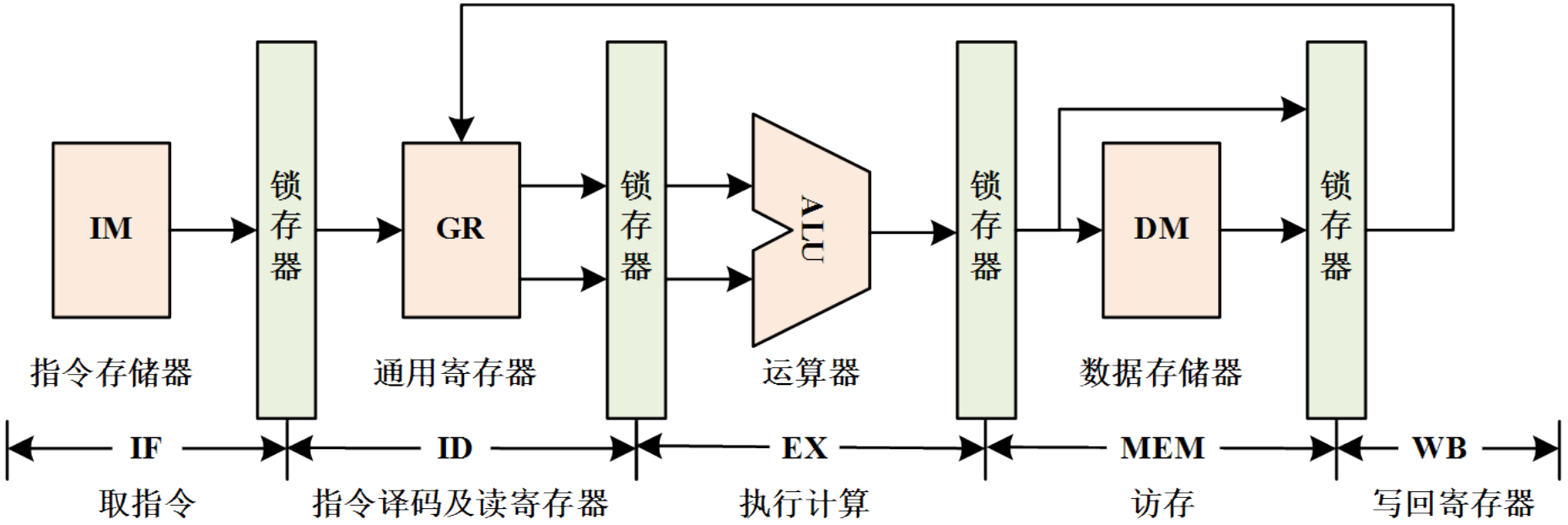
2、多周期CPU

- **多周期CPU：每条指令都需要多个时钟周期才能完成**
  - **CPI > 1：每个时钟周期只执行指令的一步操作**
  - **时钟周期可大大缩短，CPU主频可大大提高**
  - **无空耗时间**
- **优点：总体运行时间更短，效率更高**
- **缺点：硬件更复杂、更多暂存器**
  - **多周期CPU的各部件可以采用总线连接，也可以采用直接连接**
  - **需要添加暂存器，以保存每个步骤的操作结果，供后续部件或操作使用**
- **主要特征：**
  - **一条指令的执行要经过多个时钟周期**
  - **不同指令所占用的时钟周期数可以不同**
  - **功能部件可在指令周期中共享（多次使用），但需要暂存每步操作的结果**

3、流水线CPU

- **流水线CPU：每条指令的执行也需要多个时钟周期**
  - 在多周期CPU的基础上，基于**并行处理技术**，将各个部件按顺序组成一条流水线，指令源源不断进入流水线，经各个部件顺序处理后流出流水线

- **经典五级流水线结构**
  - 每个阶段对应**独立的部件**，可同时为5条指令服务
  - 需要在各个阶段之间插入锁存器
  - 流水线时钟周期要采用**各个阶段中操作时间最长**的阶段为准



例题

- 【例7.2】假设某机一条最复杂指令执行需要5个阶段，各段所需的时间分别为：取指令（IF）40ns、指令译码及读寄存器（ID）20ns、执行运算（EX）15ns、访存（MEM）40ns、写回（WB）20ns。
- （1）如果使用单周期CPU实现，则其主频10MHz是否合适?
  - 单周期CPU：每条指令在一个时钟周期内完成
  - 每条指令的执行时间=40+20+15+40+20=135ns
  - 主频10MHz：时钟周期=100ns
  - 在一个时钟周期内完成不了指令的执行，因此不合适
- （2）如果使用多周期CPU实现，每个阶段一个时钟周期，程序中需要5个、4个、3个和2个阶段的指令分别占10%、10%、40%、40%，则其CPI是多少？其主频最大值是多少?
  - 多周期CPU：每个阶段就是1个时钟周期
  - CPI=5×10%+4×10%+3×40%+2×40%=0.5+0.4+1.2+0.8=2.9
  - 时钟周期：应选择操作时间最长的取指令和访存的40ns，则主频最大是1/40ns=25MHz

例题

- （3）如果使用流水线CPU实现，忽略锁存延迟，则其流水线时钟周期是多少？理想情况下，其执行速度是单周期CPU的多少倍，是多周期CPU的多少倍?
  - 单周期CPU：每条指令的执行时间是135ns
  - 多周期CPU：平均每条指令的执行时间是40ns×2.9=116ns
  - 流水线CPU：时钟周期也应该选择操作时间最长的阶段，为40ns
    - 理想情况下：每条指令的执行只需要40ns
    - 流水线CPU的执行速度：
      - 是单周期CPU的135/40=3.375倍
      - 是多周期CPU的116/40=2.9倍



7.1.5 控制器的设计步骤

1、硬布线控制器的CPU设计步骤：

① 确定指令系统，包括每条指令的格式、功能和寻址方式， 分配操作码。

② 确定CPU的内部结构，包括运算器的功能和组成，控制器的类型、结构组成及各部件的连接方式和数据通路，时序系统的构成。

③ 分析每条指令的执行过程，按机器周期顺序，写出所必需发送的微操作控制信号序列。

④ 综合每个微操作控制信号的逻辑函数，化简和优化。

⑤ 用逻辑电路实现。

2、微程序控制器的CPU设计步骤：

① 确定指令系统，包括每条指令的格式、功能和寻址方式，分配操作码。

② 确定CPU的内部结构，包括运算器的功能和组成，控制器的类型、结构组成及各部件的连接方式和数据通路， 时序系统的构成。

③ 分析每条指令的执行过程，画出指令系统的*微程序流程图*。

④ 写出每条微指令所发送的微操作控制信号序列。

⑤ 设计微程序控制器的结构，确定控制存储器容量，*设计微指令格式*。

⑥ 分配微程序流程图中各微指令的微地址，并*编写微指令代码*。

⑦ 将所有的*微指令代码装入*控制存储器相应单元。

7.1.5 控制器的设计步骤

- 需要注意两点：
- (1)在一个CPU中，既可以使用硬布线控制器，也可以同时使用微程序控制器
  - 在现代的RISC技术中
    - 采用硬布线方法：实现绝大多数的简单指令
    - 采用微程序控制的方法：实现少数复杂指令的功能
- (2)在上述设计步骤中，不是单向线性的过程，而可能会交错进行
- ①②：本课程要求掌握基本的设计和分析，其更深入、更宏观的探讨属于“计算机系统结构”的高级范畴
- 硬件具体实现：涉及电子器件与技术，属于“数字电路与电子学”的低层范畴。
- 本章：在既定的指令系统基础上，构造系统结构和数据通路，然后从③步骤开始进行分析设计，隶属“计算机组成”的中间范畴，也是本课程的重点



7.2 Yy-z02模型机的系统结构与数据通路

- 7.2.1 Yy-z02模型机的系统结构
- 7.2.2 Yy-z02模型机的数据通路
- 7.2.3 Yy-z02模型机的指令执行过程

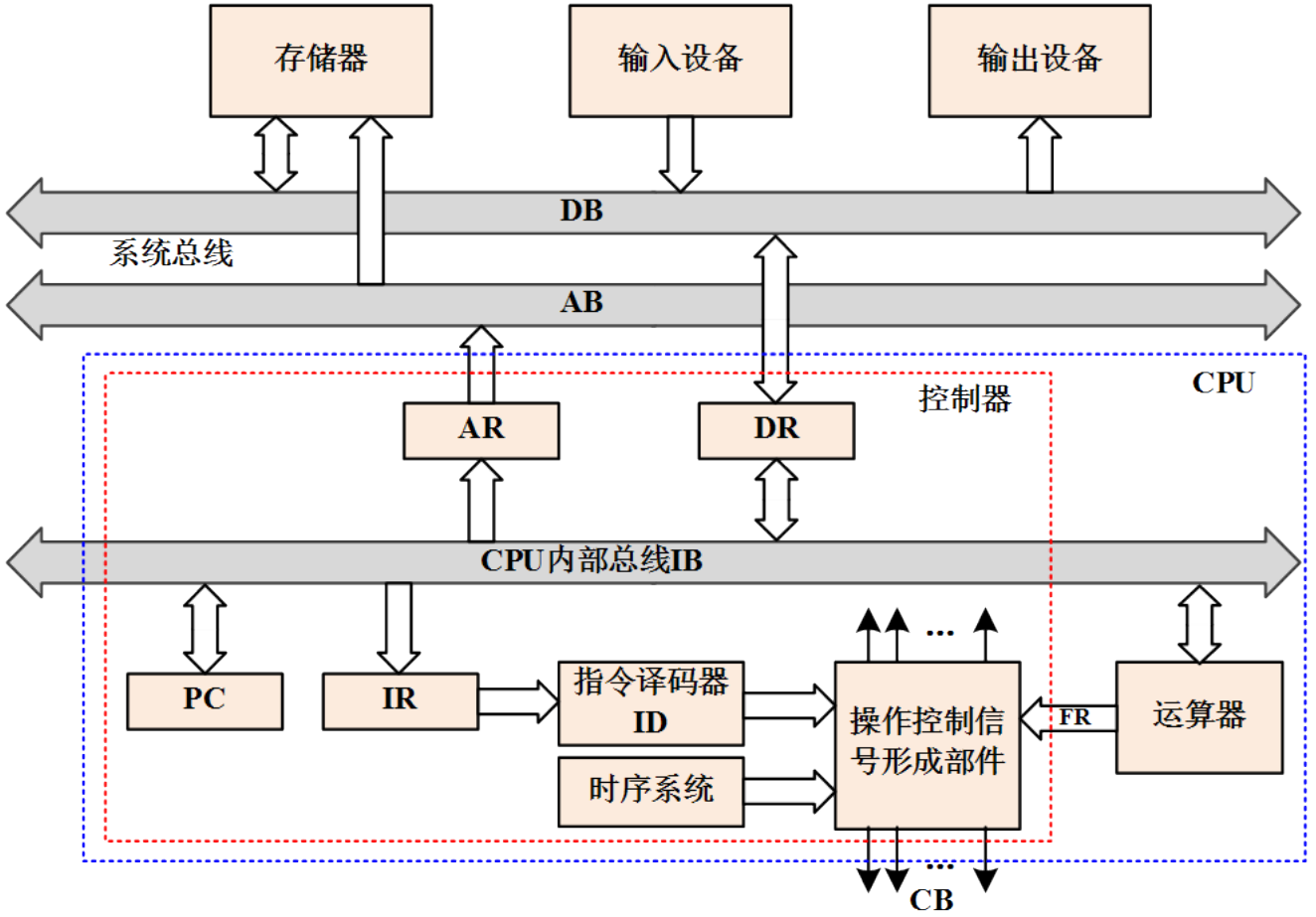


7.2.1 Yy-z02模型机的系统结构

- 指令系统的实现←构造它的硬件系统
- 构造过程：
  - 分析指令格式和各指令功能
  - 确定部件连接方式（总线连接/直接连接）、CPU类型（单周期/多周期/流水线）
  - 依据各条指令的功能，设计所需的功能部件、确定对应控制信号
  - 按既定方式进行连接，在部件之间建立数据通路
- **部件的连接方式**：两种
  - **直接连接**：在所有需要传送数据的部件之间创建一条直接通路
    - 比较适合CPU流水线设计中的串行连接结构
    - 本课程：RISC-V和ARM，两种直接连接的多周期CPU系统结构
  - **总线连接**：所有的部件都挂在总线上，部件之间通过总线来交换数据
    - 优点：结构简单；缺陷：总线需要分时共享，指令的执行需要多个周期
    - 举例：Yy-z02模型机系统结构，以总线连接方式构造

1、总线连接的简单计算机系统结构

- **系统总线：连接CPU、存储器、输入设备、输出设备**
- **CPU内部总线：连接CPU内部的专用寄存器、运算器（含ALU、GR和暂存器）**
- **CPU访问存储器和输入输出设备：均需通过AR和DR完成**

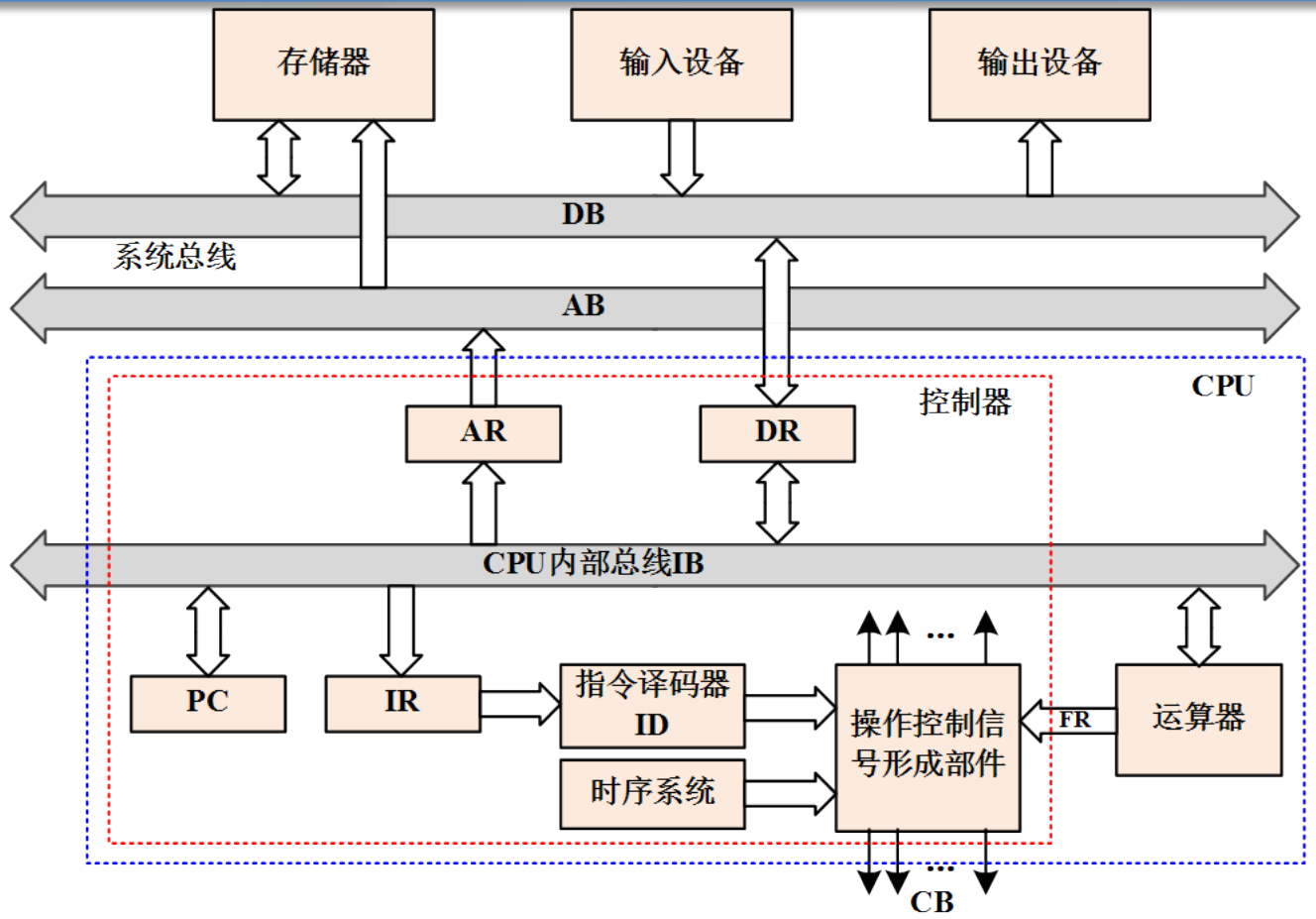


- **CPU读操作：**
  - ①送地址到AR：CPU→IB→AR
  - ②读数据到DR：AR→AB；CPU发送 存储器/IO 读信号；读出的数据→DB→DR寄存器
  - ③送DR到目的部件：DR→CPU内部总线→目的部件

- **取指令的过程：典型的读存储器操作**
  - ①PC→AR（经由IB）
  - ②Mem[AR]→DR（读存储器，经由AB和DB）
  - ③DR→IR（经由IB）

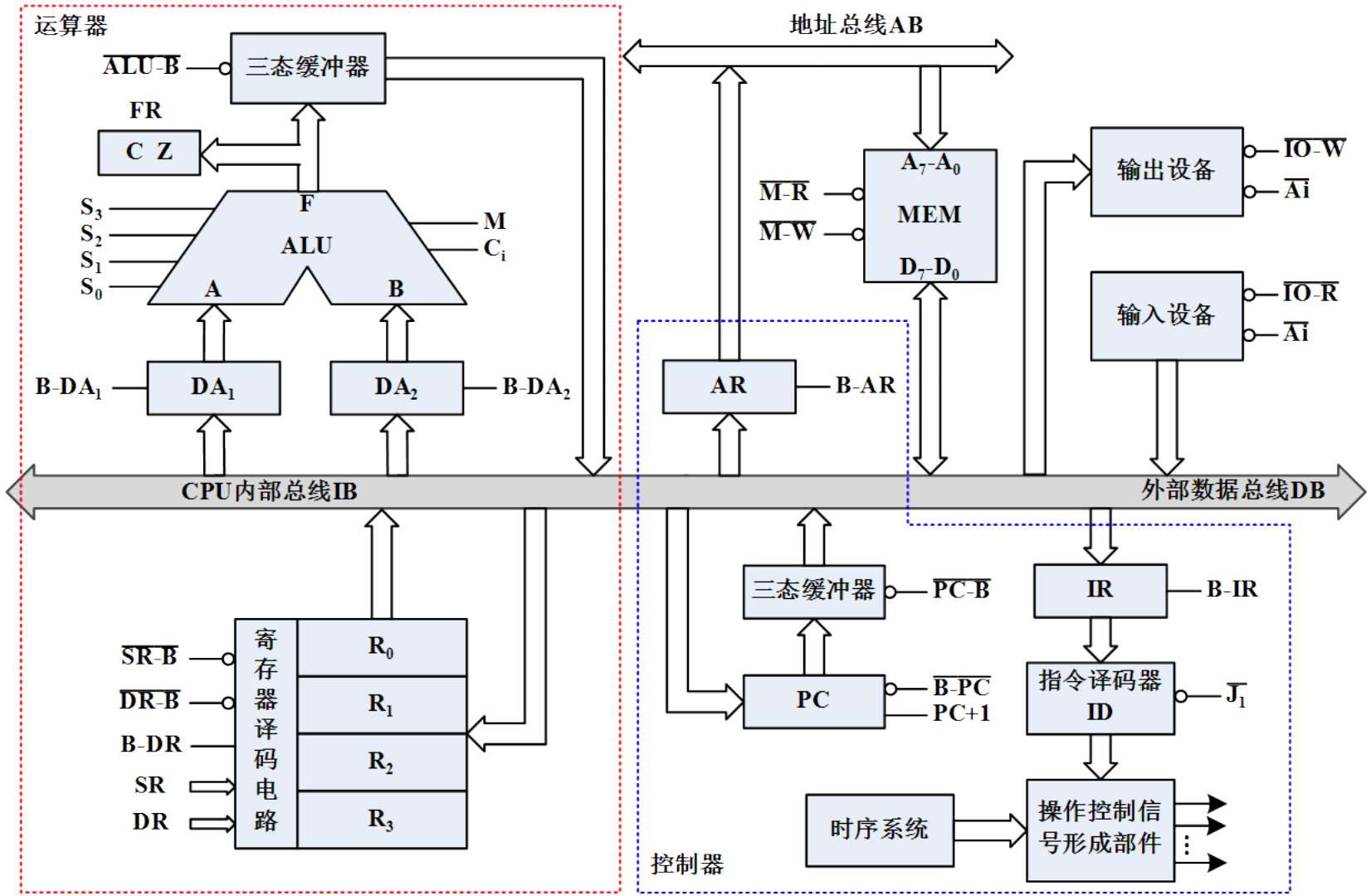
1、总线连接的简单计算机系统结构

- **CPU写操作：**
  - ①送地址到AR：CPU→IB→AR
  - ②送数据到DR：CPU→IB→DR
  - ③执行写操作：AR→AB；DR→DB；CPU发送 存储器/IO 写信号



- **子程序调用指令：将PC写入SP指向的存储器单元，典型的写存储器操作**
  - ①SP→AR（经由IB）
  - ②PC→DR（经由IB）
  - ③DR→Mem[AR]（写存储器，经由AB和DB）

2、 Yy-z02模型计算机的系统结构



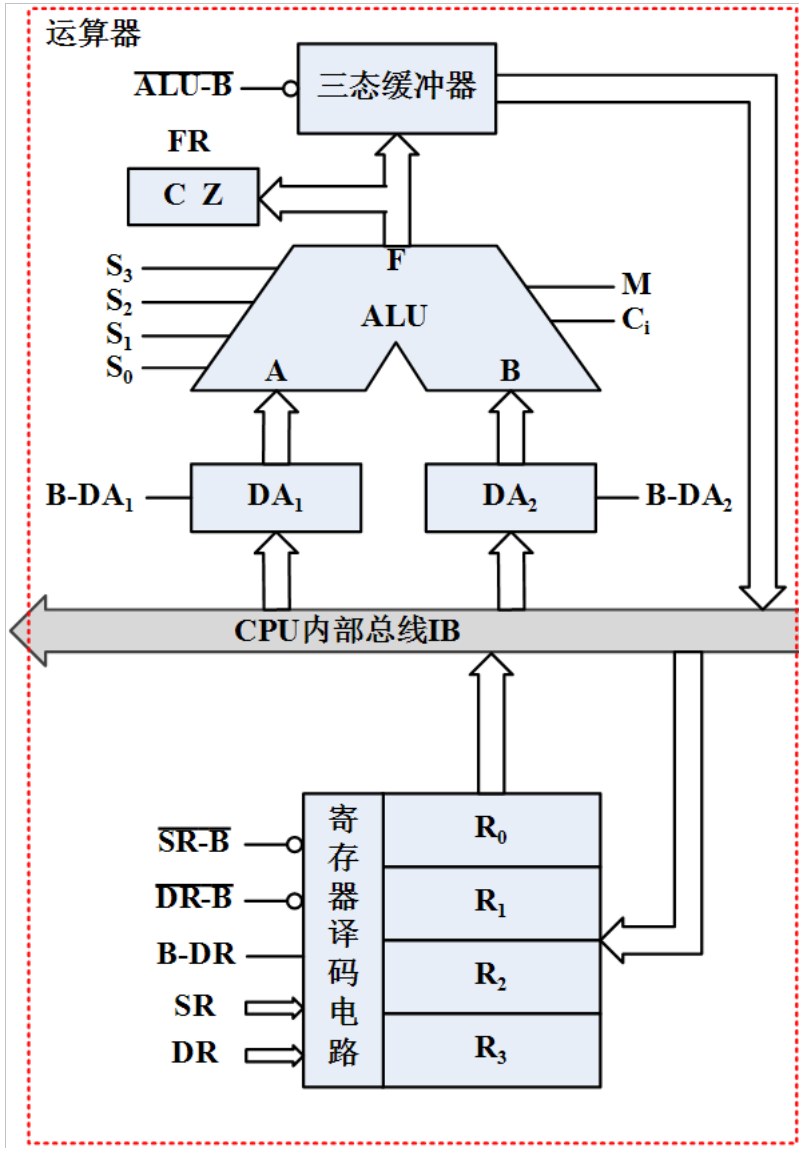
- 两处改造：
  - ①将外部数据总线和CPU内部总线合二为一
  - ②省略了数据寄存器DR
- 单总线结构
- 内外总线合并
- 8位AB
- 8位DB

2、Yy-z02模型计算机的系统结构

No Image

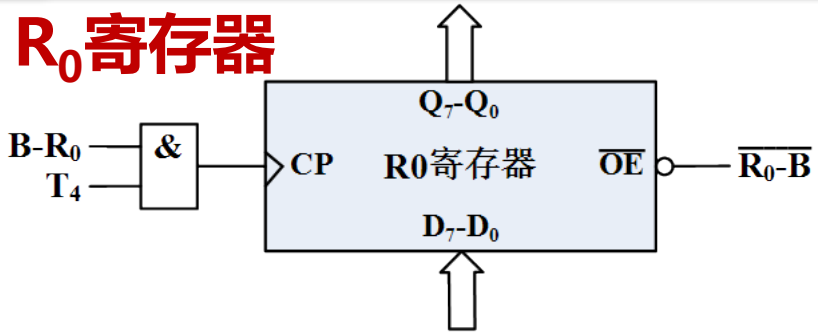
Why?

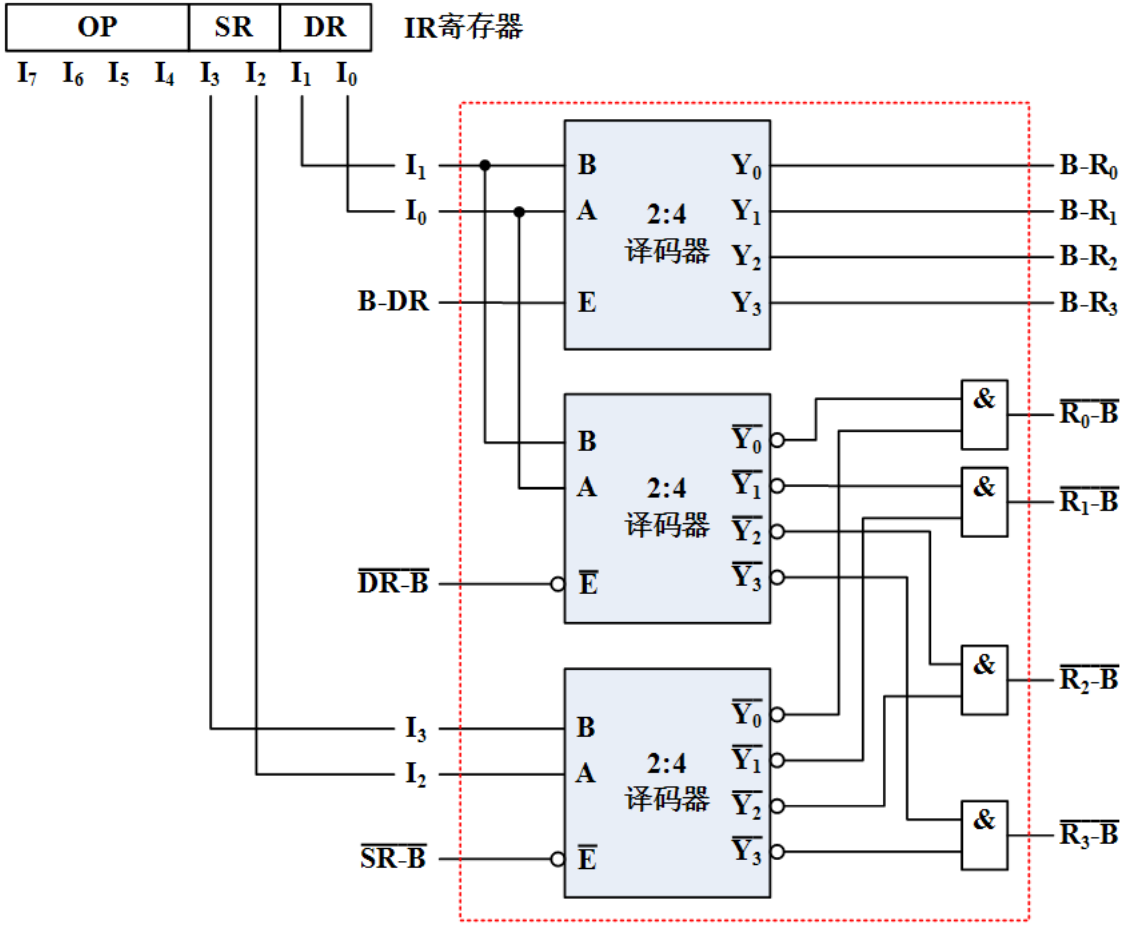
- 控制器如何选择其中一个寄存器进行读或者写呢?
- 通过寄存器译码电路实现



Yy-z02通用寄存器堆译码电路

R_0寄存器

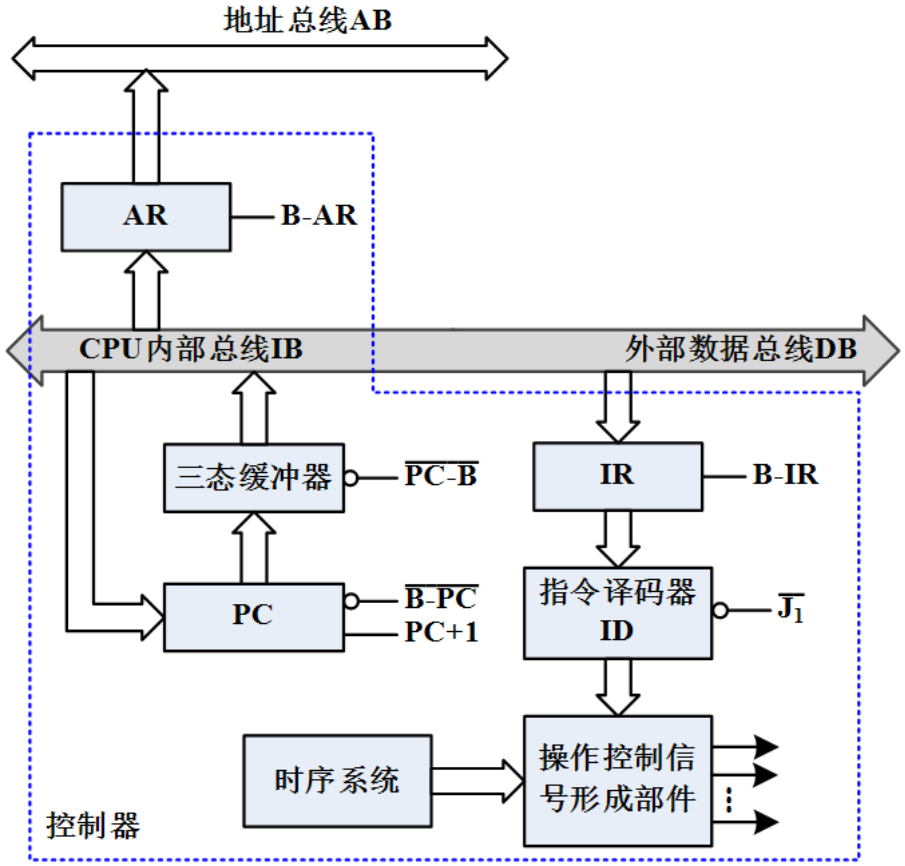




- 构成：74LS374（带输出使能端 $\overline{OE}$ 的8D寄存器）
- $B-R_0$：T_4节拍有效，总线数据写入R_0，写信号
- $\overline{R_0-B}$：R_0的$\overline{OE}$信号，=0，Q端数据送到总线上，读信号
  - 输入：
    - 控制信号$B-DR$、$\overline{DR-B}$、$\overline{SR-B}$
    - 指令寄存器IR的SR、DR字段
  - 输出：4个寄存器的读写信号
    - 4个寄存器打入信号$B-R_i$
    - 4个寄存器输出使能信号$\overline{R_i-B}$
  - 部件：三个2:4译码器
  - $\overline{DR-B}$：用DR字段选择要读出的寄存器
  - $\overline{SR-B}$：用SR字段选择要读出的寄存器
  - $B-DR$：用DR字段选择要写入的寄存器

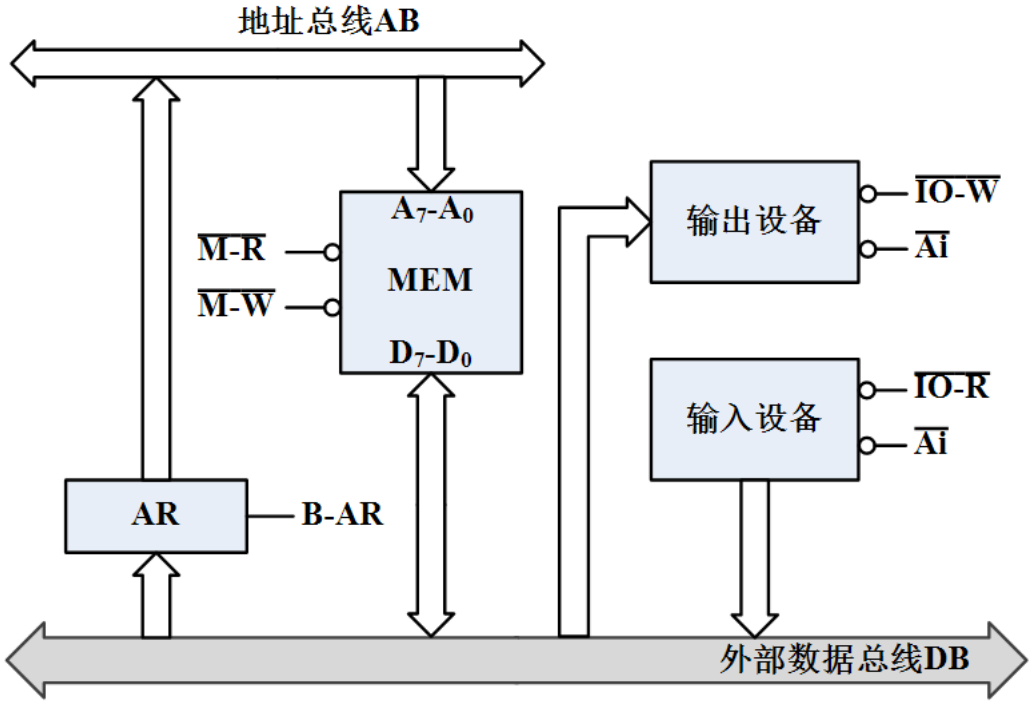
2、Yy-z02模型计算机的系统结构

No Image



2、Yy-z02模型计算机的系统结构

- （3）存储器
  - 容量：256×8位
  - 类型：SRAM
  - 存储器读、写控制信号：$\overline{M-R}$、$\overline{M-W}$
  - 地址：AB提供，AB来源只有AR
  - 数据：送到DB上
- （4）输入和输出设备
  - I/O读、写控制信号：$\overline{IO-R}$、$\overline{IO-W}$
  - 设备选择信号：$\overline{A_i}$，来自AB，=0选中
  - 输入设备：8位逻辑开关
  - 输出设备：8位LED显示灯



Yy-z02模型计算机的控制信号一览表

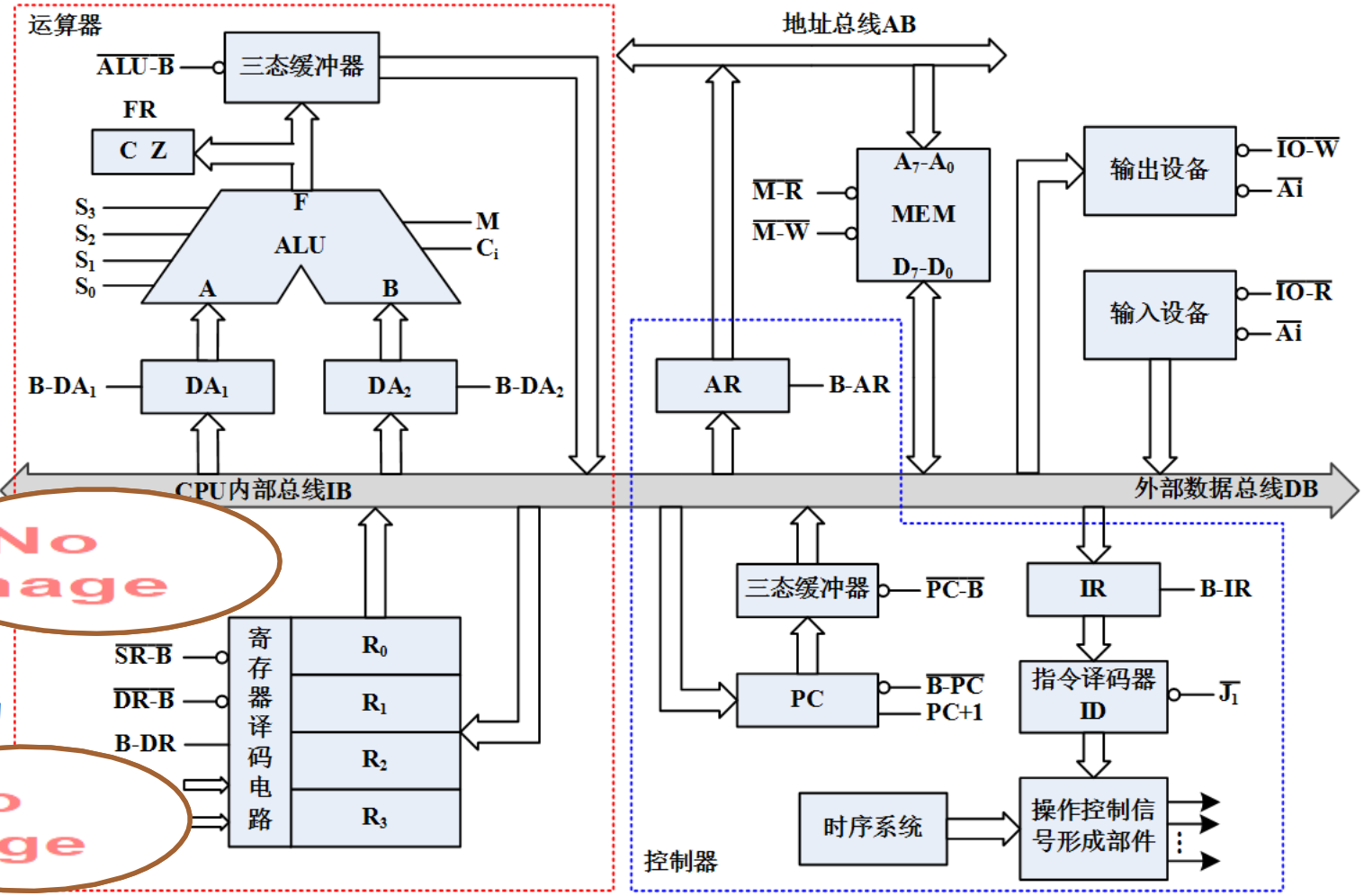
序号	控制信号	功能	有效节拍
1	$PC+1$	PC内容自增1	T_4
2	$\overline{PC-B}$	PC内容送总线	
3	$\overline{B-PC}$	总线上内容打入PC	T_4
4	$B-IR$	总线上内容打入IR	T_3
5	$\overline{J_1}$	指令译码器工作	T_4
6	$B-AR$	总线上内容打入AR	T_3
7	$B-DA_1$	总线上内容打入DA_1	T_4
8	$B-DA_2$	总线上内容打入DA_2	T_4
9	$S_3S_2S_1S_0MC_i$	选择ALU运算功能	

序号	控制信号	功能	有效节拍
10	$\overline{ALU-B}$	ALU运算结果送总线	
11	$\overline{SR-B}$	SR字段指定的寄存器内容送总线	
12	$\overline{DR-B}$	DR字段指定的寄存器内容送总线	
13	$B-DR$	总线上数据打入DR字段指定的寄存器	T_4
14	$\overline{M-R}$	存储器执行读操作	
15	$\overline{M-W}$	存储器执行写操作	T_3
16	$\overline{IO-R}$	输入设备执行读操作	
17	$\overline{IO-W}$	输出设备执行写操作	T_3
18	$\overline{A_i}$	端口地址选中	

7.2.2 Yy-z02模型机的数据通路

- （1）存储器读操作
  - ①送地址到AR
  - ②读数据到目的部件
- 两个机器周期完成

- 举例：取指令操作
  - M_0：PC→AR，PC+1→PC
    - 发送的控制信号?
  - M_1：Mem[AR]→IR，指令译码
    - 发送的控制信号?

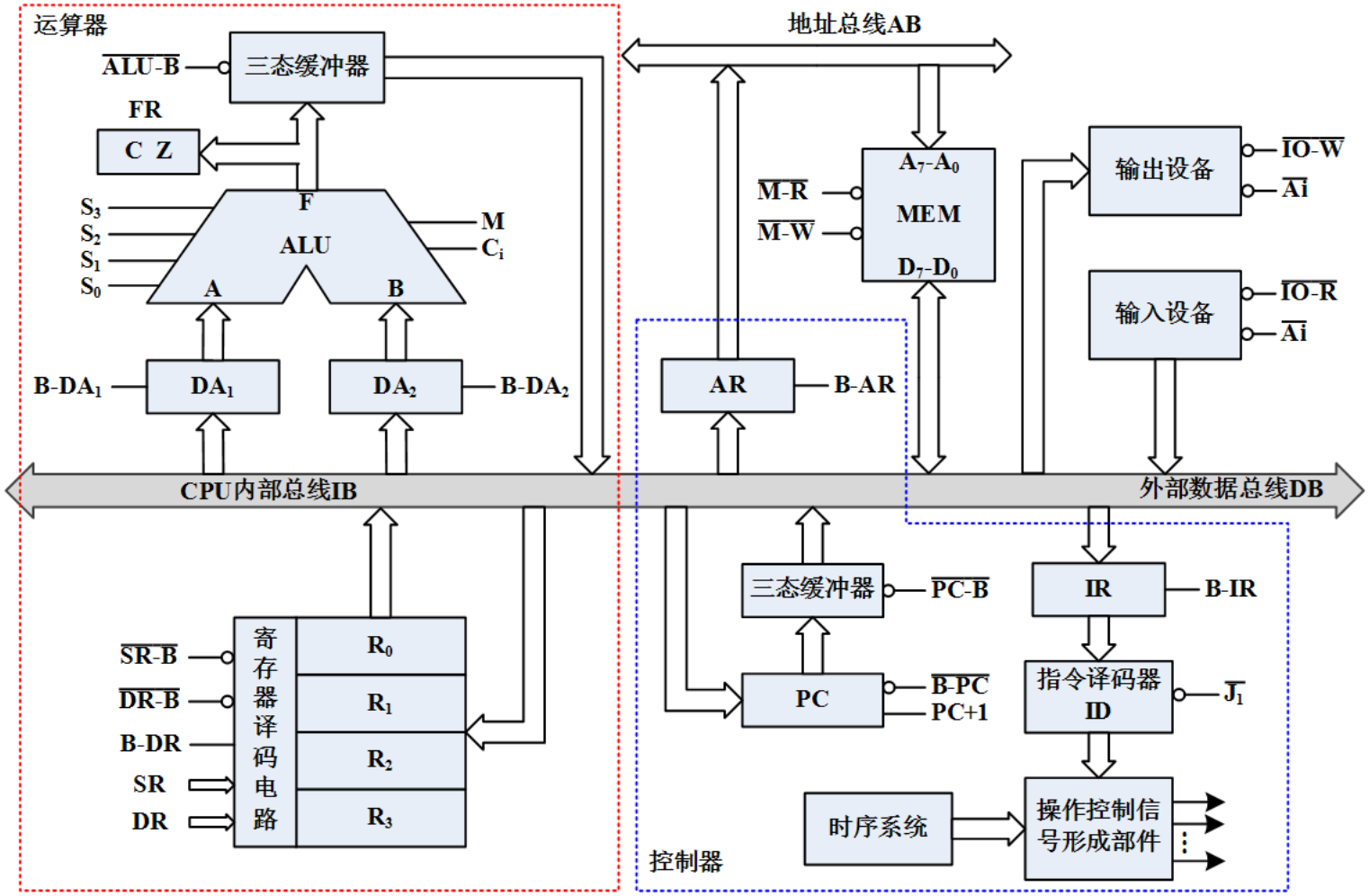


- $\overline{PC-B}$：M_0周期内始终有效
- $B-AR$：T_3节拍动作
- $PC+1$：T_4节拍动作

7.2.2 Yy-z02模型机的数据通路

- （2）存储器写操作
  - ①送地址到AR
  - ②写数据到存储器
- （3）I/O设备读操作
  - ①送地址到AR
  - ②读数据到目的部件
- （4）I/O设备写操作
  - ①送地址到AR
  - ②写数据到输出设备
- （5）运算器运算操作
  - ①送第一个数据到DA_1
  - ②送第二个数据到DA_2
  - ③运算：DA_1 (alu_op) DA_2→总线→目的部件

需要两个机器周期完成



❖ 需要三个机器周期完成

7.2.3 Yy-z02模型机的指令执行过程

- 1、指令执行过程概述
- 2、Yy-z02的两条典型指令
- 3、Yy-z02典型指令的执行过程

1、指令执行过程概述

- 一条指令的执行过程包括取指令、分析指令、执行指令三个步骤：
- （1）取指令：以PC为地址读存储器，读出的指令码送IR，PC自增
  - Mem[PC]→IR，PC自增
- （2）分析指令：控制指令译码器ID对IR进行译码，产生能识别指令的特定信息或信号
- Yy-z02：两个机器周期
  - M_0：PC→AR，PC+1→PC；　　发送的控制信号：$\overline{PC-B}$、$B-AR$和$PC+1$
  - M_1：Mem[AR]→IR，指令译码；　　发送的控制信号：$\overline{M-R}$、$B-IR$和$\overline{J_1}$
- （3）执行指令：实现指令规定的功能，具体操作取决于：
  - 指令的功能：数据传送、运算、转移等各种操作
  - 寻址方式：取指令第二字、计算地址、取操作数等其他操作



2、Yy-z02模型机的两条典型指令

❖ （1）ADD　　DR，Data

OP（= 0101）	××（2位）	DR（2位）
Data（8位立即数）		

- ❖ 功能：加法
  - Reg[DR]+Data→Reg[DR]
  - 简记：（DR）+Data→DR
- ❖ 寻址方式：
  - 源操作数：立即寻址
  - 目的操作数：寄存器寻址

❖ （2）JMP　　Addr

OP（= 1000）	×××× （4位）
Addr（8位直接转移地址）	

- ❖ 功能：无条件转移指令
  - Addr→PC
- ❖ 寻址方式：
  - 直接转移

2、Yy-z02模型机的两条典型指令

- 假设存储器的04H~07H单元存放了两条指令：
  - ADD R_0, 06H
  - JMP 04H

地址	机器码	助记符	功能
04H	0101 0000	ADD R_0, 06H	$(R_0)+06H \rightarrow R_0$
05H	0000 0110		
06H	1000 0000	JMP 04H	04H→PC
07H	0000 0100		

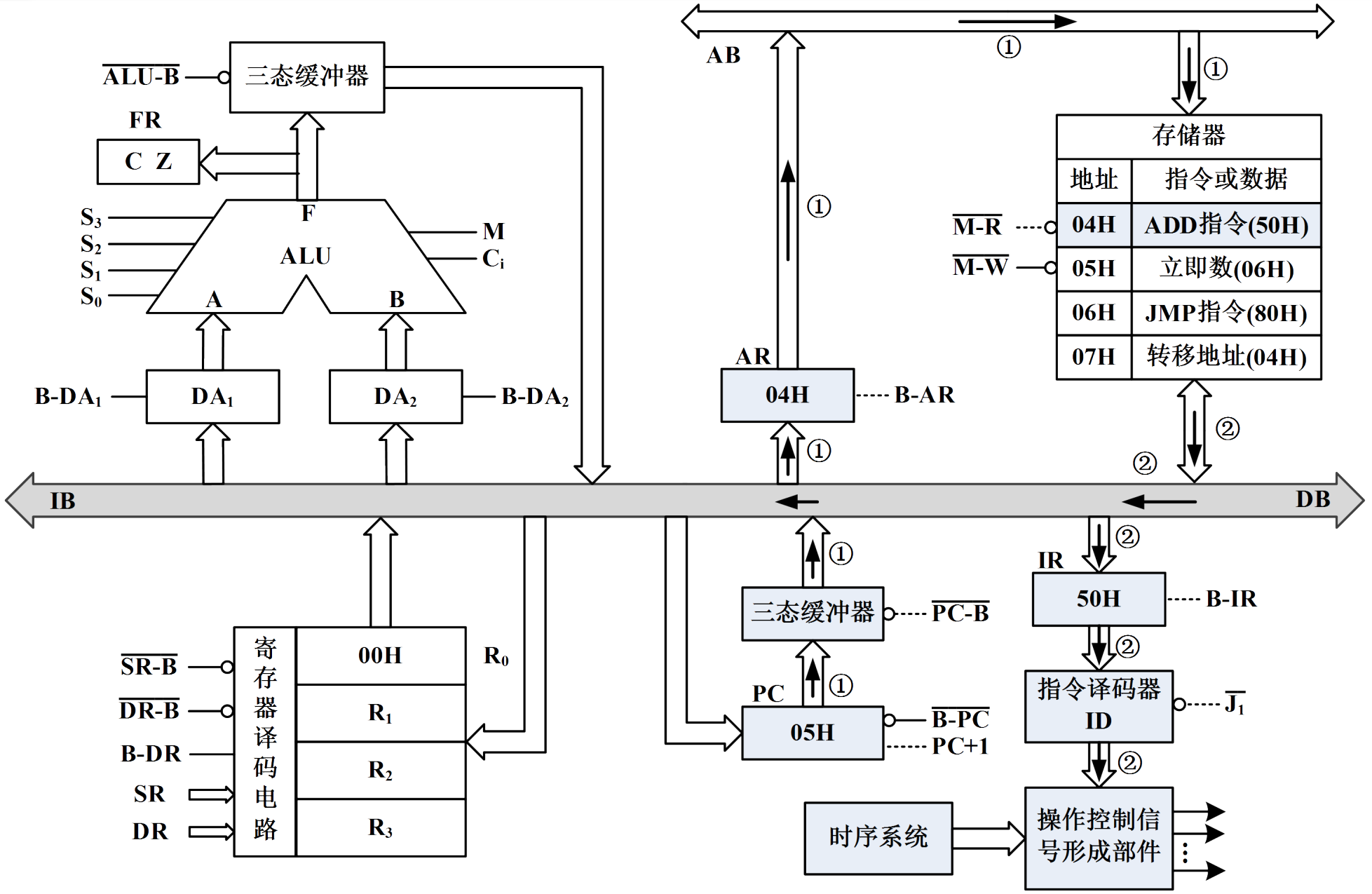
3、Yy-z02典型指令的执行过程

❖（1）ADD DR, Data指令的执行过程

❖ADD的指令周期：含6个机器周期

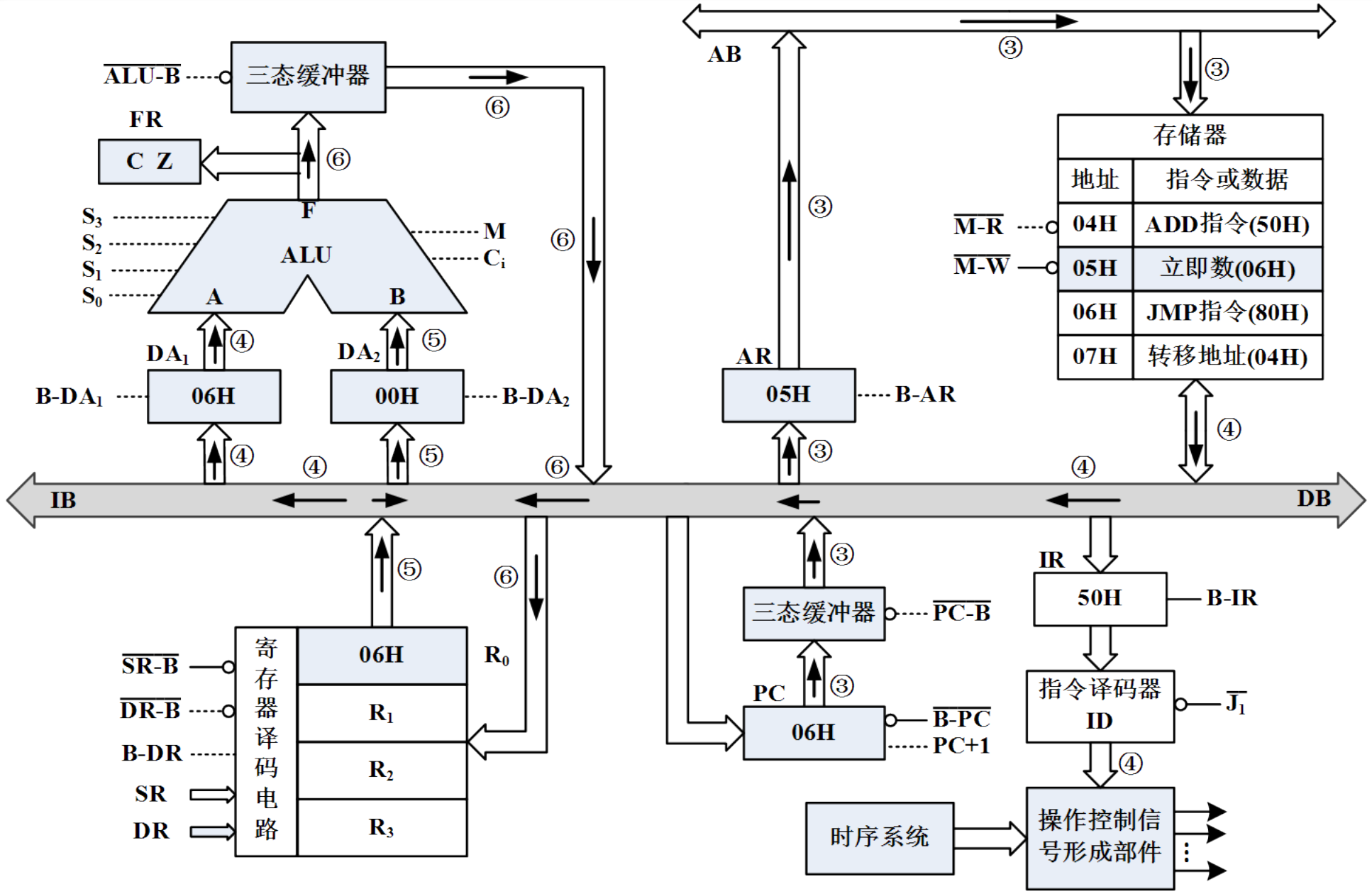
机器周期	操作	发送的控制信号
M_0	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
M_1	Mem[AR]→IR，指令译码	$\overline{M-R}$，$B-IR$，$\overline{J_1}$
M_2	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
M_3	Mem[AR]→DA_1	$\overline{M-R}$，$B-DA_1$
M_4	Reg[DR]→DA_2	$\overline{DR-B}$，$B-DA_2$
M_5	DA_1+ DA_2→Reg[DR]	$S_3S_2S_1S_0MC_i=100101$，$\overline{ALU-B}$，$B-DR$

ADD R_0, 06H指令的运行过程 $(R_0)+06H \rightarrow R_0$



取指令过程

ADD R_0, 06H指令的运行过程　　$(R_0)+06H \rightarrow R_0$



执行指令过程

3、Yy-z02典型指令的执行过程

❖（2）JMP Addr指令的执行过程

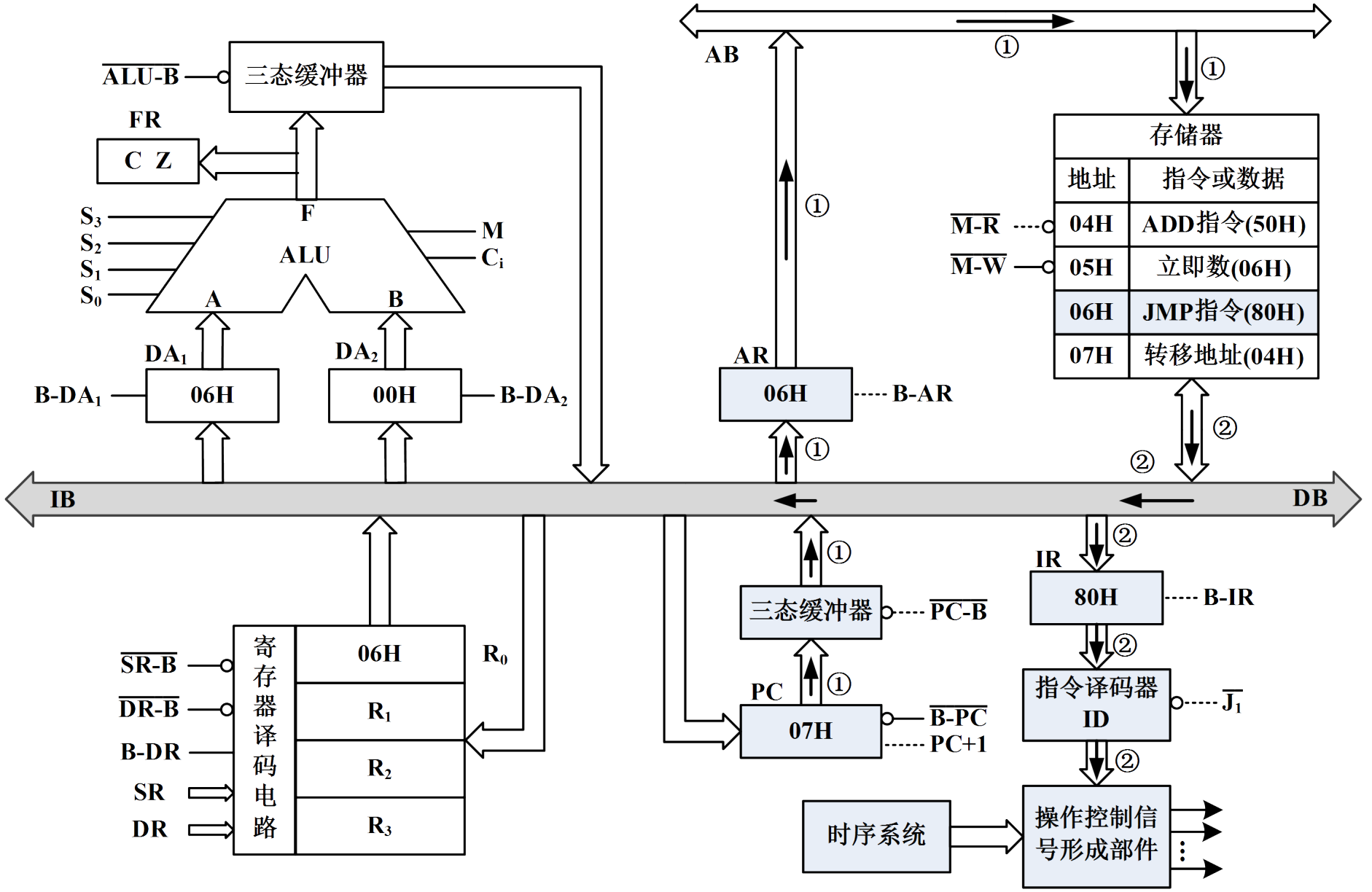
❖JMP的指令周期：含4个机器周期

机器周期	操作	发送的控制信号
M_0	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
M_1	Mem[AR]→IR，指令译码	$\overline{M-R}$，$B-IR$，$\overline{J_1}$
M_2	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
M_3	Mem[AR]→PC	$\overline{M-R}$，$\overline{B-PC}$

JMP 04H指令的运行过程

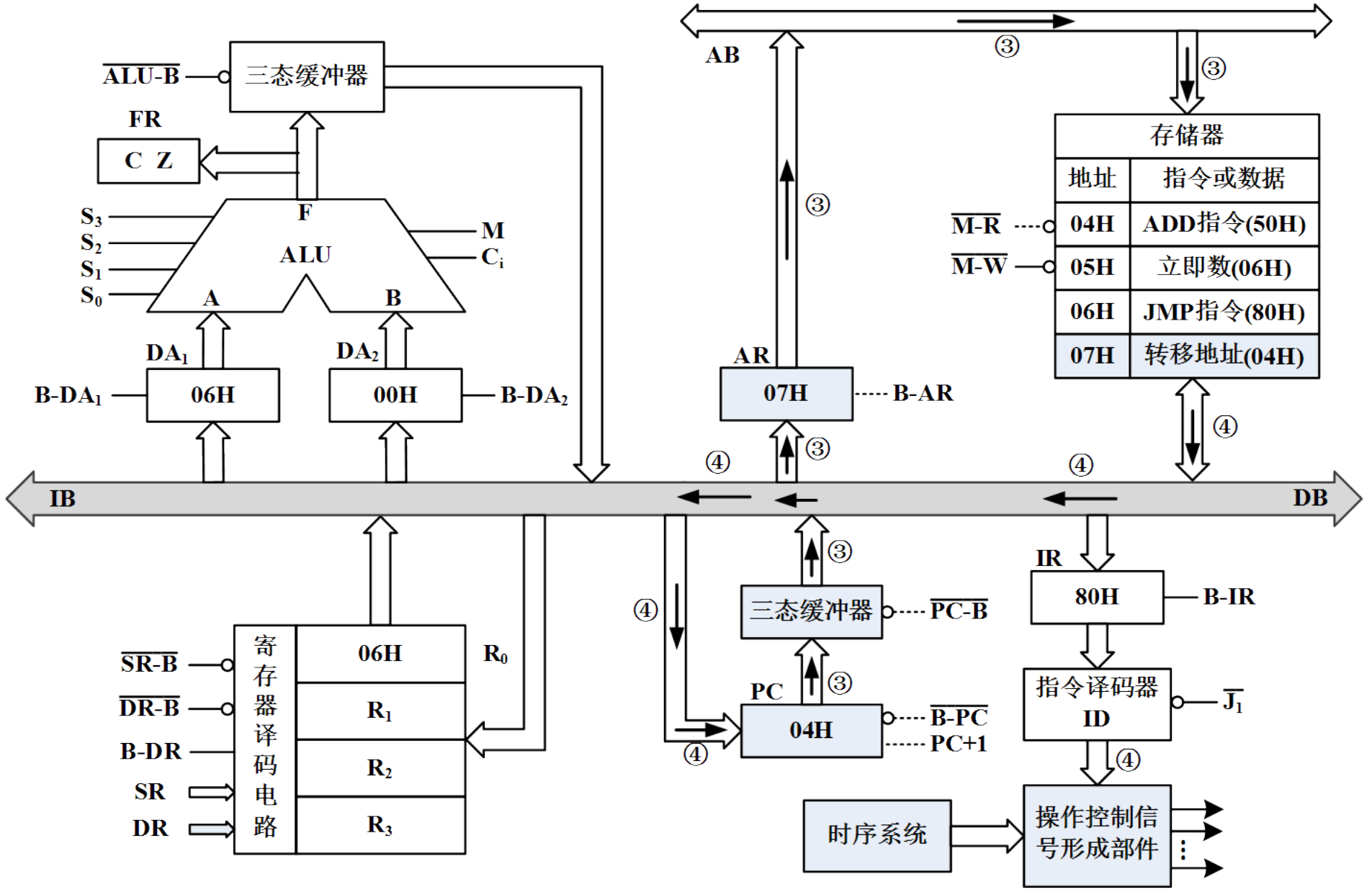
04H→PC

取指令过程



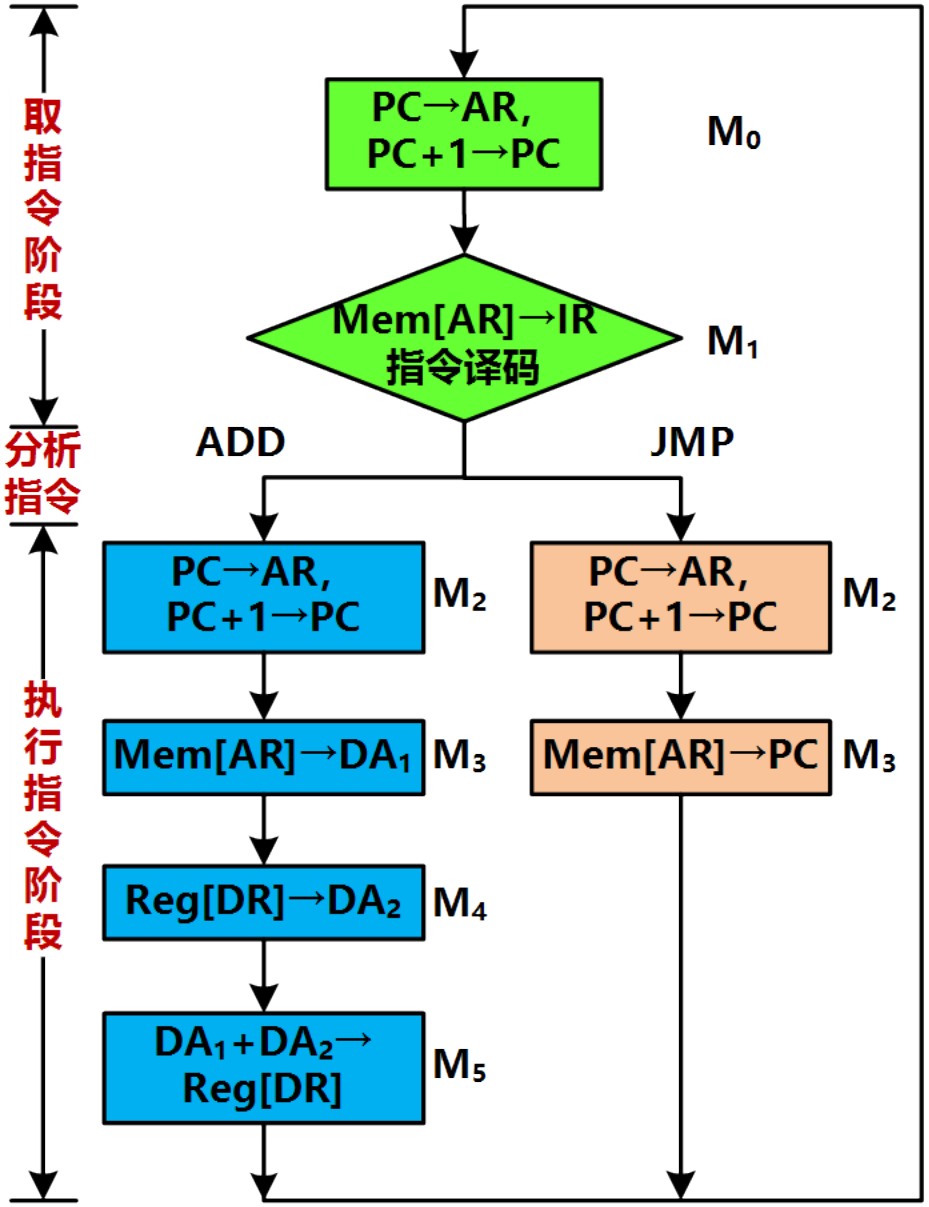
JMP 04H指令的运行过程

04H→PC



执行指令过程

指令执行的流程图

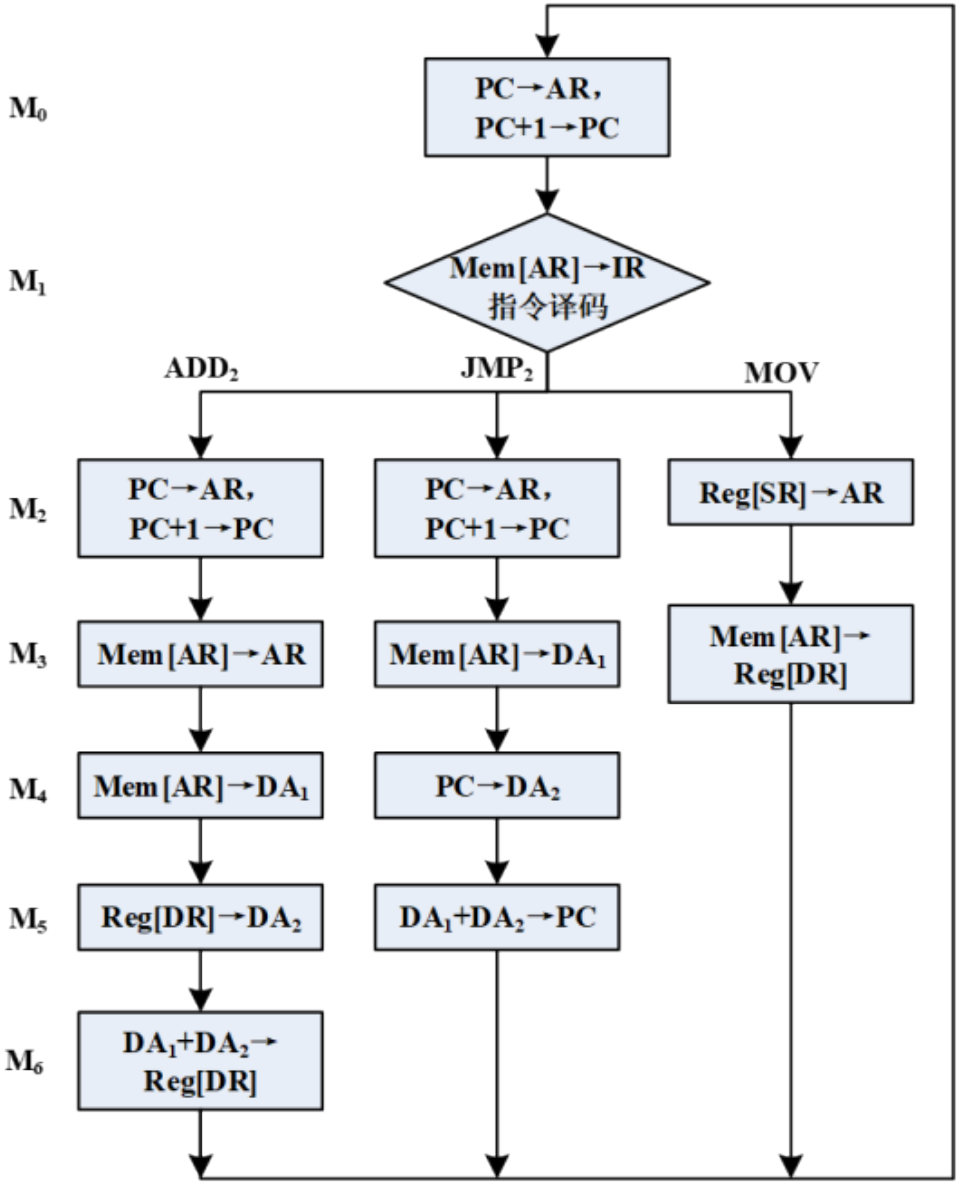

取指令阶段
分析指令
执行指令阶段
PC→AR,
PC+1→PC
M0
Mem[AR]→IR
指令译码
M1
ADD
JMP
PC→AR,
PC+1→PC
M2
PC→AR,
PC+1→PC
M2
Mem[AR]→DA1
M3
Mem[AR]→PC
M3
Reg[DR]→DA2
M4
DA1+DA2→
Reg[DR]
M5

例题：

- 【例7.3】基于Yy-z02模型机的系统结构，写出以下3条格式1指令的执行过程，画出指令流程图。
  - ADD_2　DR，[addr] ;功能：Reg[DR]+Mem[Addr]→Reg[DR]，Addr在指令第二字
  - JMP_2　Disp ;功能：PC+Disp→PC，即实现相对转移，相对偏移量Disp在指令第二字
  - MOV　DR，[SR]; Mem[Reg[SR]]→Reg[DR]，即从存储器读出数据存入DR寄存器，存储器地址由SR寄存器给出。

	ADD_2 DR，[addr]	JMP_2 Disp	MOV DR，[SR]
功能如何?	加法	转移	传送
指令几个字?	2个字	2个字	1个字
寻址方式?	源：直接寻址	相对寻址	源：寄存器间接寻址

例题：



- ADD_2 DR，[addr]
- JMP_2 Disp
- MOV DR，[SR]

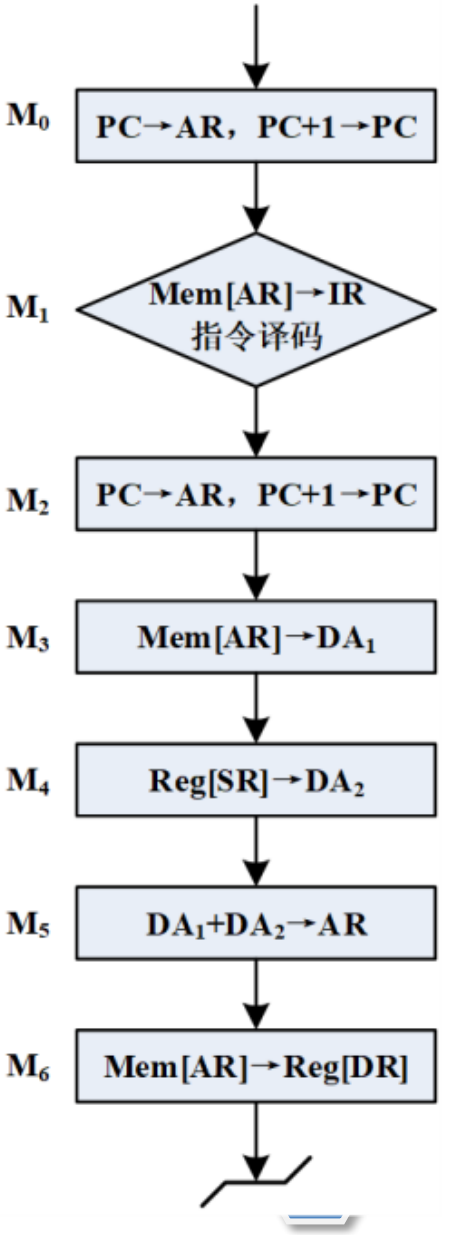
- 取指令
- 分析指令
- 执行指令：
  - 要不要取指令第二字?
  - 取出的是什么？数据 or 地址?
  - 取出后送到哪里?
  - 进行什么操作?

例题：

- 【例7.4】右图是一条Yy-z02指令的执行流程图，请分析：

（1）指令有几个字？格式如何？操作数类型和寻址方式是什么？描述指令的功能。

（2）从M_3机器周期开始，写出每个周期发送的控制信号。



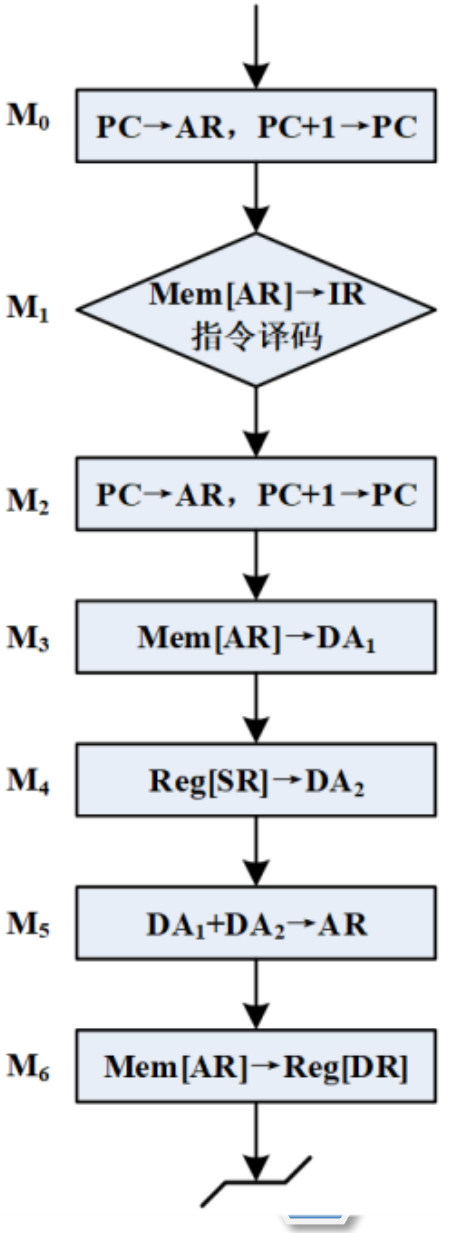
- （1）执行指令：M_2~M_6
  - 周期M_6：存储器的读操作，读出数据存入了DR寄存器
  - 指令功能：传送指令MOV（也称装数指令LDA）
  - 目的操作数：寄存器类型、寄存器寻址方式（即DR）
  - 源操作数：存储器类型，什么寻址方式（有效地址EA是怎么得到的）？
    - M_2 ~M_3周期：根据PC再次访问主存，故指令有第二字；读出的数据X送DA1
    - M_4、M_5周期：EA= Reg[SR]+X，源操作数：相对寄存器寻址
  - 指令的功能：Mem[Reg[SR]+X]→Reg[DR]
  - 助记符：MOV DR，[SR+X]　　LDA DR，[SR+X]
  - 格式：带双寄存器的格式1

例题：

❖ 【例7.4】右图是一条Yy-z02指令的执行流程图，请分析：

（1）指令有几个字？格式如何？操作数类型和寻址方式是什么？描述指令的功能。

（2）从M_3机器周期开始，写出每个周期发送的控制信号。



❖ （2）控制信号

M_3：Mem[AR]→DA_1；控制信号：$\overline{M-R}$，$B-DA_1$

M_4：Reg[SR]→DA_2；　控制信号：$\overline{SR-B}$，$B-DA_2$

M_5：DA_1 + DA_2→AR；控制信号：$S_3S_2S_1S_0MC_i=100101$，$\overline{ALU-B}$，$B-AR$

M_6：Mem[AR]→DR；　控制信号：$\overline{M-R}$，$B-DR$

7.3 硬布线控制器



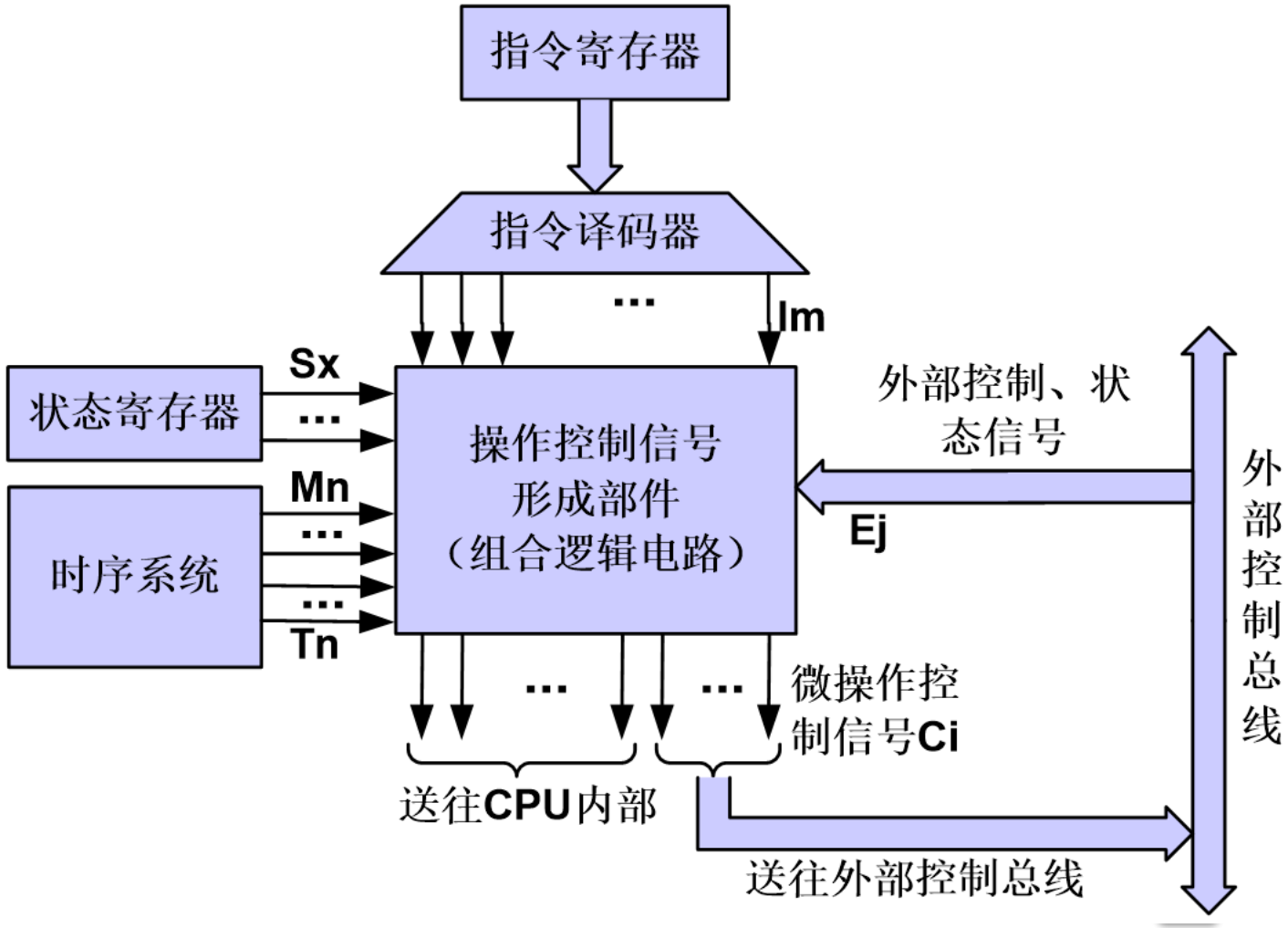
7.3.1 硬布线控制器的结构和原理

7.3.2 Yy-z02模型机的硬布线控制器设计

7.3.1 硬布线控制器的结构与原理

- 控制器的操作控制信号形成部件：由复杂的组合逻辑门电路和一些触发器构成，因此又称为组合逻辑控制器，或常规逻辑控制器。

- 基本原理：根据指令的功能、当前的时序、外部和内部的状态情况，按时间的顺序发送一系列微操作控制信号。
- 特点：速度快，设计较为繁琐、不规整，修改、扩充较难。



7.3.1 硬布线控制器的结构与原理

- **输入：**
  - 指令信息I_m：指令译码器译码产生，指出当前是哪一条指令的指令周期
  - 机器周期信号M_n、节拍信号T_n：时序系统产生，指出当前处于哪一个机器周期和哪一个节拍
  - 状态信号S_x：来自FR，指出运算器的结果状态及机器内部的其他状态，以决定某些操作信号是否发送
  - 外部控制、状态信号E_j：指出和传递CPU外部各部件的状态和控制信号。

No Image

$$C_i = f_i(I_m, M_n, T_n, S_x, E_j)$$

No Image



7.3.2 Yy-z02模型机的硬布线控制器设计

❖设计步骤：

- 1、确定指令系统、分析指令功能
- 2、确定计算机系统结构
- 3、分析指令执行过程及发送的微操作控制信号
- 4、综合每个微操作控制信号的逻辑函数
- 5、逻辑电路实现

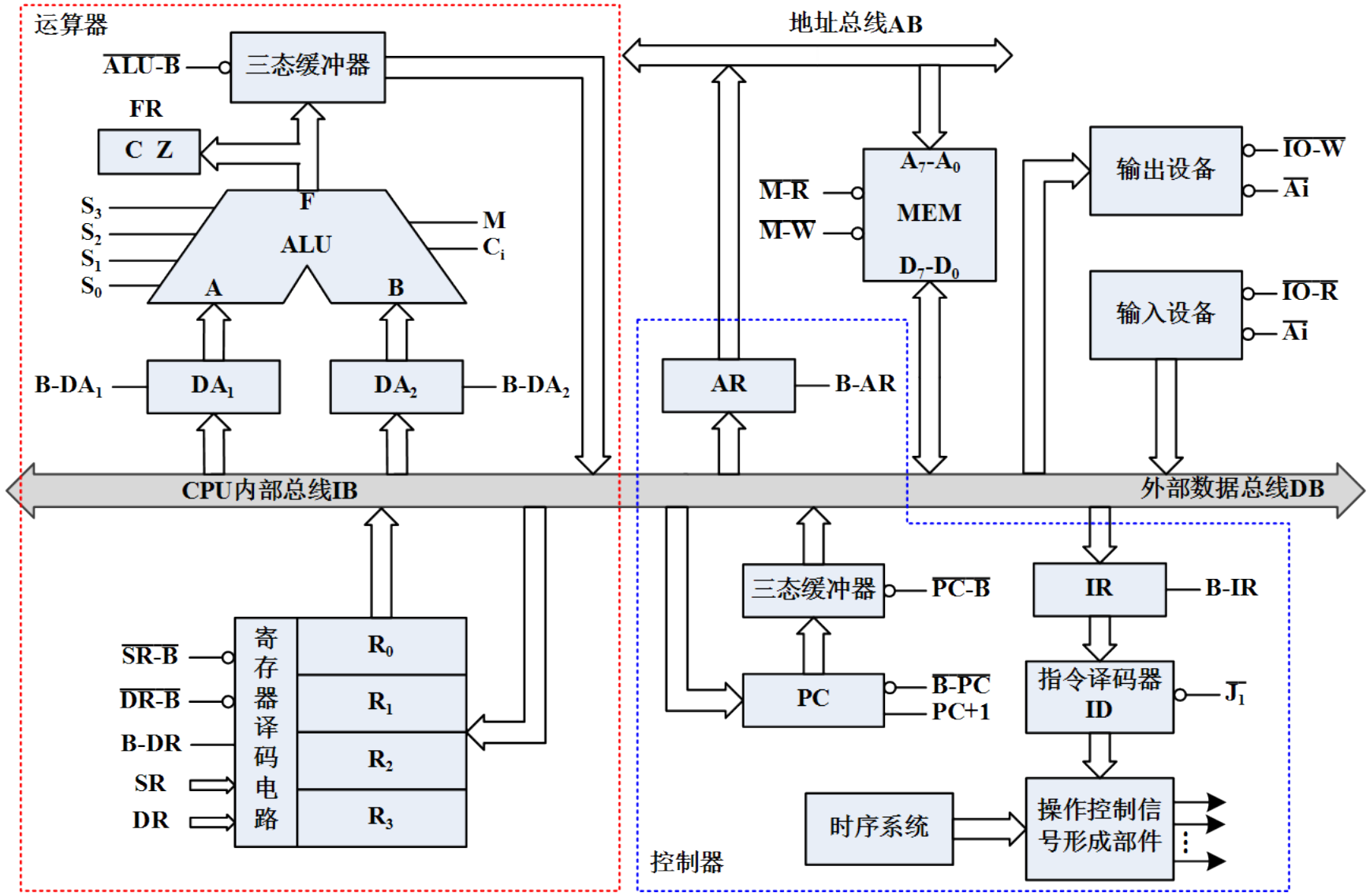
1、确定指令系统

❖两条典型指令

OP（4位）	SR（2位）	DR（2位）
Data/Addr/Disp（8位）		

助记符	格式	操作码 OP	指令功能
ADD DR, Data	OP ×× DR Data 立即数	0101	（DR）+ Data→DR
JMP Addr	OP ×××× Addr 转移地址	1000	Addr→PC

2、确定部件组成和系统结构



3、分析指令执行过程和发送的控制信号序列

指令周期阶段	机器周期：操作	机器周期：发送的控制信号序列
取指令与分析指令阶段	M_0：PC→AR，PC+1→PC	M_0：$\overline{PC-B}$，$B-AR$，$PC+1$
	M_1：Mem[AR]→IR，指令译码	M_1：$\overline{M-R}$，$B-IR$，$\overline{J_1}$
ADD DR，Data指令的执行阶段	ADD·M_2：PC→AR，PC+1→PC	ADD·M_2：$\overline{PC-B}$，$B-AR$，$PC+1$
	ADD·M_3：Mem[AR]→DA_1	ADD·M_3：$\overline{M-R}$，$B-DA_1$
	ADD·M_4：Reg[DR]→DA_2	ADD·M_4：$\overline{DR-B}$，$B-DA_2$
	ADD·M_5：DA_1+ DA_2→Reg[DR]	ADD·M_5：$S_3S_2S_1S_0MC_i=100101$，$\overline{ALU-B}$，$B-DR$
JMP Addr 指令的执行阶段	JMP·M_2：PC→AR，PC+1→PC	JMP·M_2：$\overline{PC-B}$，$B-AR$，$PC+1$
	JMP·M_3：Mem[AR]→PC	JMP·M_3：$\overline{M-R}$，$\overline{B-PC}$

4、综合与优化每个微操作控制信号的逻辑函数

（1）对所有的微操作控制信号进行综合

- 凡是在冒号“：”右边出现该信号的，把其左边的条件作为一个或项，全部进行或运算

$$\overline{PC-B}=\overline{M_0+ADD\cdot M_2+JMP\cdot M_2}$$

$$B-AR=M_0+ADD\cdot M_2+JMP\cdot M_2$$

$$PC+1=M_0+ADD\cdot M_2+JMP\cdot M_2$$

$$\overline{M-R}=\overline{M_1+ADD\cdot M_3+JMP\cdot M_3}$$

$$B-IR=M_1$$

$$\overline{J_1}=\overline{M_1}$$

$$B-DA_1=ADD\cdot M_3$$

$$B-DA_2=ADD\cdot M_4$$

$$\overline{DR-B}=\overline{ADD\cdot M_4}$$

$$S_3=S_0=C_i=ADD\cdot M_5$$

$$S_2=S_1=M=\overline{ADD\cdot M_5}$$

$$\overline{ALU-B}=\overline{ADD\cdot M_5}$$

$$B-DR=ADD\cdot M_5$$

$$\overline{B-PC}=\overline{JMP\cdot M_3}$$

4、综合与优化每个微操作控制信号的逻辑函数

❖ （2）对逻辑函数进行化简与优化

- ①从逻辑代数的角度来化简
- ②从指令系统的整体逻辑关系上来考虑

$$\overline{PC-B}=\overline{M_0+ADD\cdot M_2+JMP\cdot M_2}$$

$$B-AR=M_0+ADD\cdot M_2+JMP\cdot M_2$$

$$PC+1=M_0+ADD\cdot M_2+JMP\cdot M_2$$

$$\overline{M-R}=\overline{M_1+ADD\cdot M_3+JMP\cdot M_3}$$

❖ 假设：指令系统中只有ADD和JMP这两条指令

❖ 可以理解为$ADD+JMP=1$

$$\overline{PC-B}=\overline{M_0+M_2}$$

$$B-AR=M_0+M_2$$

$$PC+1=M_0+M_2$$

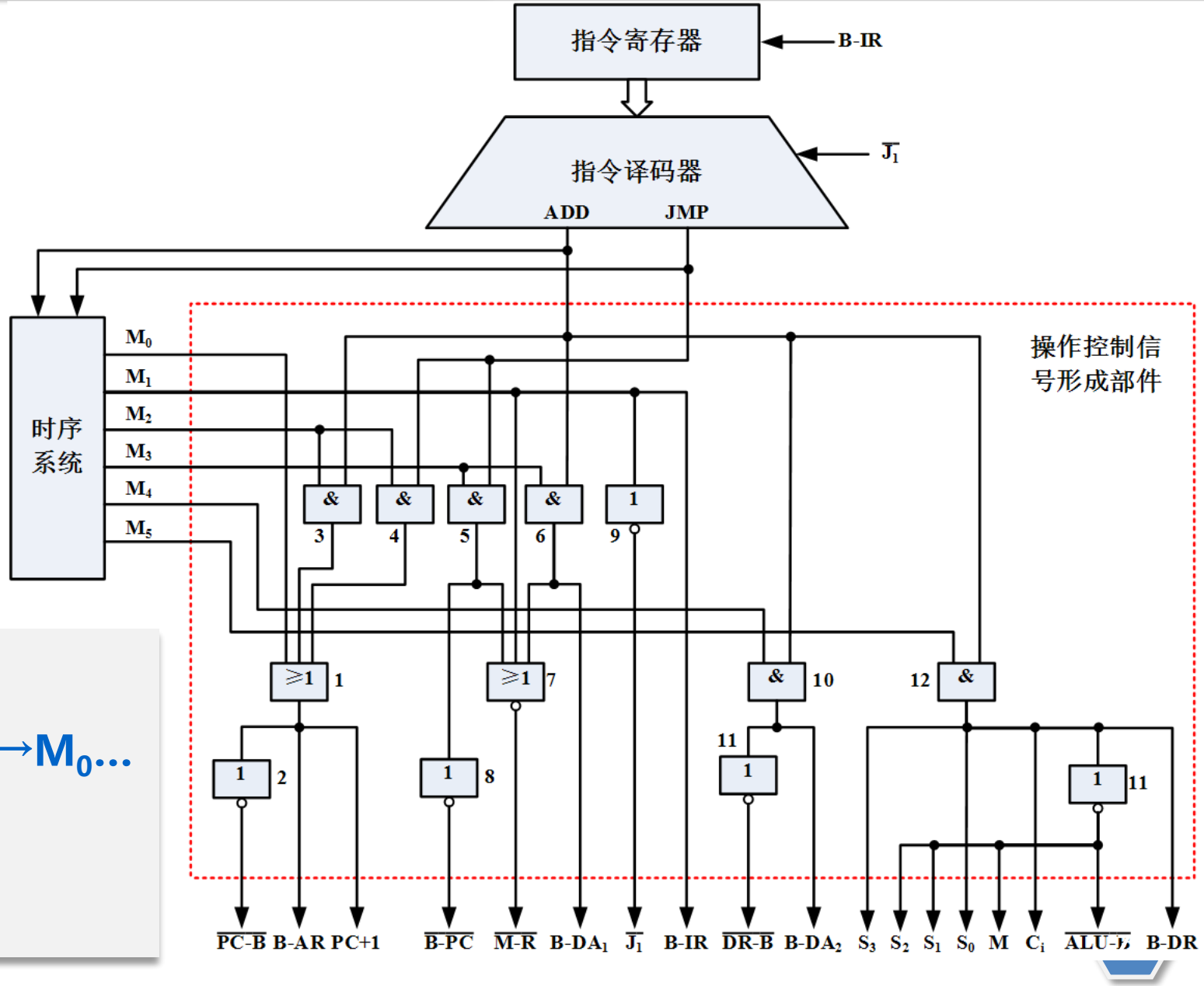
$$\overline{M-R}=\overline{M_1+M_3}$$

- 如果某个微操作控制信号在规定的T_n节拍有效，则该信号还要和T_n信号进行“与”操作
- 当指令系统被扩充或者修改时，需要按照上述方法写出新指令的机器周期序列和微操作控制信号序列，更新相关微操作控制信号的逻辑函数

$$B-AR=(M_0+M_2)\cdot T_3$$

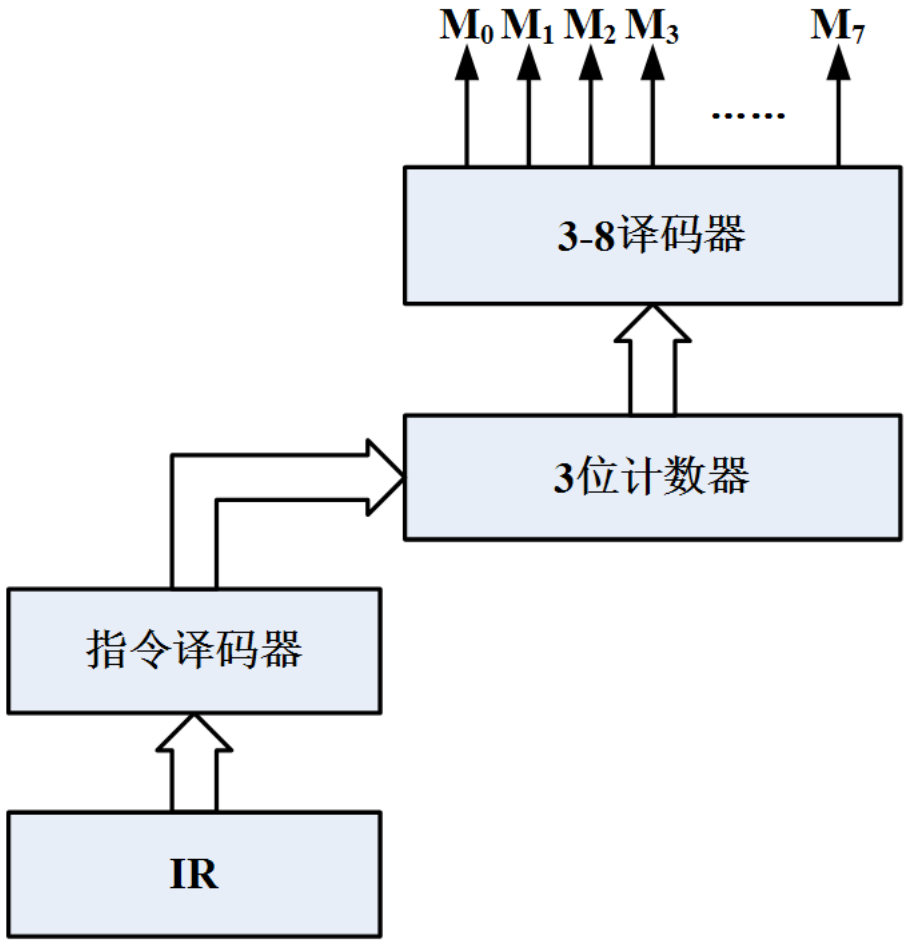
$$\overline{B-PC}=\overline{JMP\cdot M_3\cdot T_4}$$

5、实现逻辑函数

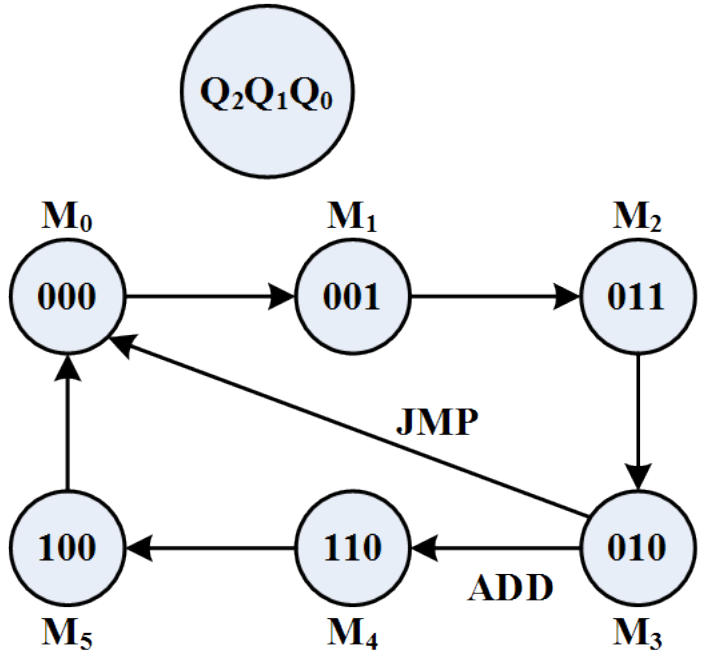


- ❖ **ADD指令：**
  - $M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_4 \rightarrow M_5 \rightarrow M_0 \cdots$
- ❖ **JMP指令：**
  - $M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_0 \cdots$

时序系统怎么产生不同的机器周期信号序列?

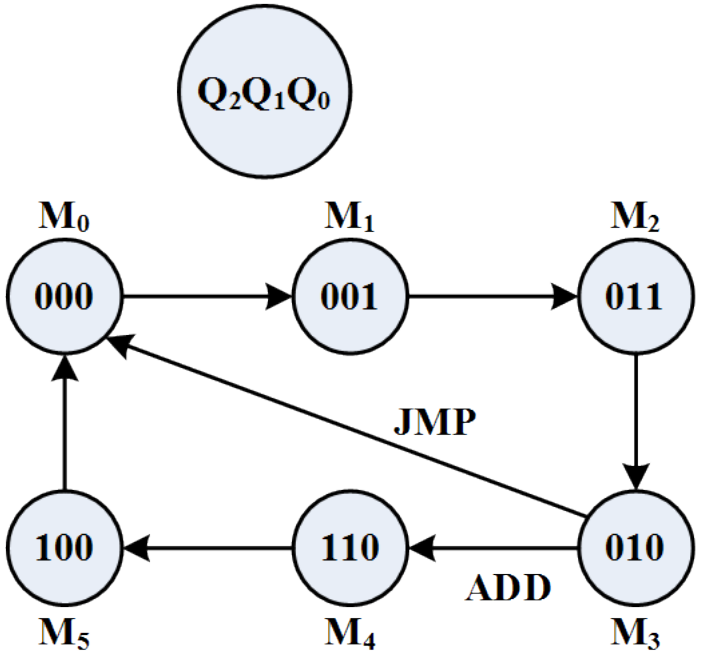


- **ADD指令:**
  - $M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_4 \rightarrow M_5 \rightarrow M_0 \ldots$
- **JMP指令:**
  - $M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow M_0 \ldots$



时序系统怎么产生不同的机器周期信号序列?

❖假设计数器用D触发器实现



$$D_2 = ADD \cdot Q_1^n \cdot \overline{Q_0^n}$$

$$D_1 = Q_0^n + ADD \cdot \overline{Q_2^n} \cdot Q_1^n$$

$$D_0 = \overline{Q_2^n} \cdot \overline{Q_1^n}$$

输入（指令信号和原态）					输出（次态）		
ADD	JMP	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	×	×	×	×	×	×
0	1	0	0	0	0	0	1
0	1	0	0	1	0	1	1
0	1	0	1	0	0	0	0
0	1	0	1	1	0	1	0
0	1	1	0	0	×	×	×
0	1	1	0	1	×	×	×
0	1	1	1	0	×	×	×
0	1	1	1	1	×	×	×
1	0	0	0	0	0	0	1
1	0	0	0	1	0	1	1
1	0	0	1	0	1	1	0
1	0	0	1	1	0	1	0
1	0	1	0	0	0	0	0
1	0	1	0	1	×	×	×
1	0	1	1	0	1	0	0
1	0	1	1	1	×	×	×
1	1	×	×	×	×	×	×

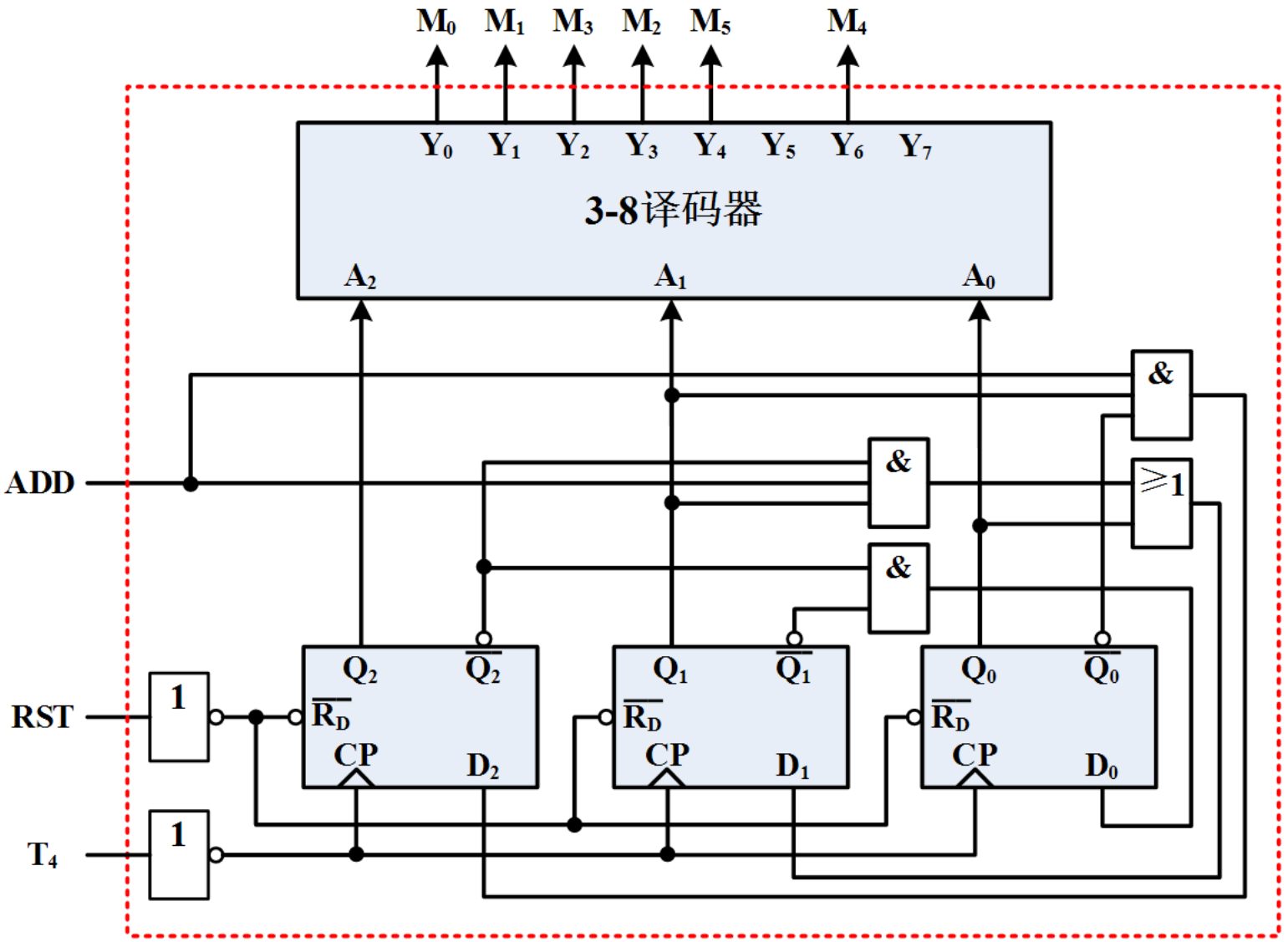
时序系统怎么产生不同的机器周期信号序列?

❖时序系统的设计

$$D_2 = ADD \cdot Q_1^n \cdot \overline{Q_0^n}$$

$$D_1 = Q_0^n + ADD \cdot \overline{Q_2^n} \cdot Q_1^n$$

$$D_0 = \overline{Q_2^n} \cdot \overline{Q_1^n}$$



指令如何被执行?

① 开机上电→硬件产生RST信号

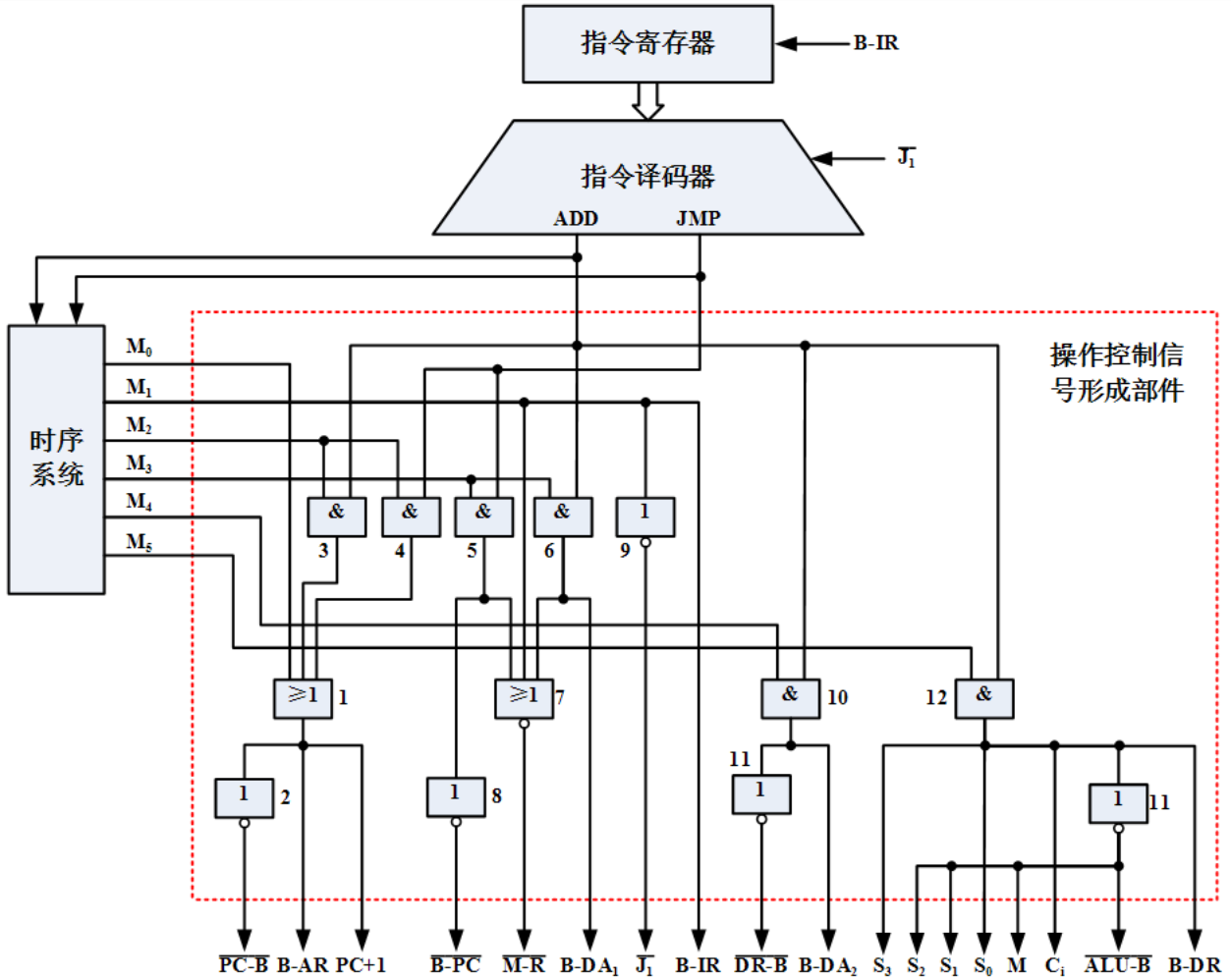
- PC置初值(第一条指令在存储器中的地址)
- 时序电路开始工作，机器周期计数器被清零，即产生的第一个机器周期信号为M_0

② M_0信号→操作控制信号形成部件

- 1号或门输出1，2号非门输出0
- 信号$\overline{PC-B}=0$、$B-AR=1$、$PC+1=1$
- 执行操作：PC→AR，PC+1→PC

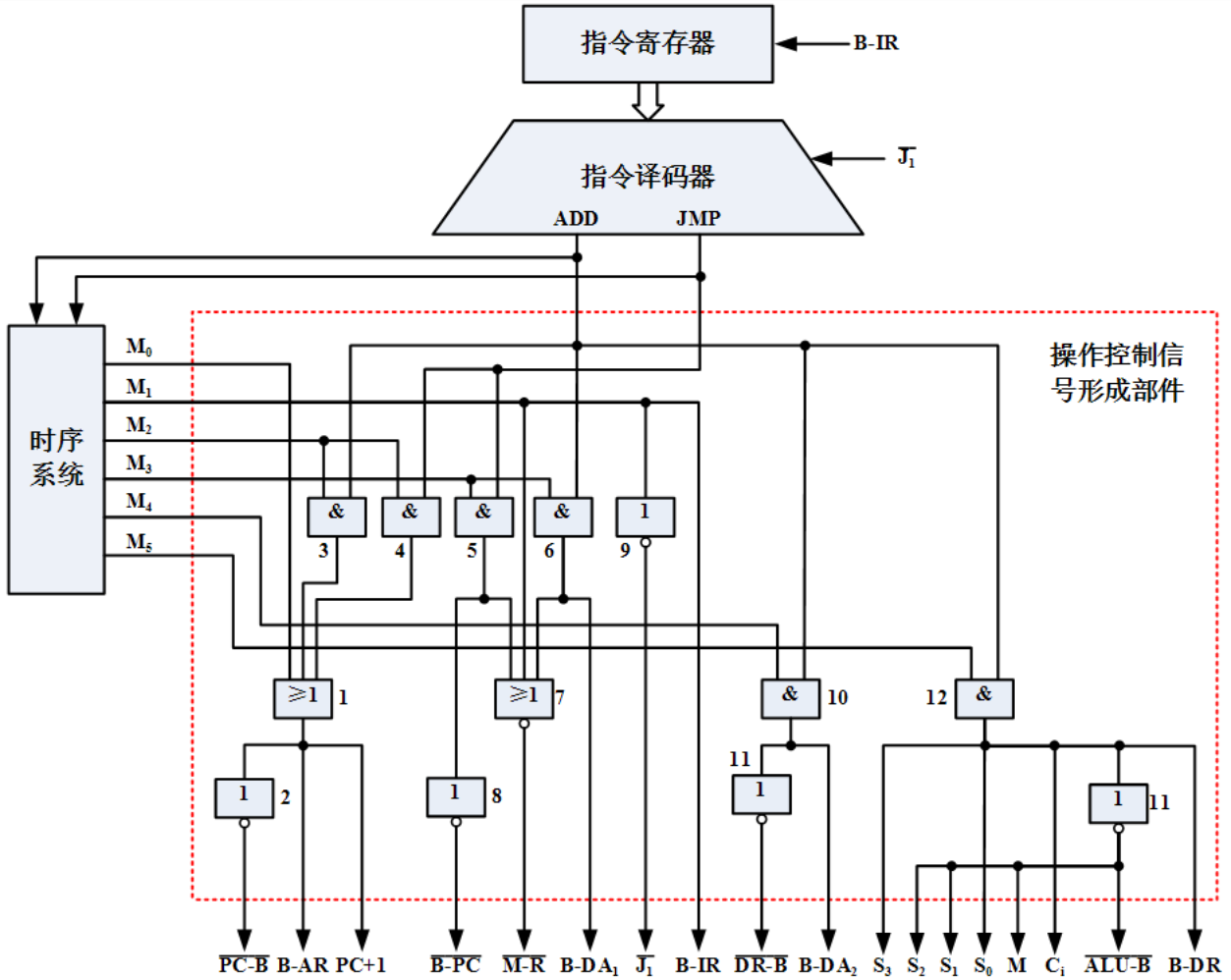
③ M_0结束→进入M_1机器周期

- 7号或非门输出0，9号非门输出0
- 信号$\overline{M-R}=0$、$B-IR=1$、$\overline{J_1}=0$
- 执行操作：Mem[AR]→IR，指令译码



指令如何被执行?

No Image



计算机的工作过程：循环往复地取指令、分析指令、执行指令

硬布线控制器的特点

- 微操作控制信号由 组合逻辑电路 即时产生
- 硬布线控制器电路设计较为繁琐、不规整
- 硬布线控制器非常不利于指令的修改和扩充
  - 现代微电子设计技术的自动化程度日益增高，极大地弥补了这个缺陷
- 执行速度快，节省芯片面积
- 硬布线控制器多应用于RISC系统

- 一个CPU的控制器，可以部分指令使用硬布线来实现，部分指令（一般是复杂指令）使用微程序控制来实现。
- RISC CPU多采用硬布线控制器来实现，而现代CISC CPU也采用了一些RISC的技术，包括：简单指令使用硬布线来实现，复杂指令使用微程序控制来实现



7.4 微程序控制器

- 7.4.1 微程序控制的基本概念
- 7.4.2 微程序控制器的组成与结构
- 7.4.3 微程序控制器的设计方法
- 7.4.4 微程序设计技术
- 7.4.5 微程序控制器与硬布线控制器的比较

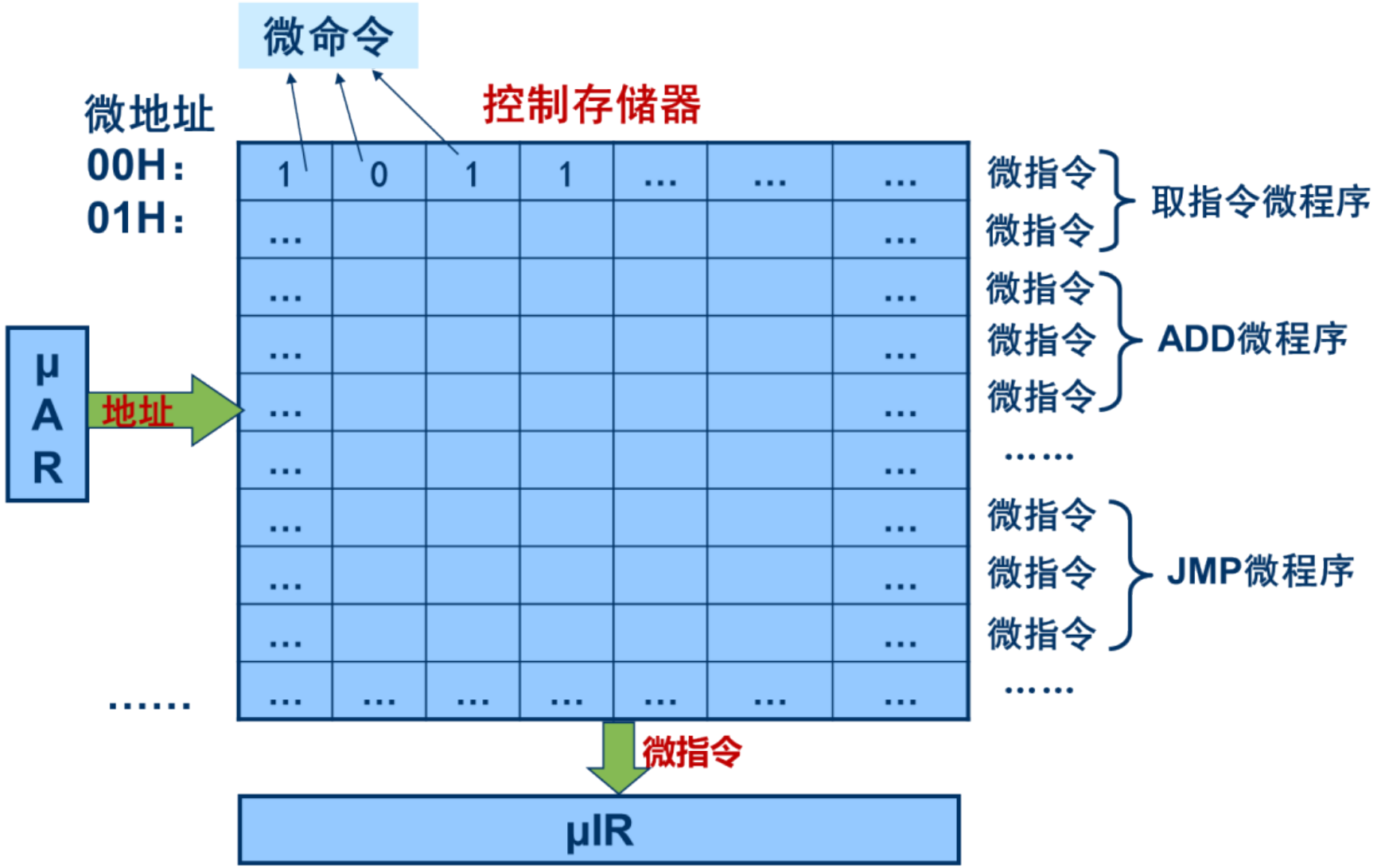
7.4.1微程序控制的基本概念

- 微程序设计思想：每条机器指令的功能都用一段相应的微程序来实现。
- 微程序：微指令构成的有序集合，用于实现机器指令的功能；一条指令对应着一段微程序
- 微指令：一组微命令构成的集合，用于完成一个功能相对完整的操作。例如存储器的读操作或写操作、ALU的一次运算或者总线的一次传送等
- 微命令：组成微指令的最小单位，是控制实现微操作的控制信号；一般用于控制数据通路上门的打开/关闭，寄存器的打入脉冲或者功能选择；一个微命令就完成一个微操作。
- 微操作：微命令执行时产生的动作。
- 例如 $\overline{PC-B}$ 是一个微命令，它对应完成的微操作就是：将PC的内容送到总线上。

7.4.1微程序控制的基本概念

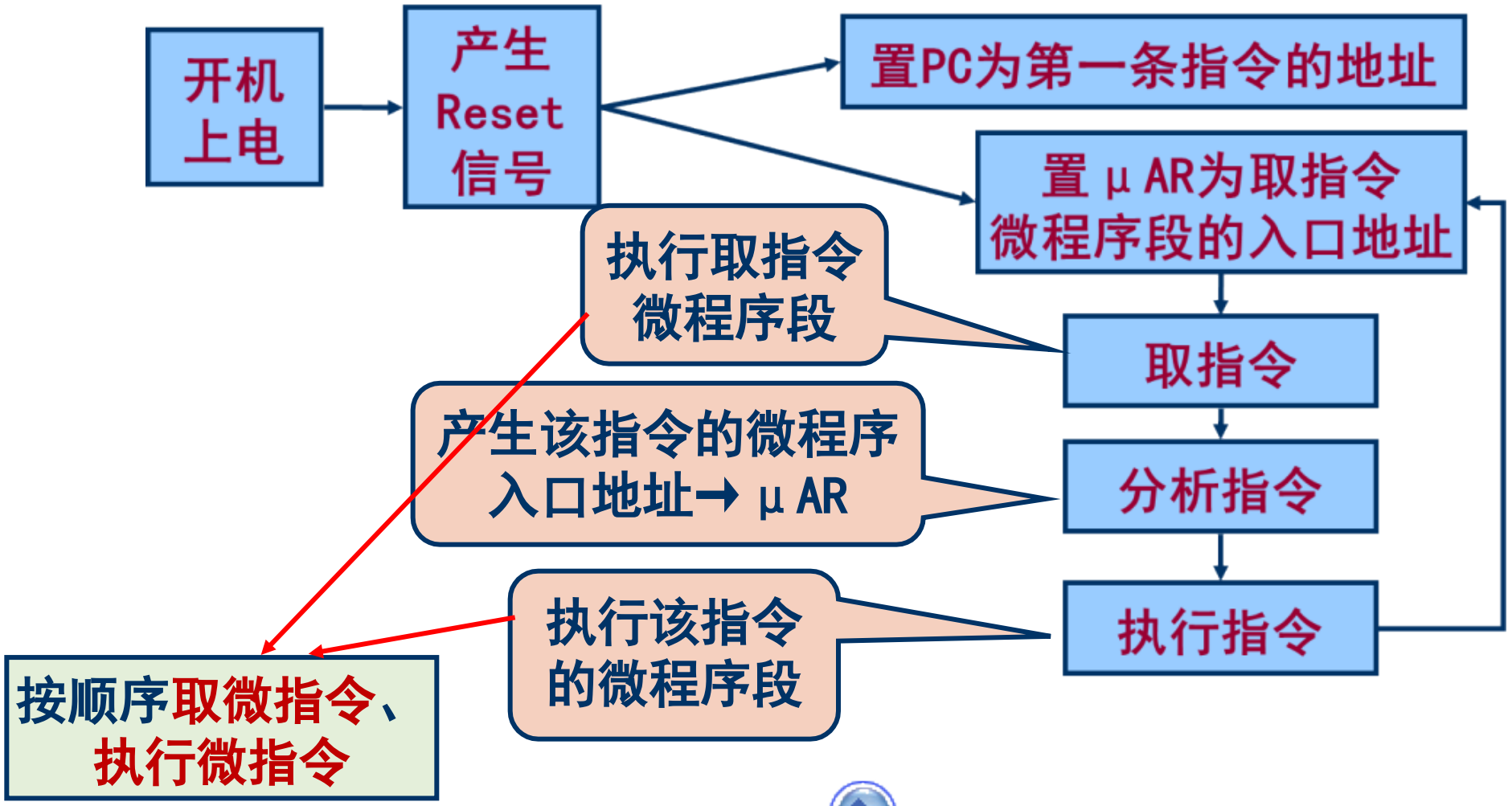
- 控制存储器：简称控存，用于存放所有指令的微程序，其中一个存储单元存放一条微指令。一般为ROM
- 微地址：微指令在控存中的地址
- 微地址寄存器μAR：存放微地址的寄存器，用于提供访问控存CM的地址，又称为控存地址寄存器CMAR（Control Memory Address Register）。
- 微指令寄存器μIR：存放从控存取出的微指令的寄存器
- 微周期：指从控存中取出并执行一条微指令所需要的时间，一般与一个机器周期相当

微程序基本概念关系


微命令
控制存储器
微地址
00H：
01H：
1 0 1 1 … … …
μAR
地址
微指令
微指令
取指令微程序
微指令
微指令
微指令
ADD微程序
……
微指令
微指令
微指令
JMP微程序
……
……
微指令
μIR

微程序控制的计算机工作原理

❖微程序控制的计算机工作过程：



7.4.2微程序控制器的组成与结构

❖1、微程序控制器的组成部件

- 其他部件均等同于硬布线控制器
- 操作控制信号形成部件：不同，由4个部件构成

① 控制存储器CM

② 微地址寄存器μAR

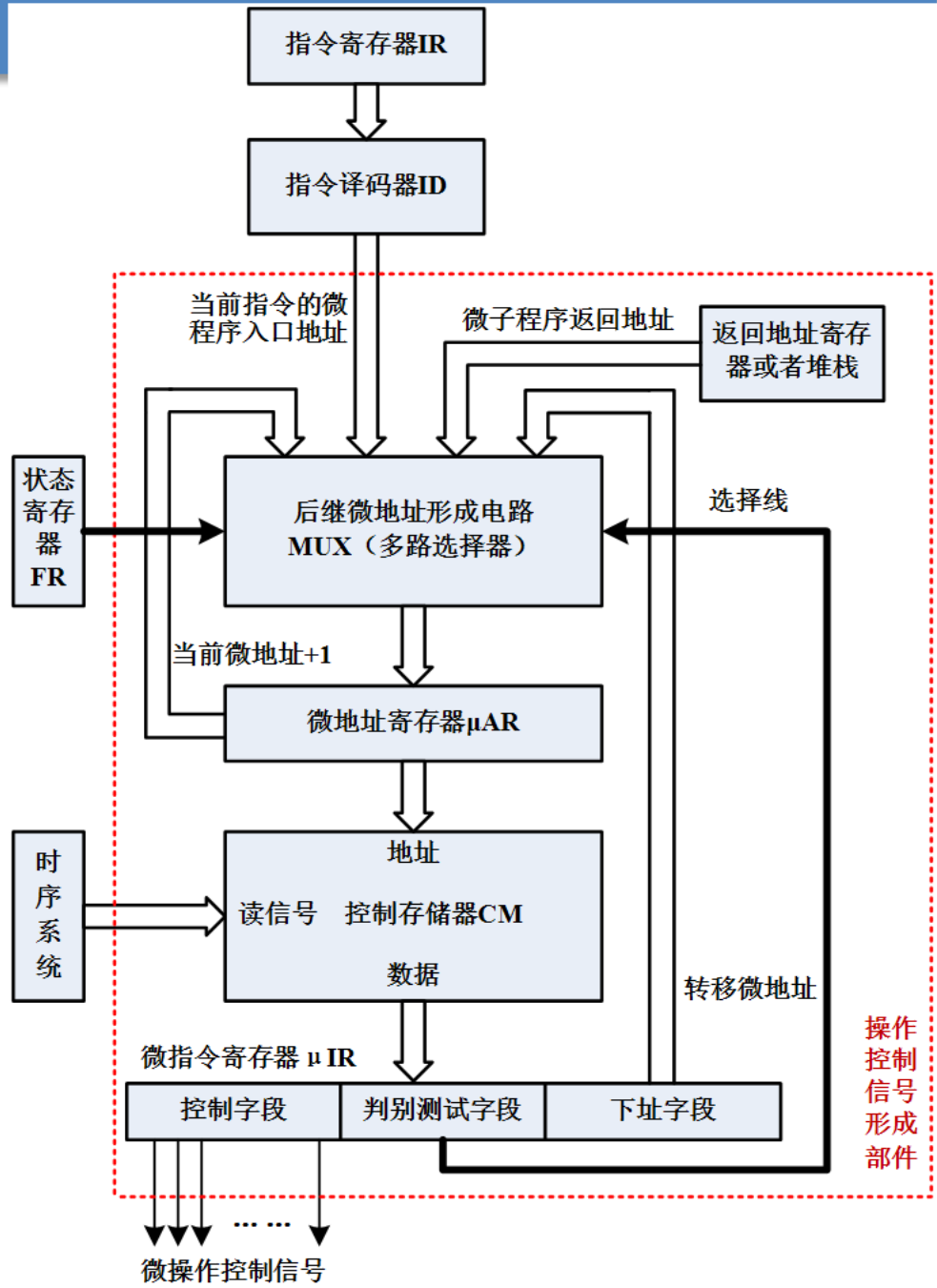
③ 微指令寄存器μIR

④ 后继微地址形成电路：多路选择器

❖取微指令：CM[μAR]→ μIR

❖执行微指令： μIR 控制字段产生微操作控制信号

❖下一条微指令地址：由后继微地址形成电路产生



7.4.2微程序控制器的组成与结构

❖ 2、微指令格式

- **控制字段：包含了一组微命令信号，用于控制完成本条微指令的操作。**
  - **用于产生：CPU以及整机的所有微操作控制信号**
  - **可以直接产生，或者经过译码产生**
- **判别测试字段：指出本条微指令发生转移的类别，以确定后继微地址。**
- **下址字段：用于指出后继微地址（下条微指令地址）的相关信息。**



7.4.2微程序控制器的组成与结构

❖ 3、后继微地址的来源：

① **自增1：** 后继微地址=当前微地址+1，用于顺序执行微程序的场合

② **指令译码器产生：** 由IR中操作码产生该指令对应的微程序入口地址，主要用于指令译码（宏操作映射）

- 实现方法：MAPROM或者逻辑电路

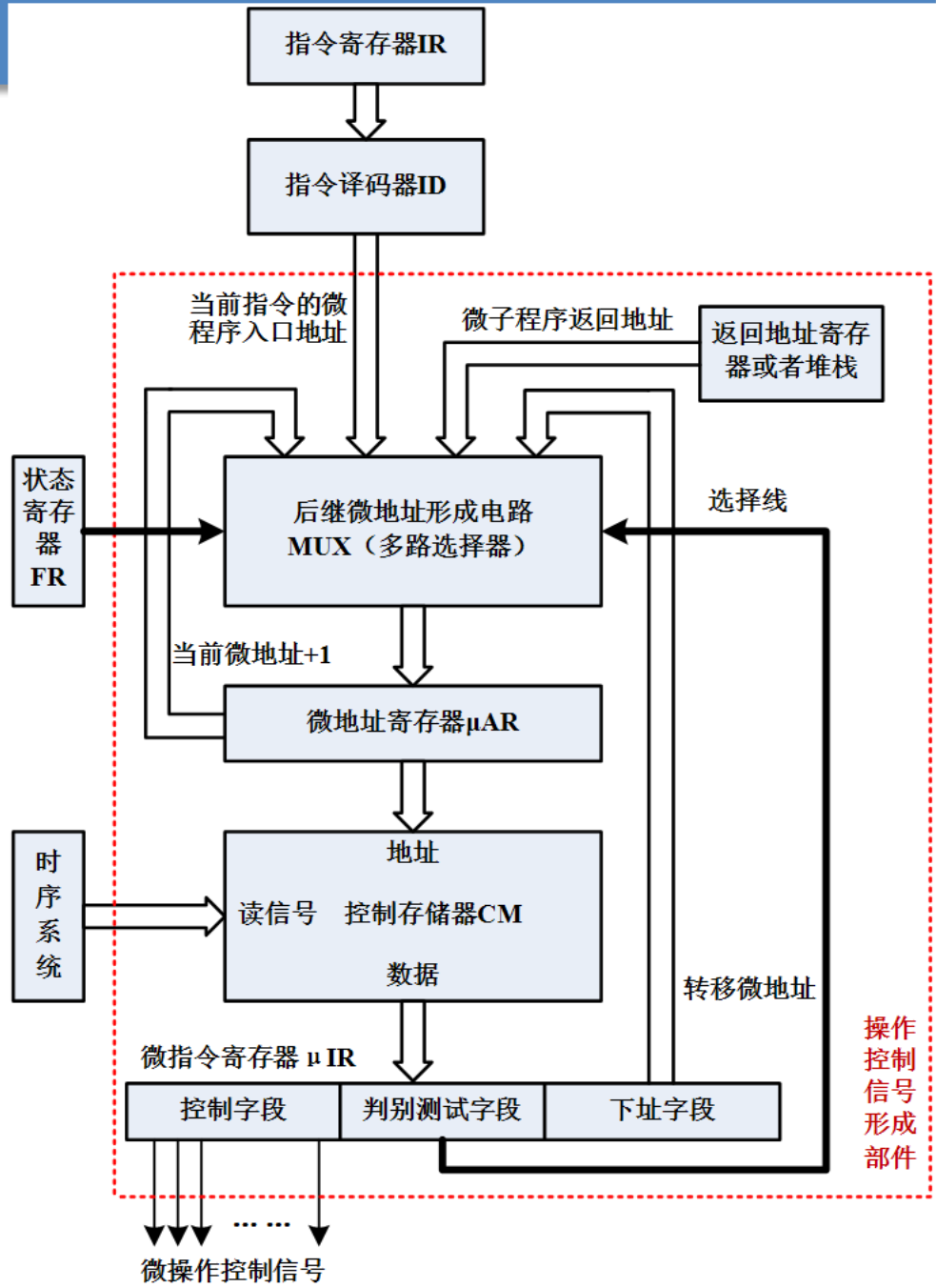
③ **下址字段产生：** 由当前微指令的下址字段指定，可以是全地址或者部分地址，用于微程序转移的场合。

- 条件/无条件转移、微子程序调用

④ **微子程序返回地址：** 由微子程序寄存器和堆栈产生，用于微子程序的返回

❖ 如何确定下一条微指令的地址来源?

❖ 判别测试字段：用于指明后继微地址的来源



7.4.3微程序控制器的设计方法

- 微程序控制器的设计：主要完成两个任务
  - 产生正确的微命令
  - 产生正确的微指令序列（即CPU状态转换序列）
- 怎样采用微程序控制的方法来设计CPU呢?
  - 1、确定指令系统
  - 2、确定部件组成和系统结构
  - 3、分析每条指令的执行过程，画出微程序流程图
  - 4、写出每条微指令所发送的微操作控制信号序列
  - 5、设计微指令格式及微程序控制器结构
  - 6、分配微地址和编写微指令代码
  - 7、将所有的微指令代码装入控制存储器的相应单元

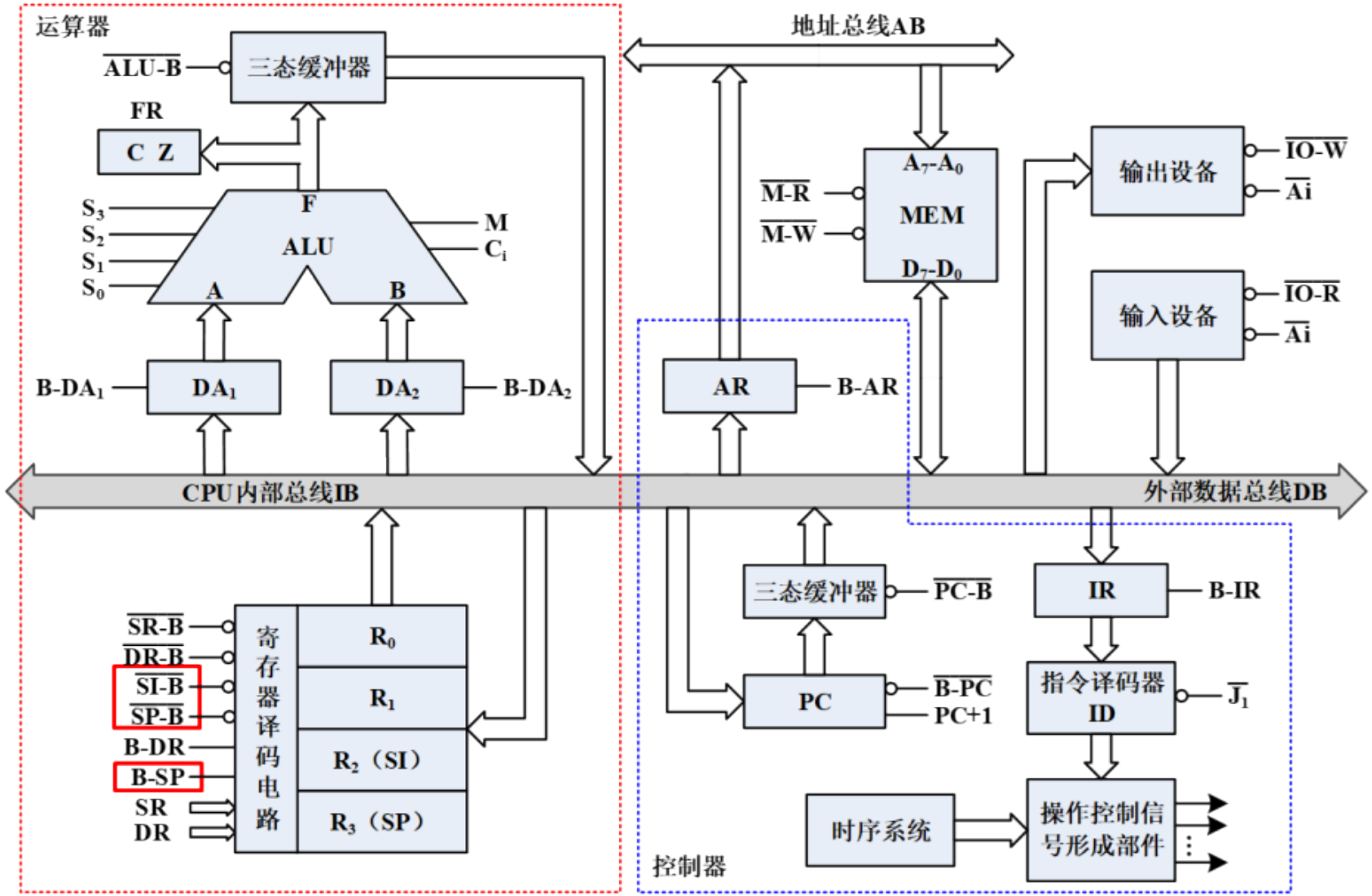
7.4.3微程序控制器的设计方法

❖1、确定指令系统：ADD和JMP

助记符	格式	操作码OP	指令功能
ADD DR, Data	I_7 I_6 I_5 I_4 I_3 I_2 I_1 I_0 OP(4位) ××(2位) DR(2位) Data(8位立即数)	0101	Reg[DR]+Data→Reg[DR]
JMP Addr	I_7 I_6 I_5 I_4 I_3 I_2 I_1 I_0 OP(4位) ×××× (4位) Addr(8位直接转移地址)	1000	Addr→PC

7.4.3微程序控制器的设计方法

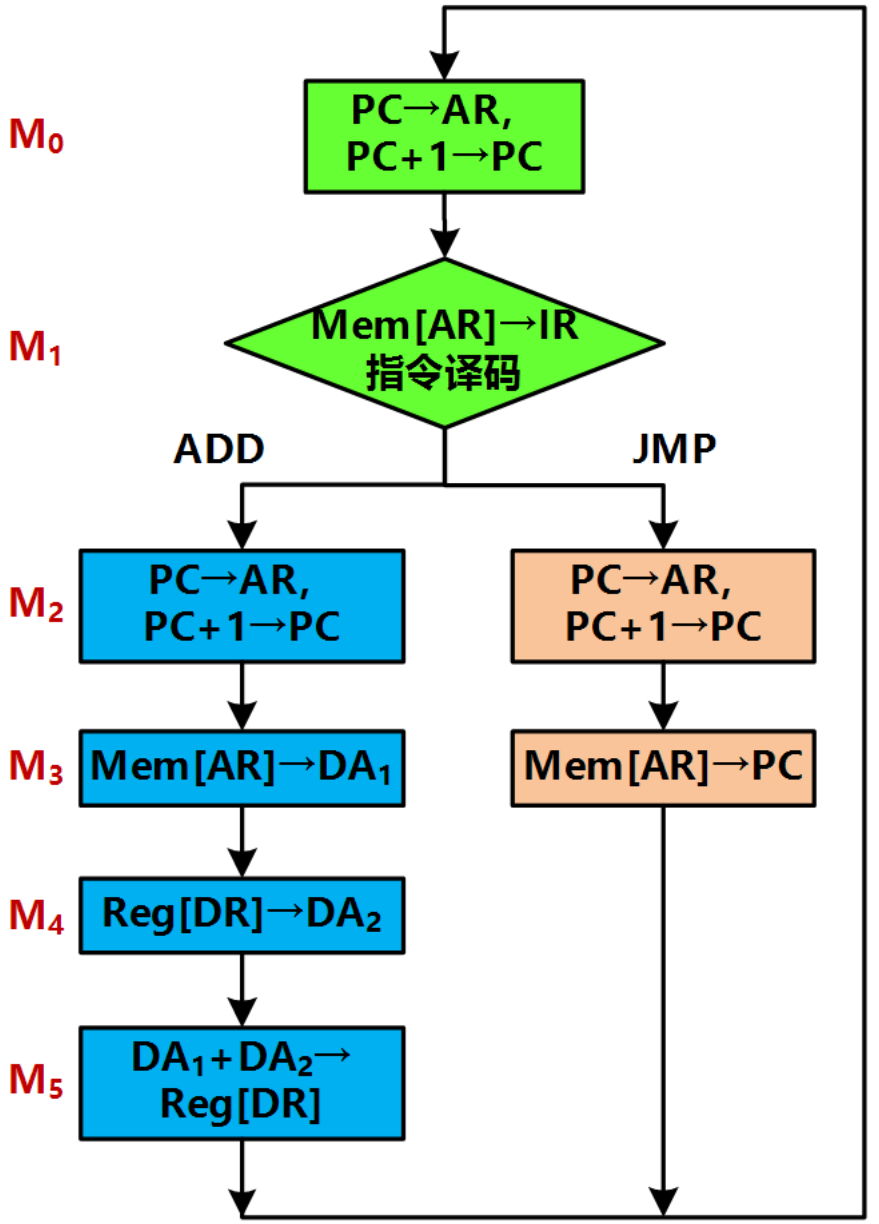
❖2、确定部件组成和系统结构



No Image

7.4.3微程序控制器的设计方法

- 3、分析每条指令的执行过程，画出微程序流程图
  - ADD指令：6个机器周期
  - JMP指令：4个机器周期
  - 每个方框：一条微指令、一个CPU状态
  - 该模型机：共8条微指令
  - CPU：有8个状态
  - 控存：有8个单元



7.4.3微程序控制器的设计方法

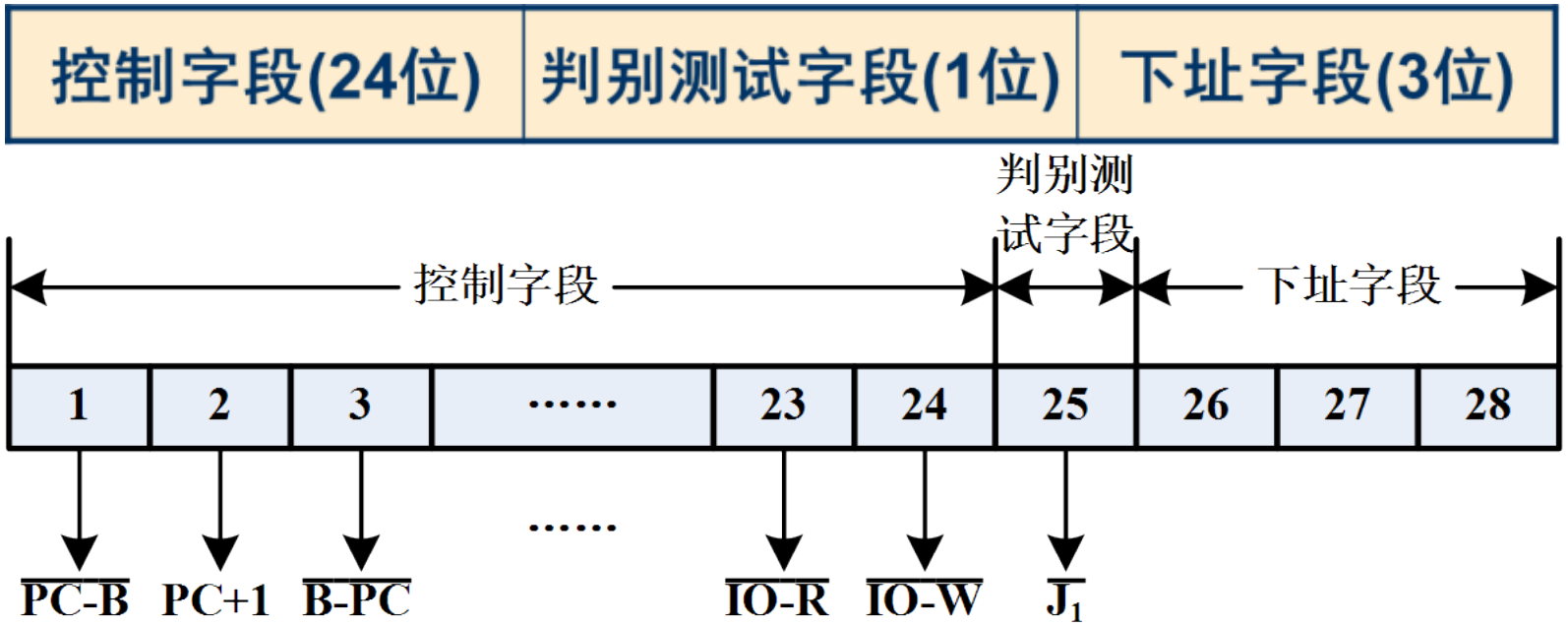
❖4、写出每条微指令所发送的微操作控制信号序列

微程序段	微周期	微指令	发送的微命令序列
取指令微程序段	M_0	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
	M_1	Mem[AR]→IR，指令译码	$\overline{M-R}$，$B-IR$，$\overline{J_1}$
ADD DR，Data指令的微程序段	ADD·M_2	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
	ADD·M_3	Mem[AR]→DA_1	$\overline{M-R}$，$B-DA_1$
	ADD·M_4	Reg[DR]→DA_2	$\overline{DR-B}$，$B-DA_2$
	ADD·M_5	DA_1+ DA_2→Reg[DR]	$S_3S_2S_1S_0MC_i=100101,\overline{ALU-B}$，$B-DR$
JMP Addr指令的微程序段	JMP·M_2	PC→AR，PC+1→PC	$\overline{PC-B}$，$B-AR$，$PC+1$
	JMP·M_3	Mem[AR]→PC	$\overline{M-R}$，$\overline{B-PC}$

7.4.3微程序控制器的设计方法

❖5、设计微指令格式及微程序控制器结构

❖（1）微指令格式



No Image

No Image

控制字段——控制信号定义

位序	控制信号	功能	位序	控制信号	功能
1	$\overline{PC-B}$	PC内容送总线	14	$\overline{ALU-B}$	ALU运算结果送总线
2	$PC+1$	PC内容自增1	15	$\overline{SR-B}$	SR寄存器内容送总线
3	$\overline{B-PC}$	总线上内容打入PC	16	$\overline{DR-B}$	DR寄存器内容送总线
4	$B-IR$	总线上内容打入IR	17	$\overline{SI-B}$	SI寄存器（R_2）内容送总线
5	$B-AR$	总线上内容打入AR	18	$\overline{SP-B}$	SP寄存器（R_3）内容送总线
6	$B-DA_1$	总线上内容打入DA_1	19	$B-DR$	总线上数据打入DR寄存器
7	$B-DA_2$	总线上内容打入DA_2	20	$B-SP$	总线上数据打入SP（R_3）
8	S_3	选择ALU运算功能	21	$\overline{M-R}$	存储器执行读操作
9	S_2	选择ALU运算功能	22	$\overline{M-W}$	存储器执行写操作
10	S_1	选择ALU运算功能	23	$\overline{IO-R}$	输入设备执行读操作
11	S_0	选择ALU运算功能	24	$\overline{IO-W}$	输出设备执行写操作
12	M	选择算术运算/逻辑运算	25	$\overline{J_1}$	指令译码器工作
13	C_i	进位输入			

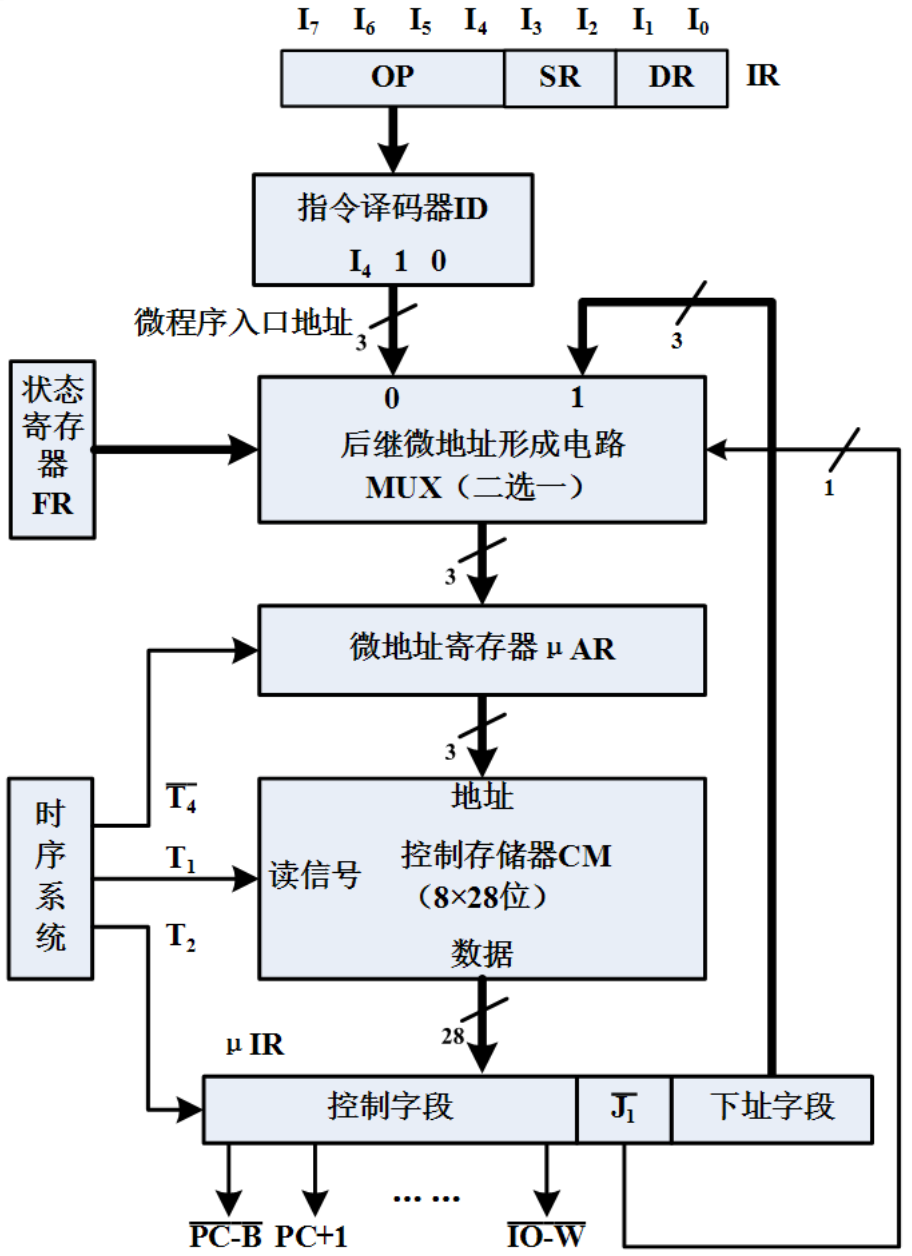
7.4.3微程序控制器的设计方法

No Image

- ADD指令：OP=0101
  - 微程序入口地址=110
- JMP指令：OP=1000
  - 微程序入口地址=010

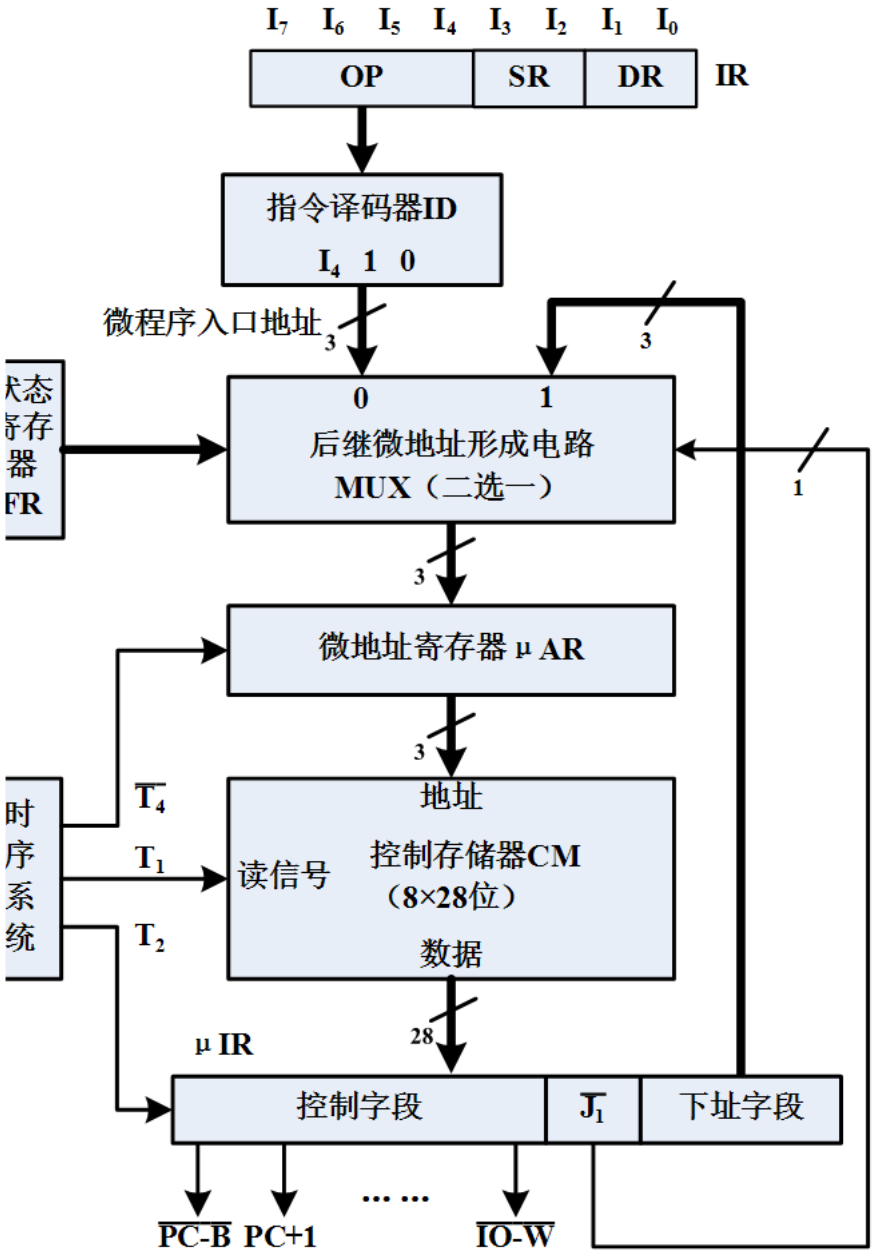
7.4.3微程序控制器的设计方法

No Image



7.4.3微程序控制器的设计方法

No Image



7.4.3微程序控制器的设计方法

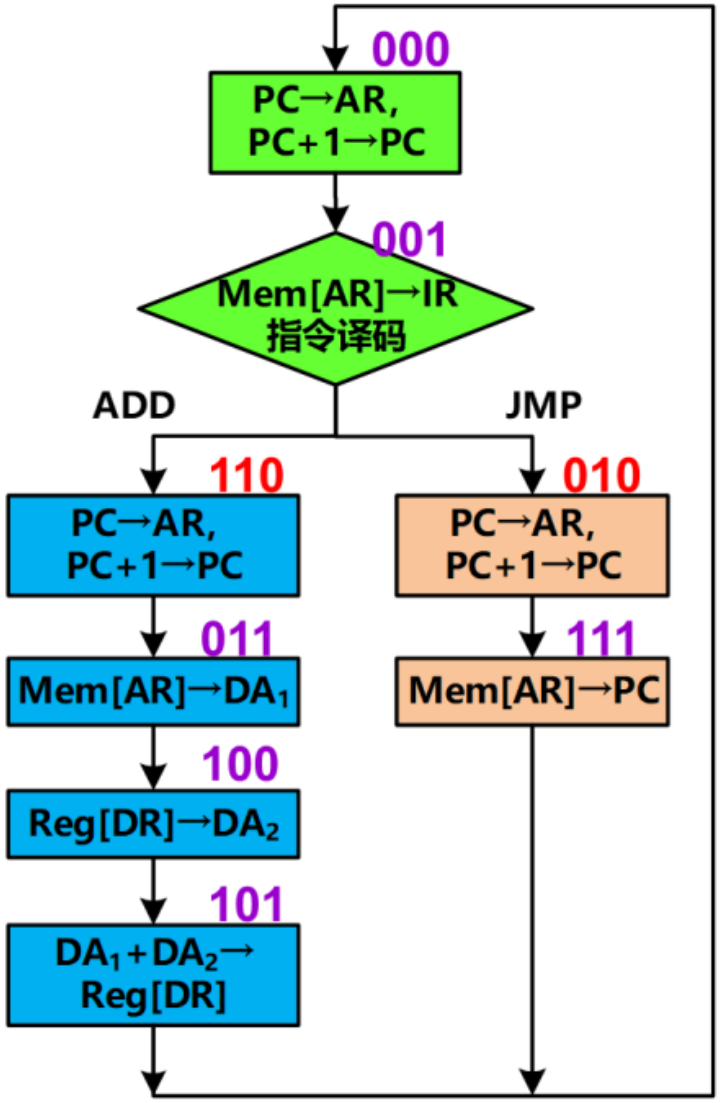
❖6、***分配微地址***，并编写微指令代码

❖①确定各条指令的微程序入口

- ADD指令的入口地址=110
- JMP指令的入口地址=010

❖②安排取指令微程序段：000、001

❖③安排其他微指令

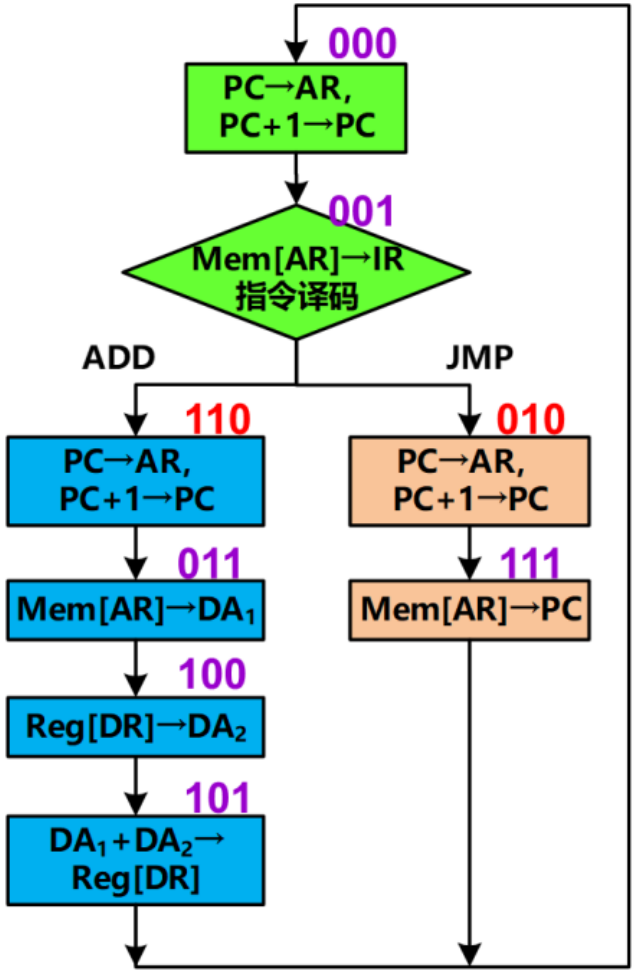


7.4.3微程序控制器的设计方法

❖6、分配微地址，*并编写微指令代码*

- ①按照分配的微地址，列出各条微指令对应的判别测试字段和下址字段

微地址	微指令	判别测试字段$\overline{J_1}$	下址字段
000	M_0：PC→AR，PC+1→PC	1	001
001	M_1：Mem[AR]→IR，指令译码	0	×××
010	JMP·M_2：PC→AR，PC+1→PC	1	111
011	ADD·M_3：Mem[AR]→DA_1	1	100
100	ADD·M_4：Reg[DR]→DA_2	1	101
101	ADD·M_5：DA_1+ DA_2→Reg[DR]	1	000
110	ADD·M_2：PC→AR，PC+1→PC	1	011
111	JMP·M_3：Mem[AR]→PC	1	000



7.4.3微程序控制器的设计方法

❖6、分配微地址，*并编写微指令代码*

- ②按照微指令的控制字段的设计，写出控制字段的编码

位序	控制信号	位序	控制信号
1	$\overline{PC-B}$	14	$\overline{ALU-B}$
2	$PC+1$	15	$\overline{SR-B}$
3	$\overline{B-PC}$	16	$\overline{DR-B}$
4	$B-IR$	17	$\overline{SI-B}$
5	$B-AR$	18	$\overline{SP-B}$
6	$B-DA_1$	19	$B-DR$
7	$B-DA_2$	20	$B-SP$
8	S_3	21	$\overline{M-R}$
9	S_2	22	$\overline{M-W}$
10	S_1	23	$\overline{IO-R}$
11	S_0	24	$\overline{IO-W}$
12	M	25	$\overline{J_1}$
13	C_i		

微地址	控制字段发送的控制信号	$\overline{J_1}$	下址字段
000	M_0: $\overline{PC-B}$, $B-AR$, $PC+1$	1	001
001	M_1: $\overline{M-R}$, $B-IR$, $\overline{J_1}$	0	×××
010	JMP•M_2: $\overline{PC-B}$, $B-AR$, $PC+1$	1	111
011	ADD•M_3: $\overline{M-R}$, $B-DA_1$	1	100
100	ADD•M_4: $\overline{DR-B}$, $B-DA_2$	1	101
101	ADD•M_5: $S_3S_2S_1S_0MC_i=100101$,$\overline{ALU-B}$, $B-DR$	1	000
110	ADD•M_2: $\overline{PC-B}$, $B-AR$, $PC+1$	1	011
111	JMP•M_3: $\overline{M-R}$, $\overline{B-PC}$	1	000

7.4.3微程序控制器的设计方法

❖6、分配微地址，*并编写微指令代码*

- ②按照微指令的控制字段的设计，写出控制字段的编码

❖ 发送某个信号：对应的控制位=有效电平

❖ 不发送某个信号：对应的控制位=无效电平

位序	控制信号	位序	控制信号
1	$\overline{PC-B}$	14	$\overline{ALU-B}$
2	$PC+1$	15	$\overline{SR-B}$
3	$\overline{B-PC}$	16	$\overline{DR-B}$
4	$B-IR$	17	$\overline{SI-B}$
5	$B-AR$	18	$\overline{SP-B}$
6	$B-DA_1$	19	$B-DR$
7	$B-DA_2$	20	$B-SP$
8	S_3	21	$\overline{M-R}$
9	S_2	22	$\overline{M-W}$
10	S_1	23	$\overline{IO-R}$
11	S_0	24	$\overline{IO-W}$
12	M	25	$\overline{J_1}$
13	C_i		

微地址	微指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
	位序	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
	空操作	1	0	1	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	1	1	1	1				
000	M_0	0	1			1																				1	0	0	1
001	M_1				1																	0				0	×	×	×
010	$JMP \cdot M_2$	0	1			1																				1	1	1	1
011	$ADD \cdot M_3$						1															0				1	1	0	0
100	$ADD \cdot M_4$							1									0									1	1	0	1
101	$ADD \cdot M_5$								1	0	0	1	0	1	0					1						1	0	0	0
110	$ADD \cdot M_2$	0	1			1																				1	0	1	1
111	$JMP \cdot M_3$			0																		0				1	0	0	0

7.4.3微程序控制器的设计方法

❖6、分配微地址，*并编写微指令代码*

- ③写出微指令编码

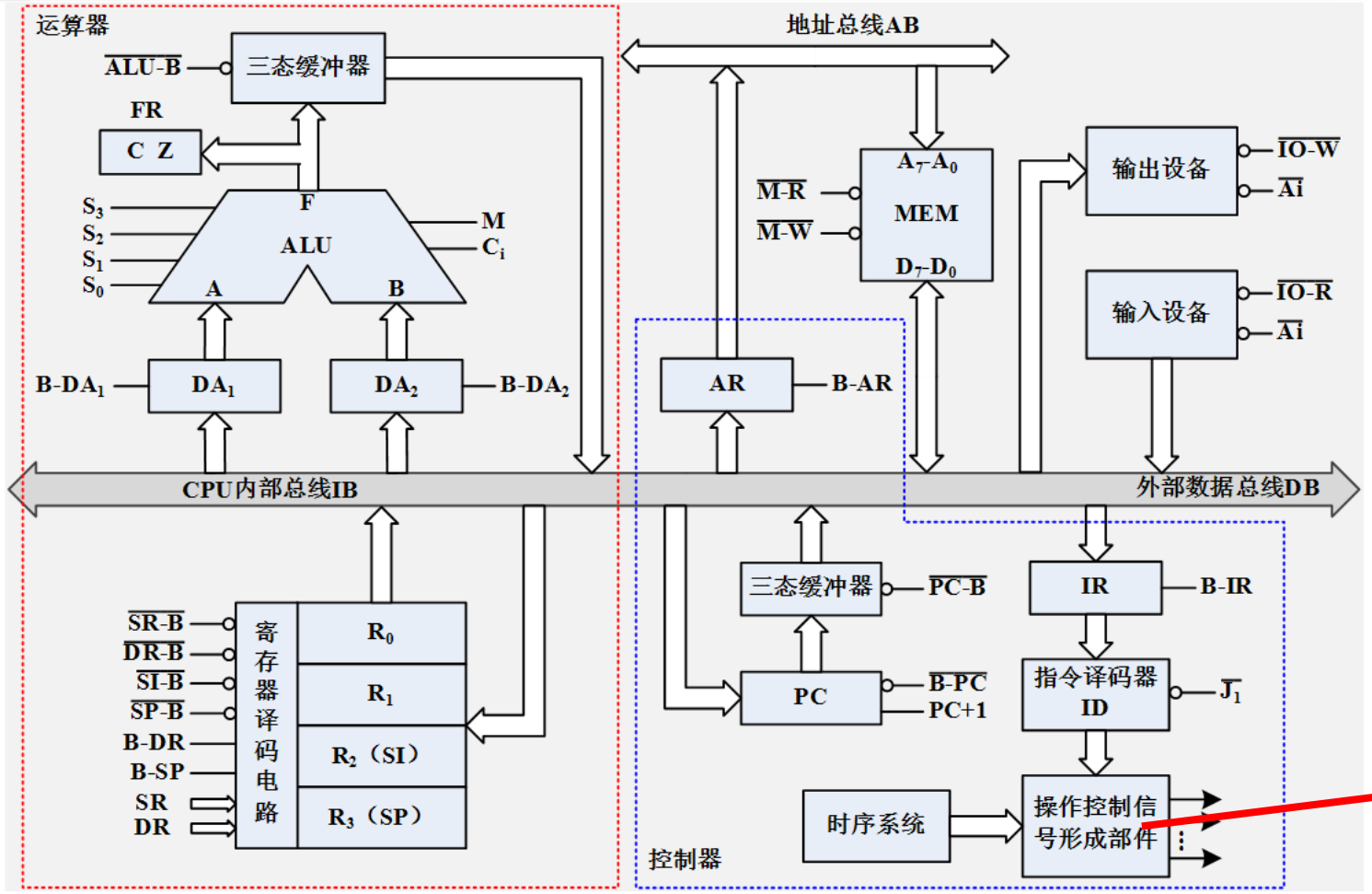
微指令编码	微地址	微指令	位序	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
		空操作		1	0	1	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	1	1	1	1				
6807CF9H	000	M_0		0	1			1																				1	0	0	1
B007C70H	001	M_1					1																	0				0	×	×	×
6807CFFH	010	$JMP \cdot M_2$		0	1			1																				1	1	1	1
A407C7CH	011	$ADD \cdot M_3$							1															0				1	1	0	0
A206CFDH	100	$ADD \cdot M_4$								1									0									1	1	0	1
A12BEF8H	101	$ADD \cdot M_5$									1	0	0	1	0	1	0					1						1	0	0	0
6807CFBH	110	$ADD \cdot M_2$		0	1			1																				1	0	1	1
8007C78H	111	$JMP \cdot M_3$				0																		0				1	0	0	0

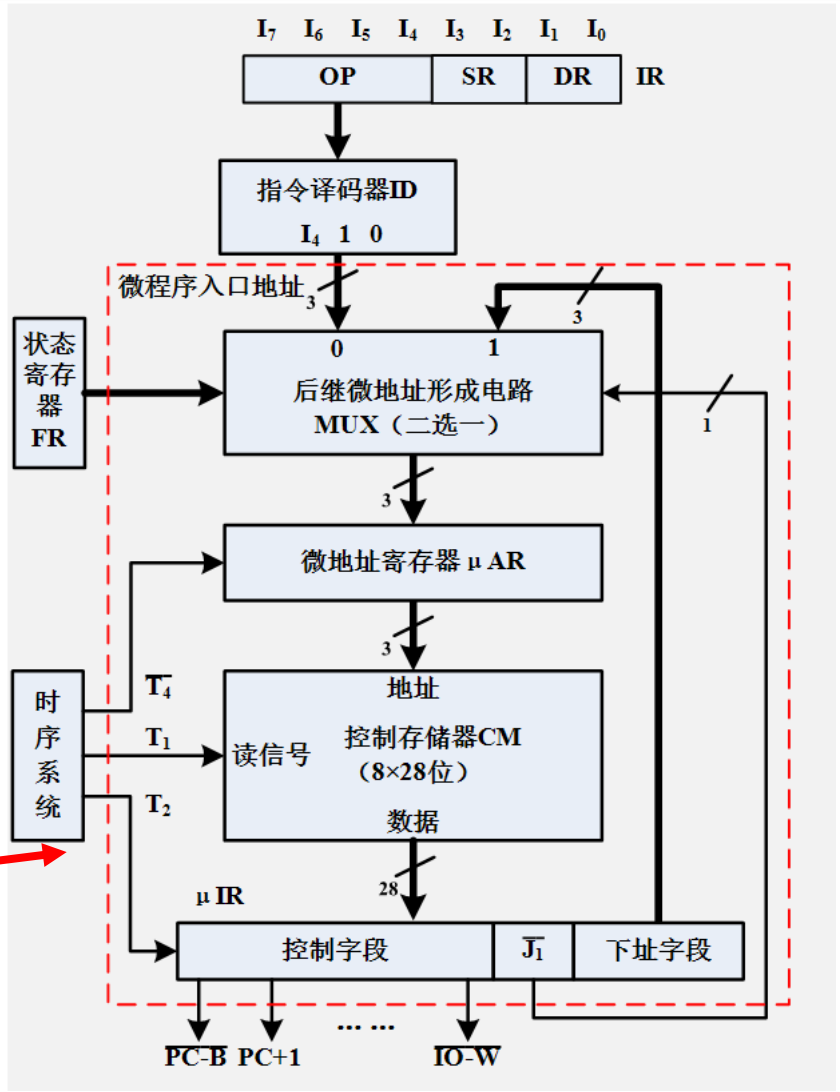
7.4.3微程序控制器的设计方法

❖7、微指令代码装入控制存储器

微地址	微指令编码	微地址	微指令编码
0H	6807CF9H	4H	A206CFDH
1H	B007C70H	5H	A12BEF8H
2H	6807CFFH	6H	6807CFBH
3H	A407C7CH	7H	8007C78H

微程序控制的Yy-z02模型机工作过程





- ①开机上电产生复位信号
  - PC=第一条指令的地址
  - μAR=000（取指令微程序段的入口地址）
  - 复位信号消失后，时序系统产生周期性的T_1~T_4节拍信号

微程序控制的Yy-z02模型机工作过程

- ②第一组T_1 ~T_4节拍：第一个微周期——取并执行第一条微指令（PC→AR，PC+1→PC）
  - T_1 节拍：读CM的微指令，CM[μAR]=CM[000]=6807CF9H
  - T_2 上跳沿：微指令6807CF9H打入μIR，控制字段24个控制信号送出，只有$\overline{PC-B}$、$B-AR$、$PC+1$三个信号是有效的，且一直保存到μIR被更新
    - T_3节拍：$B-AR$信号动作
    - T_4节拍：$PC+1$信号动作
    - T_4下跳沿：$\overline{J_1}$=1，下址字段（=001）置入μAR
- ③第二组T_1 ~T_4节拍：第二个微周期——取并执行第二条微指令（Mem[AR]→IR，指令译码）
  - T_1 节拍：读CM的微指令，CM[001]=B007C70H
  - T_2 上跳沿：微指令B007C70H打入μIR，高24位只有$\overline{M-R}$=0、$B-IR$=1、$\overline{J_1}$=0三个信号有效
  - T_3节拍：$B-IR$信号动作
  - T_4下跳沿：$\overline{J_1}$=0，指令译码器输出的微指令入口地址置入μAR；
    - 如果取到的指令是ADD，则110→μAR
    - 如果取到的指令是JMP，则010→μAR
- ④第三组T_1~T_4节拍：完成第一条指令的微程序段的第一条微指令的读取、执行、更新后继微地址
- 循环往复，直到第一条指令的微程序段的最后一条微指令执行完毕，则一定转入取指令微程序入口（μAR=000），开始取下一条指令
- 每个微周期：
  - 取微指令
  - 发送微命令（执行）
  - 产生后继微地址

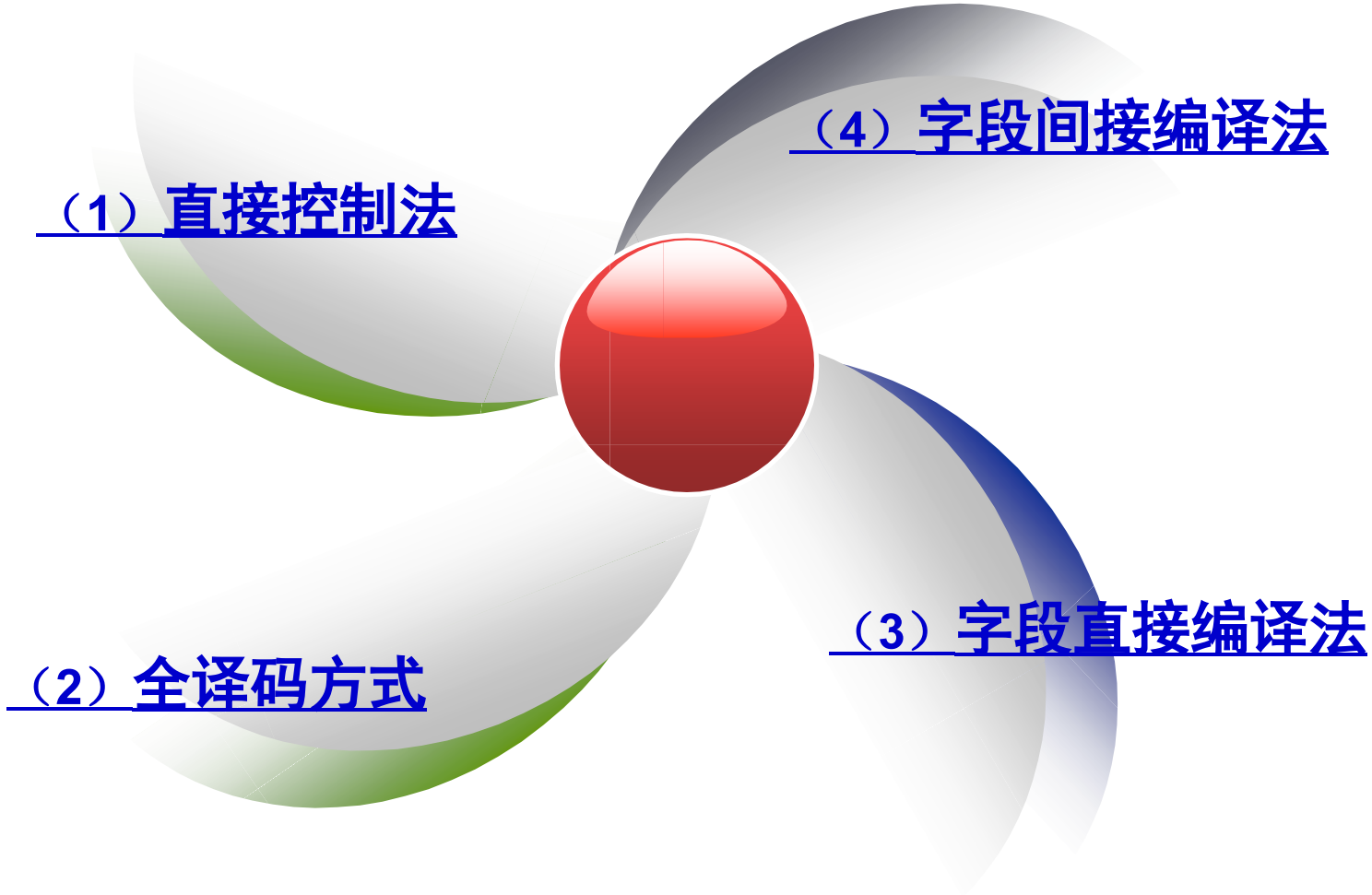
7.4.4 微程序设计技术

1. 控制字段的编码方式
2. 下址字段设计方法
3. 微程序入口地址的产生方法
4. 微指令格式的类型
5. 毫微程序设计
6. 动态微程序设计



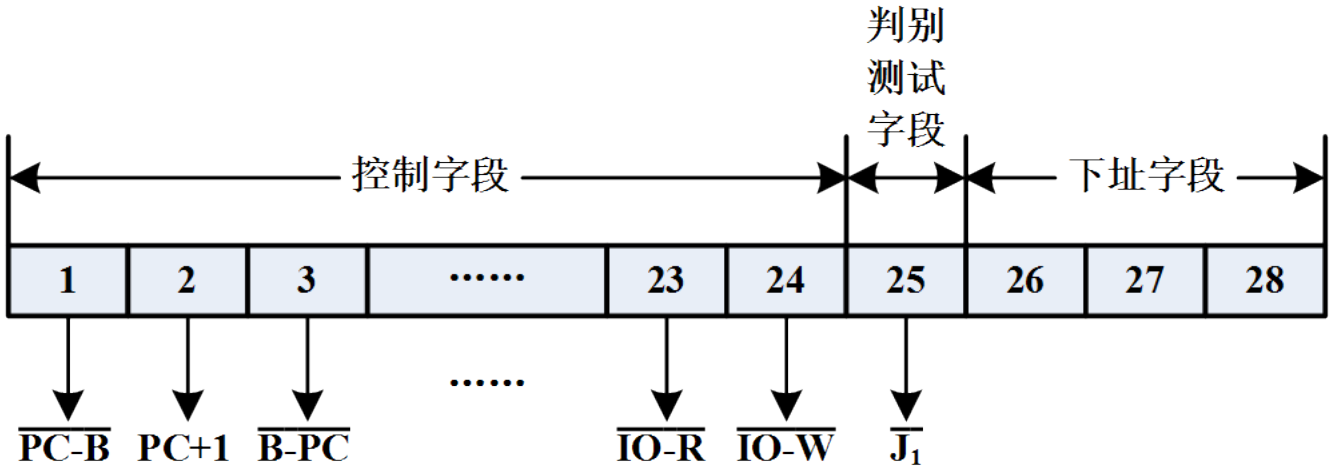
- 研究与应用微程序设计技术的目的：
  - ① 缩短微指令字长
  - ② 减少控制存储器的容量
  - ③ 加快微程序的执行速度
  - ④ 便于微程序的修改与扩充
  - ⑤ 提高微程序设计的灵活性
- 微指令由两部分构成：
  - 控制字段
  - 下址字段
- 如何编码与设计？
- 不同的应用？

1、控制字段的编码方式



（1）直接控制法

- 控制字段的编码方法：每一位代表一个微命令（控制信号）
- 执行微指令时：控制字段的各位控制信号，直接连接到部件
- 编写微指令方法：直接控制
  - 如果要发送某个微命令：将控制字段中对应位，置有效电平
  - 如果不要发出某个微命令：将控制字段中对应位，置无效电平
- 优点：无需译码，执行速度快；微程序较短
- 缺点：微指令字长很长，占用控存容量大



（2）全译码方式

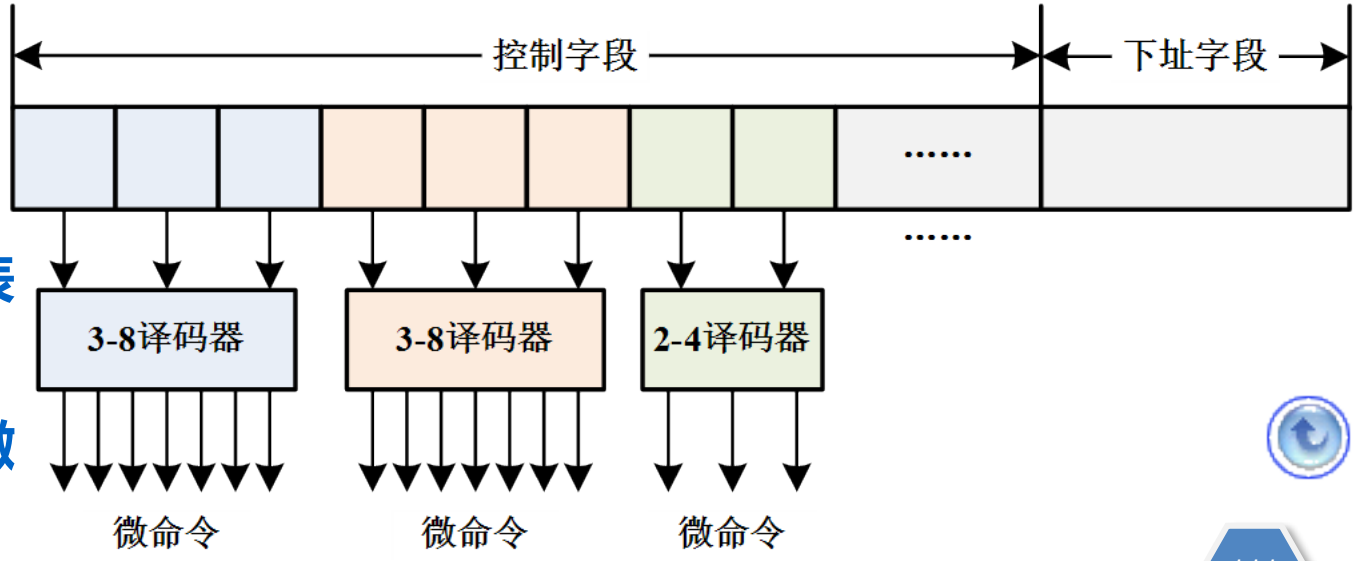
- 控制字段的编码方法：将所有的控制信号进行二进制编码
- 执行微指令时：译码产生各个微命令
  - 每条微指令只能发送1～2个微命令
- 优点：微指令字长很短
- 缺点：并行操作能力弱，微程序很长，执行速度慢
- 一般用于垂直微指令格式

（3）字段直接编译法

- 控制字段的编码方法：将控制字段分成若干小段，每小段通过编码-译码方式，对应到各个控制信号。
- 优点：并行操作能力较强，字长较短
- 遵循的分段原则（选择将哪些微命令放在一组进行小字段编码）：
  - 相斥性微命令分在同一字段内：可以提高信息位的利用率，缩短微指令字长
  - 相容性微命令分在不同字段内：有利于实现并行操作，加快指令的执行速度

指在同一个微周期中不可能同时出现的微命令

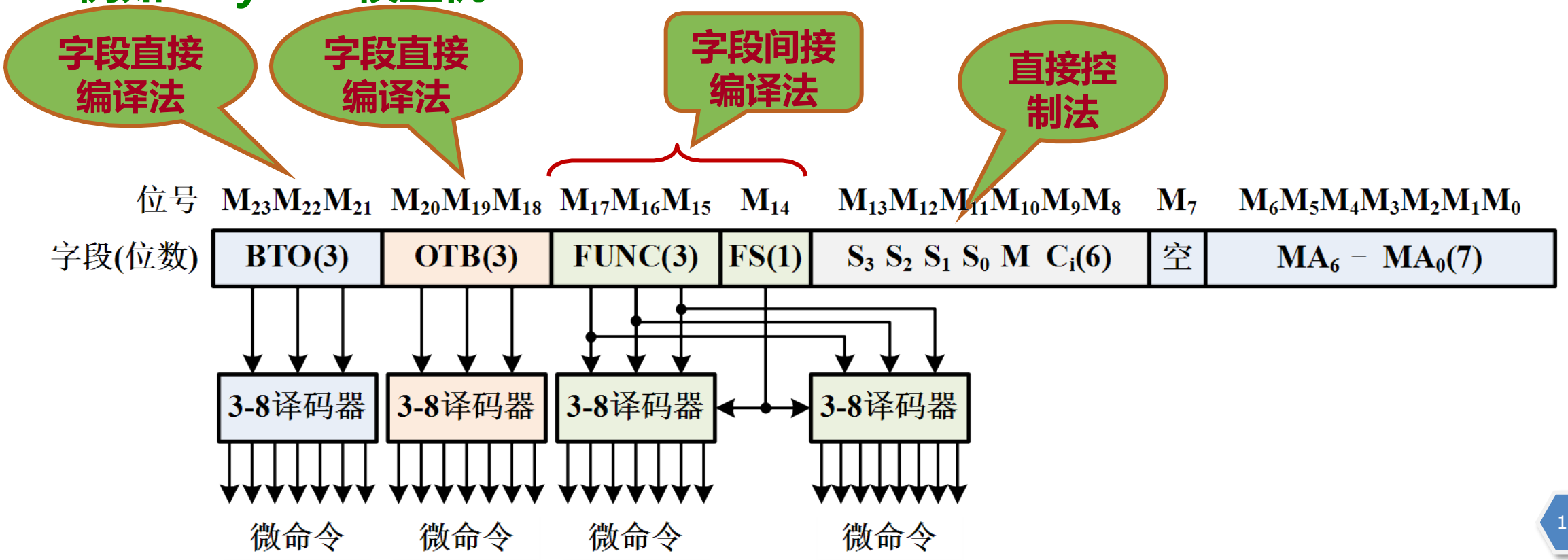
指在同一个微周期中可以同时出现的微命令



- 每个小字段要保留一个编码作为空操作：表示本组所有控制信号都不发送
- 将同类操作的微命令划分在一个字段内：微指令结构清晰、规整，易于编制微程序和扩充功能

（4）字段间接编译法

- 控制字段的编码方法：某字段的编码含意，需要由另一字段来加以解释。
- 某一字段所产生的微命令，是和另一字段的代码联合定义出来的。
- 4种控制字段编码方法，可以混合应用在微指令的格式设计中。
- 例如：Yy-z02模型机



Yy-z02模型机实验系统的微指令字段编码表

字段直接编译法

字段直接编译法

字段间接编译法

编码	BTO	OTB	FS=1	FS=0
			FUNC	FUNC
000	空操作	空操作	$PC+1(T_4)$	空操作
001	$B-DA_1(T_4)$	$\overline{ALU-B}(T_2)$	$\overline{J_1}(T_4)$	$\overline{M-W}(T_3)$
010	$B-DA_2(T_4)$	$\overline{299-B}(T_2)$	$\overline{J_2}(T_4)$	$\overline{M-R}(T_2)$
011	$B-IR(T_3)$	$\overline{SR-B}(T_2)$	$\overline{J_3}(T_4)$	$\overline{IO-W}(T_3)$
100	$B-DR(T_4)$	$\overline{DR-B}(T_2)$	$\overline{J_4}(T_4)$	$\overline{IO-R}(T_2)$
101	$B-SP(T_4)$	$\overline{SI-B}(T_2)$	$\overline{J_5}(T_4)$	$\overline{INT-R}(T_2)$
110	$B-AR(T_3)$	$\overline{SP-B}(T_2)$	$\overline{C_yC_n}(T_2)$	$\overline{INT-E}(T_3)$
111	$\overline{B-PC}(T_4)$	$\overline{PC-B}(T_2)$	$\overline{C_yNC_n}(T_2)$	

例题

- 【例7.5】如果某计算机采用微程序控制器，全机有50个微操作控制信号；微程序转移的方式有5种；假设下址字段有8位，能转移到控存的全部空间；判别测试字段采用译码方法，则：
  - （1）如果控制字段采用直接控制法，微指令字长几位？控制存储器的容量多少位?
  - （2）如果控制字段采用字段直接编译法，50个控制信号被分为6组，各组分别有6个、8个、13个、15个、3个、5个控制信号，则其微指令字长几位？控制存储器容量多少位?

控制字段	判别测试字段	下址字段
?	?	8位

例题

- （1）如果控制字段采用直接控制法，微指令字长几位？控制存储器的容量多少位？

❖ 判别测试字段采用译码方法，5种微程序转移方式 → ❖ 需要3位编码

❖ 下址字段8位且能访问控存全部空间 → ❖ 控存有2^8=256个单元

❖ （1）控制字段采用直接控制法，50个控制信号 → ❖ 需要50位编码

- 微指令字长=50+3+8=61位
- 控制存储器容量：256×61位

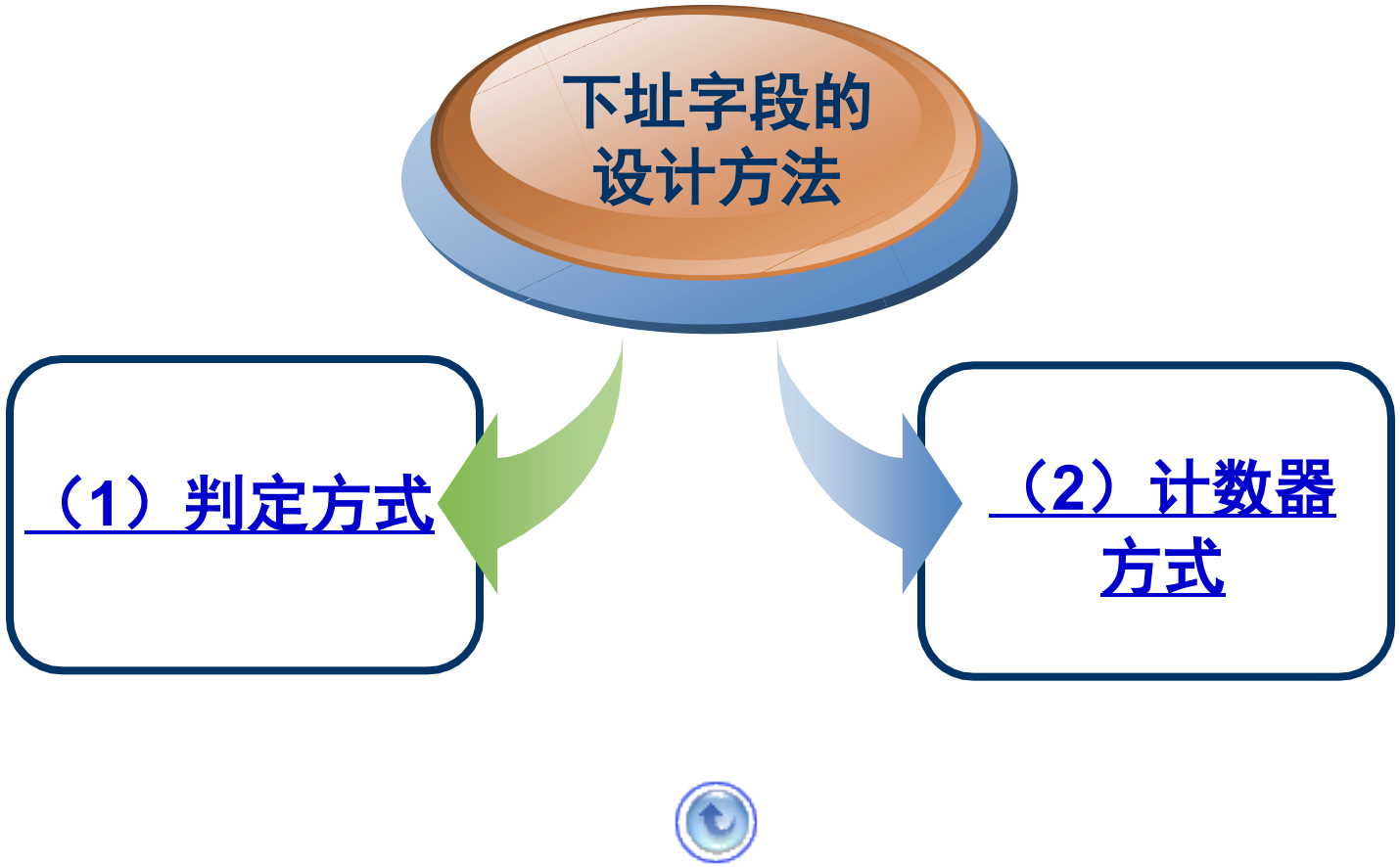
控制字段	判别测试字段	下址字段
50位	3位	8位

- （2）控制字段采用字段直接编译法，50个控制信号分为6组：
- 6个、8个、13个、15个、3个、5个控制信号
- 分别需要：3位、4位、4位、4位、2位、3位编码，合计控制字段20位
- 微指令字长=20+3+8=31位
- 控制存储器容量：256×31位

控制字段	判别测试字段	下址字段
20位	3	8位

2、下址字段的设计方法

❖ 微指令的下址字段：用于产生后继微地址，实现微指令的定序



（1）判定方式

❖ 微指令结构：

控制字段	判别测试字段	下址字段

- 控制字段
- 判别测试字段：转移控制字段，指出转移的类型和条件
- 下址字段：指出转移微地址的信息

❖ 当前微指令的后继微地址：由判别测试字段和下址字段共同决定

❖ 非常灵活，可以有各种变化：

- 下址字段：全地址（控存全局转移） or 部分地址（控存局部转移）
- 转移微地址：通过逻辑电路生成 or 在下址字段基础上修改，有时还需要状态条件参与

❖ 优点：每一条微指令都能实现微程序的转移，可以实现快速多路分支，执行速度快，而且微程序在控制存储器中不必连续存储，微程序设计灵活

❖ 缺点：形成后继微地址的电路结构比较复杂，微指令字长也比较长。

（2）计数器方式

- 在微程序控制器中设置一个微程序计数器μPC；由μPC来提供后继微地址：
  - 在顺序执行微指令时，μPC自动+1
  - 遇到转移微指令时，由微指令给出转移微地址，置入μPC
- 微指令的格式有两种：

非转移类的微指令格式



- 操作控制：只发送微操作控制信号
- 顺序控制：后继微地址=μPC+1

转移类的微指令格式



- 顺序控制：后继微地址，由转移微地址和转移类型共同决定

- 优点：微指令字较短，便于编写微程序，后继微地址产生电路比较简单；
- 缺点：微程序较长，执行速度相对较慢

3、微程序入口地址的产生方法

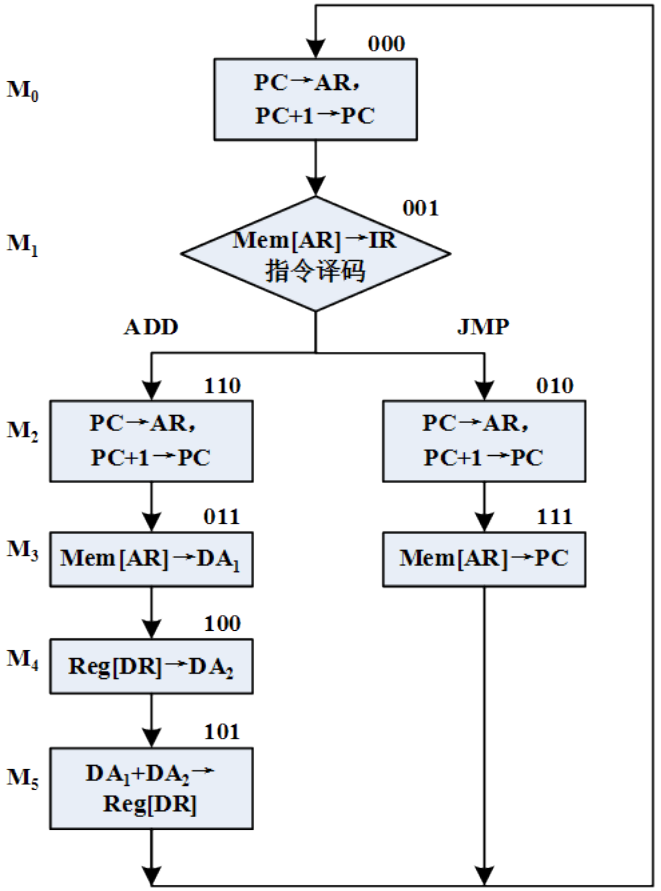
- 微程序入口地址：当前指令对应的微程序段的第一条微指令在控存中的地址
  - 是后继微地址的必不可少的一个来源
  - 由指令译码器产生，微程序控制器的指令译码器的功能是：根据指令操作码，产生各指令的微程序入口地址
- 实现方法：

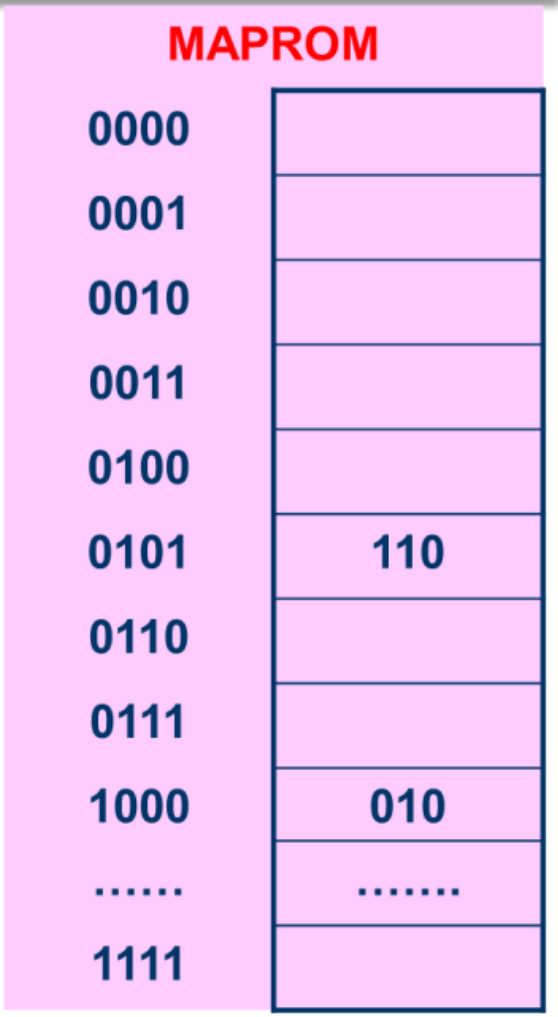
- （1）映射存储器（MAPROM）
  - 属于存储（时序）逻辑电路
  - 地址：机器指令的操作码
  - 数据：该指令对应的微程序入口地址

- （2）逻辑电路
  - 属于组合逻辑电路
  - 输入：机器指令的操作码
  - 输出：该指令对应微程序入口地址

（1）映射存储器（MAPROM）方法

- **MAPROM：可编程存储器PROM**
  - 地址N：指令I_N的操作码OP
  - 存储单元N中的数据：为指令I_N的微程序入口地址
- **指令译码：将IR中的OP字段作为地址，对MAPROM执行读操作，读出的数据就是该指令对应的微程序入口地址**





- **例：**
  - ADD：操作码=0101，入口地址=110
    - MAPROM：5号单元地址中写入 6
  - JMP：操作码=1000，入口地址=010
    - MAPROM：8号单元地址中写入 2

（1）映射存储器（MAPROM）方法

- 【例7.6】 如果一个指令系统，其指令格式中，操作码固定为6位；平均每条指令的微程序段含有5条微指令，另有取指令微程序段含有3条微指令：
  - （1）控制存储器包含多少个单元?
  - （2）如果采用MAPROM的方法产生每条指令的微程序段入口地址，则MAPROM容量是多少位?

- （1）操作码固定为6位：指令系统有 $2^6=64$ 条机器指令
  - 取指令微程序段：3条微指令
  - 各指令的微程序段： 64×5=320 条微指令
  - 控制存储器：有3+320=323个单元

地址几位?

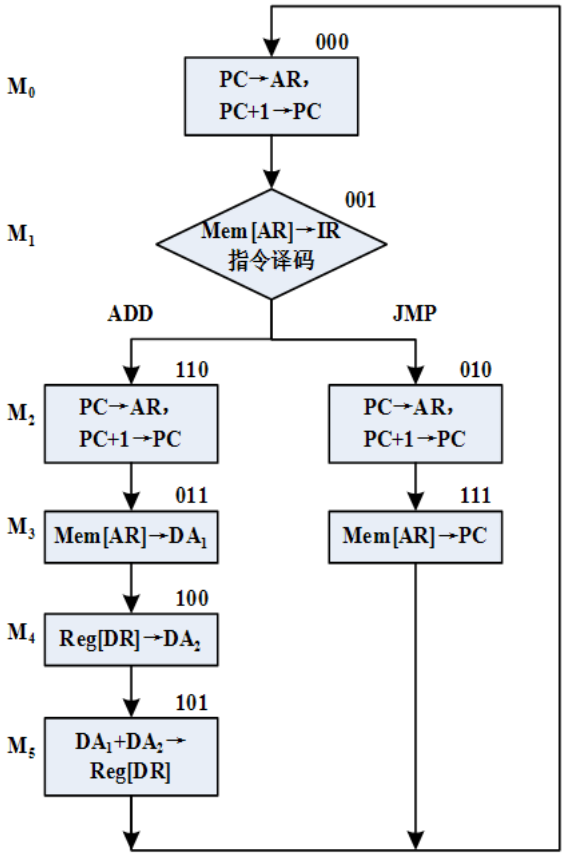
- （2）MAPROM
  - 地址：指令的操作码，故地址有6位
  - 单元内容：微程序入口地址，即控存的地址， 9 位
  - 容量=2^6×9位=64×9位

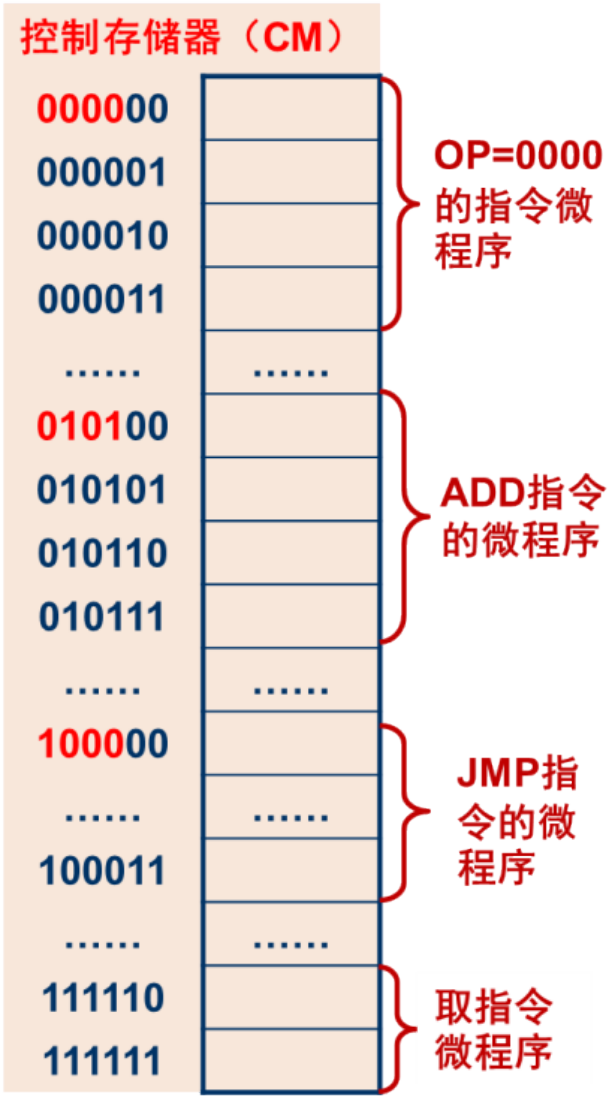
（2）逻辑电路方法

- 使用组合逻辑电路来设计实现
  - 电路输入：机器指令的操作码OP
  - 电路输出：指令的微程序入口地址

- 例：最简单的电路
  - 输入：指令操作码OP=$I_7I_6I_5I_4$；
  - 输出：该指令的微程序入口地址=I_4 10

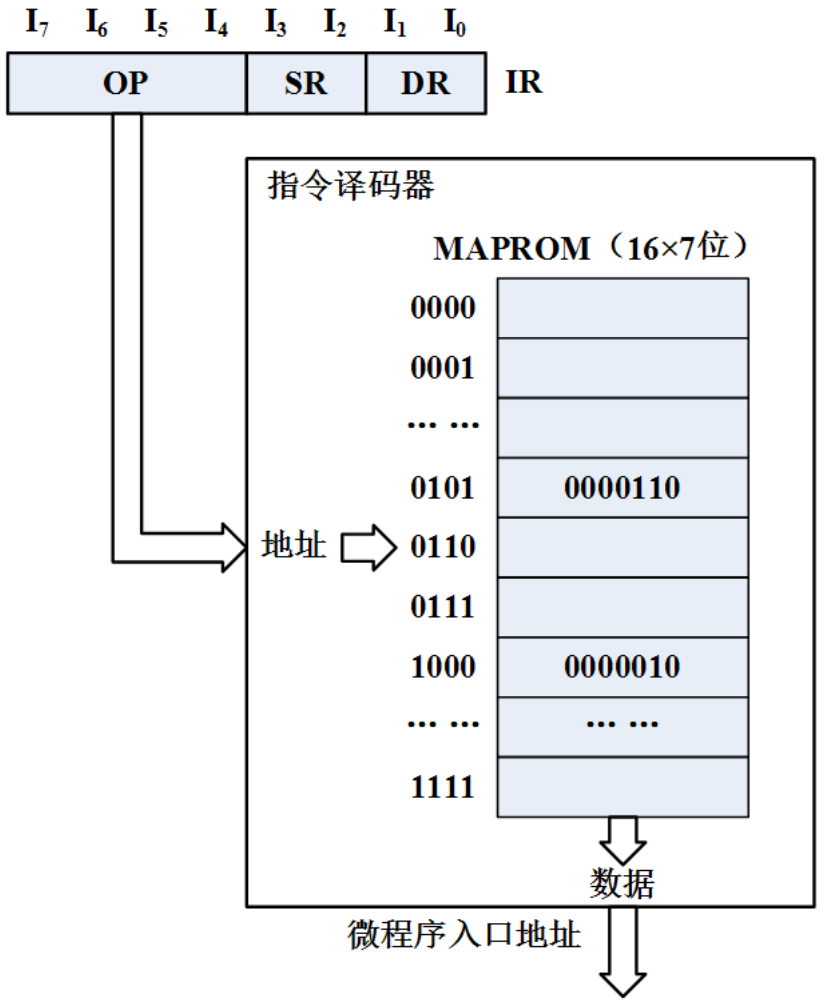
- 只能包含2条指令，如果扩充到16条呢？
  - 输入：指令操作码OP=$I_7I_6I_5I_4$
  - 输出：该指令的微程序入口地址=$I_7I_6I_5I_4$ 00





3、微程序入口地址的产生方法

- 【例7.7】为7.4.3节的简单微程序控制器重新设计指令译码器，假设指令格式仍旧采用高4位为操作码OP，容纳16条机器指令，每条机器指令对应的微程序段最多8条微指令，则：
  - （1）使用MAPROM方法设计指令译码器。
  - （2）使用逻辑电路方法设计指令译码器。

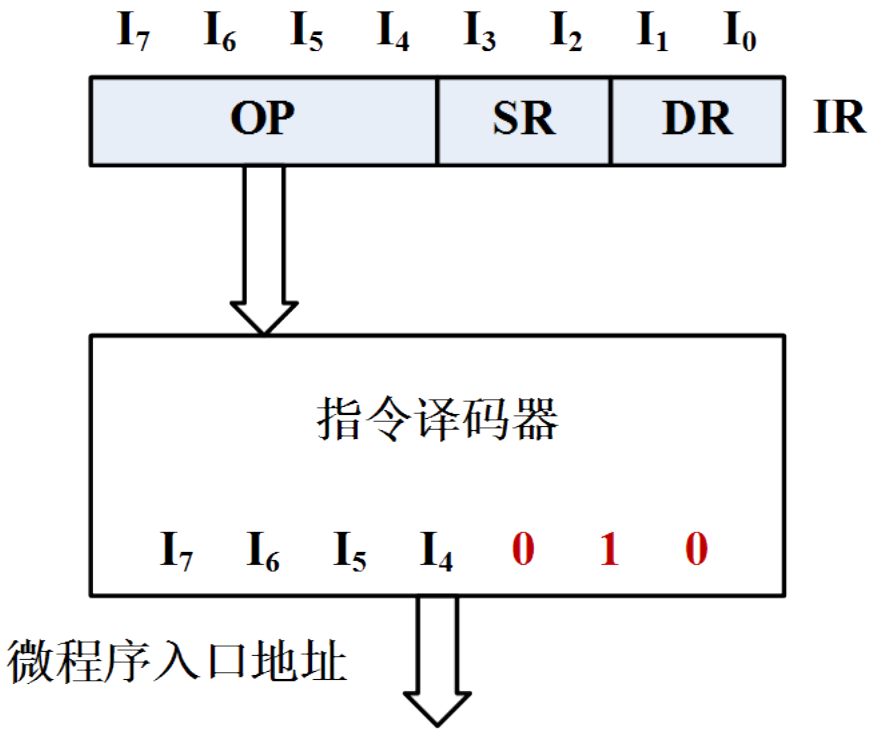


- （1）映射存储器（MAPROM）
  - 地址：机器指令的操作码，4位
  - 数据：该指令对应的微程序入口地址，7位
    - 控存：16×8=128个单元，7位地址

- 假设ADD和JMP指令的微程序入口仍旧是6号和2号地址
- ADD指令操作码=0101B=5，则05H单元要存放数据06H
- JMP指令操作码=1000B=8，则08H单元要存放数据02H

3、微程序入口地址的产生方法

- （2）逻辑电路
  - 输入：机器指令的操作码，4位OP
  - 输出：该指令对应微程序入口地址={OP，010}



- 取指令的微程序段：00H和01H单元
- OP=0000B的指令：微程序入口为0000 010B=02H
- ADD指令（OP=0101B）：微程序入口地址是0101 010B=2AH
- JMP指令（OP=1000B）：微程序入口地址是1000 010B=42H

两种方法对比

- MAPROM方法：
  - 优点：非常灵活、不受限制，便于自由安排指令的微程序在控存中的位置
  - 缺点：需要访存才能得到微程序入口，速度慢
- 逻辑电路方式：
  - 优点：能快速地产生微程序入口
  - 缺点：微程序入口是指令OP的逻辑函数，故有一定的限制，即：某条指令的微程序入口一定在某个控存地址



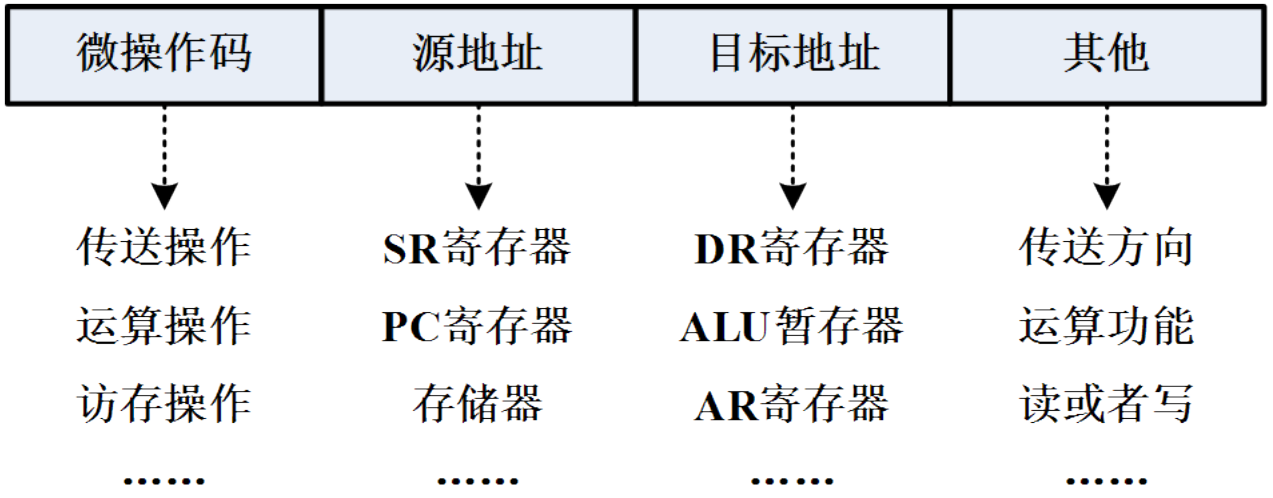
4、微指令格式的类型

- （1）水平型微指令
- 基本特征：一条微指令能定义并发送多个微命令，控制数据通路中多个功能部件并行操作。
- 控制字段：采用直接控制法、字段直接编译法、字段间接编译法的微指令格式，都属于水平型的微指令。

- 优点：一条微指令可同时发许多个微命令，微指令执行效率高、速度快、灵活，各部件执行操作的并行能力强；编制的微程序比较短。
- 缺点：微指令字太长，增加了控制存储器的横向容量。

4、微指令格式的类型

- （2）垂直型微指令
- 基本特征：一条微指令只能控制1～2种微操作；
- 控制字段：全译码方式，编码为微操作码字段μOP，由μOP规定微指令功能
- 垂直型微指令的格式：

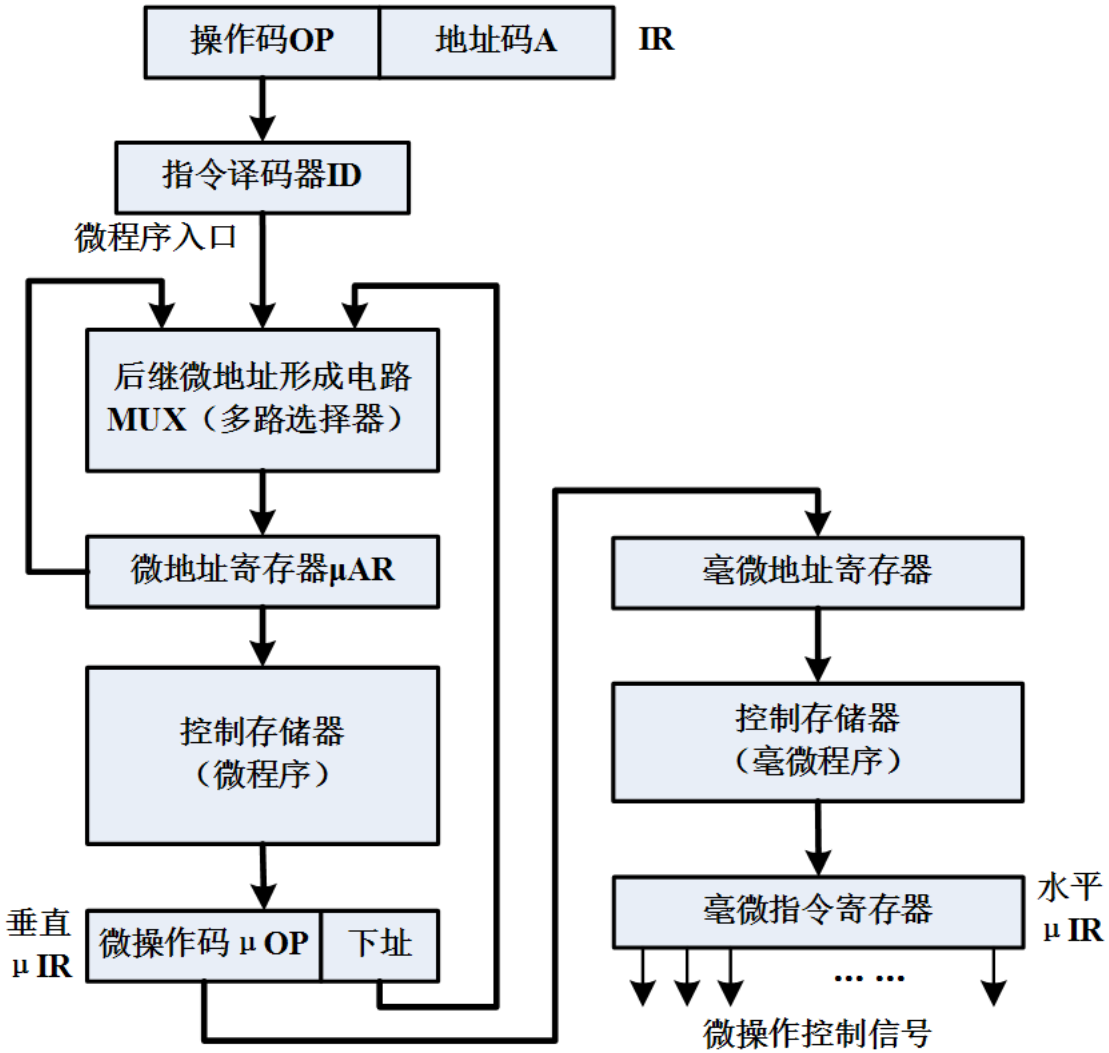


- 优点：比较直观，容易掌握和便于使用；微指令字短，减少了横向控制存储器的容量
- 缺点：微指令要经过译码才能发出微命令，微指令的执行效率低，并行操作性比较差，增加了纵向微程序容量。

5、毫微程序设计

❖两级微程序：垂直型+水平型微指令

- 第一级：垂直微程序/微程序，用来解释机器指令
  - 存放在微程序存储器中（一级控存）
- 第二级：水平微程序/毫微程序，用来解释垂直微指令，并产生相应微命令，实现数据通路的控制
  - 存放在毫微程序存储器中（二级控存）



❖ 优点：使得微程序流的控制（定序）和微命令发出完全分离

- 微程序流控制：微程序级实现（垂直型微指令）
- 微命令发出：毫微程序产生（水平型微指令）

6、动态微程序设计

- 控制存储器：一般由ROM构成
  - 指令系统是固定的，微程序用于解释执行指令
  - 故：微程序也是固定的，所以使用只读存储器来存放微程序

- 动态微程序设计：在一台微程序控制的计算机中，假如能根据用户的要求改变微程序，那么这台机器就具有动态微程序设计功能
  - 要求：控制器的CM必须是可改写的存储器，如RAM或者E^2PROM。
  - 功用：可以通过修改微程序来实现不同的指令系统，或者实现指令系统的扩充或调整
  - 目的：使计算机能更灵活、更有效地适应于各种不同的应用场合



7.4.5 微程序控制器与硬布线控制器的比较

比较内容	微程序控制器	硬布线控制器
工作原理	微操作控制信号事先以微程序的形式存放在控存中，执行指令时读出即可	微操作控制信号由组合逻辑电路根据当前的指令码、状态和时序，即时产生
执行速度	慢	快
规整性	较规整	繁琐、不规整
应用场合	CISC CPU	RISC CPU
易扩充性	易扩充修改	困难

本章小结

- **中央处理器CPU**：包含运算器和控制器，它是整个计算机系统的核心部件，具有指令控制、操作控制、时间控制、数据加工等功能
- **控制器**：计算机硬件五大部件的控制中心，根据机器指令来产生指令执行时全机所需要的操作控制信号，协调控制计算机各个部件有序工作。
- **控制器的功能**：取指令、分析指令、执行指令、进行中断处理和响应特殊请求
- **控制器的组成**：
  - **专用寄存器**：PC(存放机器指令在主存中的地址)、IR(存放当前机器指令的指令码)、AR(存放访问存储器的单元地址或者访问IO设备的端口地址)、DR(存放访问存储器或者IO设备的数据)
  - **指令译码器ID**：对IR中的操作码字段进行译码，以识别当前指令是哪一条指令
  - **时序系统**：提供时钟基准和各级时序信号，以规定每个操作控制信号的动作时间
  - **操作控制信号形成部件（控制单元）**：产生取指令、分析指令和执行这条指令所需的操作控制信号序列

本章小结

- **指令周期：从存储器取出一条指令并执行完这条指令所需的时间**
  - 不同指令的指令周期（动作及所需时间）是不相同的
  - 指令周期：划分为若干个**机器周期**(CPU周期)：分阶段完成取指令、分析指令和执行指令
  - **机器周期**：也可能划分为若干个时钟周期或节拍
  - **时钟周期**：一般就是CPU主频脉冲的周期
- **时序系统：通常按照机器指令的需要，产生机器周期信号、节拍信号和工作脉冲三级时序信号，对控制信号（动作）进行定序。**
- **单周期CPU：每条指令都在一个时钟周期内完成**
- **多周期CPU：每条指令都需要多个时钟周期才能完成**
- **流水线CPU：将各个部件按顺序组成一条流水线，指令按序进入流水线，经各个部件顺序处理后流出流水线，指令的执行虽然也需要多个时钟周期，但是指令并行执行，大大提高了整体执行速度**

本章小结

- 根据操作控制信号形成部件设计方法及组成的不同，将控制器分成两种类型：
  - 硬布线控制器：用组合逻辑电路来产生控制信号，电路的输入是机器指令译码信号、各级时序信号和状态条件，输出是全机所有的微操作控制信号；硬布线控制器速度快，但设计复杂繁琐，适合于RISC结构
  - 微程序控制器：采用存储逻辑电路来产生控制信号
    - 基本思想是每条机器指令都用一段微程序来解释执行，微程序是微指令的有序集合，事先编写好存放在控制存储器中；微指令包含一组微命令（微操作控制信号）
    - 执行机器指令时从控制存储器中依次取出微指令，由微指令发送机器指令所需要的微操作控制信号，从而完成机器指令的执行
    - 微程序控制器执行速度慢，但设计比较规整，易于实现指令系统修改与扩充，适合于CISC结构
  - 微程序控制器的组成和基本概念
  - 微程序设计技术





The End !